Template List

- System Architecture — Template Set (Locked v1)
- Architecture & Contracts (ARC)
  ARC-01 System Architecture (domains/boundaries)
  ARC-02 Core Contracts Overview (APIs, events, entities, shared models)
  ARC-03 Key Workflow Data Flows (end-to-end request/event paths)
  ARC-04 Authentication Architecture (sessions/JWT/SSO boundaries)
  ARC-05 Realtime Architecture (channels, presence, delivery, scaling)
  ARC-06 Error Model + Reason Codes (canonical failure taxonomy)
  ARC-07 Integration Architecture (3rd-party boundaries + trust model)
  ARC-08 Deployment Topology (services, environments, network boundaries)
  ARC-09 Cross-Cutting Concerns (logging, tracing, rate limits, caching touchpoints)
  ARC-10 Architecture Constraints & Invariants (must-hold rules + enforcement points)
- System Interfaces & Integration Contracts (SIC)
  SIC-01 External Interface Inventory (systems, purpose, direction)
  SIC-02 Contract Spec (per interface: requests/responses/events/auth/errors)
  SIC-03 Webhook Contract Spec (producer/consumer, retries, signatures)
  SIC-04 Data Mapping Contract (field mappings + transforms + validation)
  SIC-05 Integration Failure Modes & Recovery (timeouts, retries, DLQ, fallbacks)
  SIC-06 Vendor/Third-Party Trust Model (data sharing, scopes, audit needs)
- Service Boundaries & Deployment Topology (SBDT)
  SBDT-01 Service Boundary Map (responsibilities, ownership, dependencies)
  SBDT-02 Runtime Topology (services, workers, queues, storage, networks)
  SBDT-03 Environment Topology (dev/stage/prod parity, isolation rules)
  SBDT-04 Scaling Model (horizontal/vertical, bottlenecks, capacity assumptions)
  SBDT-05 Resilience Topology (redundancy, failover, circuit boundaries)
  SBDT-06 Deployment Constraints (rollouts, canary, migration safety)
- Permission Model & Authorization Design (PMAD)
  PMAD-01 Permission Model Overview (roles, resources, actions)
  PMAD-02 AuthZ Policy Rules (RBAC/ABAC, inheritance, exceptions)
  PMAD-03 Enforcement Points Map (UI/API/service/DB)
  PMAD-04 Permission Check Patterns (standard decision flow + reason codes)
  PMAD-05 Privileged Operations Policy (admin/mod/support actions, approvals)
  PMAD-06 Audit Requirements for AuthZ (what must be logged)
- Error Model & Reason Codes (ERR)
  ERR-01 Error Taxonomy (classes, categories, severity)
  ERR-02 Reason Codes Registry (rc_* catalog + meanings)
  ERR-03 API Error Contract (shape, status mapping, localization hooks)
  ERR-04 UX Error Mapping Rules (reason_code → DES/CDX surfaces)
  ERR-05 Retryability & Idempotency Rules (which errors retry, how)
  ERR-06 Observability Requirements for Errors (logs/metrics/traces fields)
- Realtime & Messaging Architecture (RTM)
  RTM-01 Realtime Use Cases Catalog (presence, chat, streams, updates)
  RTM-02 Protocol & Transport Map (WS/WebRTC/pubsub, fallback rules)

RTM-03 Channel/Topic Model (naming, scoping, permissions)
RTM-04 Delivery Semantics (ordering, dedupe, at-least-once, ack)
RTM-05 Presence & State Sync Model (source of truth, TTLs, conflicts)
RTM-06 Abuse/Rate Control for Realtime (limits, moderation hooks)

- Workflow & Orchestration Design (WFO)
  WFO-01 Workflow Catalog (sagas/jobs/background tasks by ID)
  WFO-02 Orchestration Patterns (state machine, retries, compensation)
  WFO-03 Idempotency & Concurrency Model (keys, locking, ordering)
  WFO-04 Scheduling & Triggers (cron/events/user actions)
  WFO-05 Failure Handling (DLQ, backoff, poison messages, alerts)
  WFO-06 Workflow Observability (traceability, audit events, metrics)
- API Governance & Versioning (APIG)
  APIG-01 API Standards (naming, pagination, filtering, consistency)
  APIG-02 Versioning Policy (v1/v2 rules, compat guarantees)
  APIG-03 Deprecation & Sunset Policy (timelines, comms, redirects)
  APIG-04 Review Gate Checklist (what must be true before shipping APIs)
  APIG-05 Compatibility Test Requirements (contract tests, schema checks)
  APIG-06 Schema Evolution Rules (backward compatible changes, migrations)
-

System Architecture (Definition)

What it is
The System Architecture group defines the **structural and contractual design** of the system: boundaries, interfaces, trust model, failure model, and the rules that make the system coherent under change. It translates Product + Experience requirements into **architecture-authoritative contracts** that implementation can build against without guessing.

What it produces

- **Service/domain boundaries** and responsibilities (what lives where)
- **Interface contracts** (API/event/message/integration) and compatibility rules
- **Authorization/permission model** (who can do what, where enforced)
- **Error model + reason codes** (predictable failure behavior + UX mapping support)
- **Realtime and messaging architecture** (channels, presence, delivery semantics, scaling constraints)
- **Workflow/orchestration design** (sagas, jobs, retries, idempotency boundaries)

- **API governance/versioning rules** (stability, deprecation, backward compatibility)
- **Deployment topology** (environments, service placement, runtime boundaries)

What it is used for (exactly)

- Turning features/flows into **implementable contracts** (feeds API/DATA/INT/OPS/SEC)
- Ensuring frontend/mobile can rely on **stable behavior** (auth, errors, realtime)
- Enabling deterministic mapping for build docs (MAP-02, MAP-03)
- Providing enforceable rules for **gates** (security, release readiness, compatibility)

What it is not

- Not implementation tasks, code structure, or repo work (that's IMP / Group 5)
- Not UX/UI behavior specification (that's Group 2)
- Not the full data schema/migration plan (that's Group 4 DATA family)
- Not standards for the AXION system itself—these are **agent-kit build artifacts**

Scope rules

- Must reference canonical truth from Group 1 (PRD/BRP/DMG/RISK) and Group 2 (DES/IAN)
- Must produce **testable contracts** (inputs/outputs, constraints, invariants, failure behavior)
- Must clearly separate **logical design** (contracts) from **physical build steps** (implementation planning)

Subcategory meaning (one-line each)

- **ARC**: High-level system architecture + core contracts (domains, interfaces, deployment shape)
- **SIC**: Integration-facing contracts (3rd parties, webhooks, external boundaries)

- **SBDT**: Service boundaries + deployment topology (runtime and environment separation)
- **PMAD**: Permission and authorization design (RBAC/ABAC, enforcement points)
- **ERR**: Error model and reason codes (canonical failure taxonomy + mapping)
- **RTM**: Realtime/messaging architecture (WS/WebRTC/pubsub, delivery semantics)
- **WFO**: Workflow/orchestration design (jobs, sagas, retries, idempotency)
- **APIG**: API governance + versioning (compat rules, deprecation, review gates)

Architecture & Contracts (ARC)

- Architecture & Contracts (ARC)
  ARC-01 System Architecture (domains/boundaries)
  ARC-02 Core Contracts Overview (APIs, events, entities, shared models)
  ARC-03 Key Workflow Data Flows (end-to-end request/event paths)
  ARC-04 Authentication Architecture (sessions/JWT/SSO boundaries)
  ARC-05 Realtime Architecture (channels, presence, delivery, scaling)
  ARC-06 Error Model + Reason Codes (canonical failure taxonomy)
  ARC-07 Integration Architecture (3rd-party boundaries + trust model)
  ARC-08 Deployment Topology (services, environments, network boundaries)
  ARC-09 Cross-Cutting Concerns (logging, tracing, rate limits, caching touchpoints)
  ARC-10 Architecture Constraints & Invariants (must-hold rules + enforcement points)

ARC-01

ARC-01 — System Architecture (domains/boundaries)

Header Block

- template_id: ARC-01
- title: System Architecture (domains/boundaries)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-01_System_Architecture.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["PRD-01", "PRD-04", "DMG-02", "RISK-02", "DES-01"]
- inputs_required: ["PRD-01", "PRD-04", "DMG-02", "DMG-03", "RISK-02", "DES-01", "STK-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the high-level system architecture: the major domains/boundaries, responsibilities, ownership, and how the system is partitioned. This is the architecture-authoritative map that downstream contracts (API/DATA/RTM/WFO) must align to.

Inputs Required

- PRD-01: {{xref:PRD-01}} | OPTIONAL
- PRD-04: {{xref:PRD-04}} | OPTIONAL
- DES-01: {{xref:DES-01}} | OPTIONAL

- DMG-02: {{xref:DMG-02}} | OPTIONAL
- DMG-03: {{xref:DMG-03}} | OPTIONAL
- RISK-02: {{xref:RISK-02}} | OPTIONAL
- STK-02: {{xref:STK-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Architecture overview (1–2 pages equivalent)
- Domain/boundary list (minimum 4 for non-trivial systems)
- For each boundary:
    - boundary_id
    - name
    - responsibility statement
    - owns (data/entities/events)
    - exposes (interfaces/APIs/messages)
    - depends_on (other boundaries)
    - invariants (must-hold rules)
    - trust boundary classification (internal/external)
    - owner (team/stakeholder)
- System context: actors and external systems (high level)
- Communication patterns between boundaries (sync/async)
- Cross-cutting concerns placement (auth, logging, caching, rate limits)

- Known constraints (hard constraints) and rationale

Optional Fields

- Diagrams (text description + pointer) | OPTIONAL
- Alternative architectures considered | OPTIONAL
- Open questions | OPTIONAL

Rules

- Boundaries must align with DMG-02 concept model; do not invent mismatched entity ownership.
- Each owned entity/event must have a single primary boundary owner.
- Dependencies must be directional and minimal; cycles must be called out and justified.
- If a boundary decision is contested, it must reference STK-02 (decision log) or be marked UNKNOWN + open question.
- Do not prescribe deployment details here; topology lives in ARC-08/SBDT (but you may reference it).

Output Format

1) Architecture Summary (required)

- **System goal:** {{arch.summary.goal}}
- **Primary constraints:** {{arch.summary.constraints}}
- **Primary quality attributes:** {{arch.summary.quality_attributes}} | OPTIONAL

2) System Context (required)

actor_or_system	type (user/service/vendor)	interaction	notes
{{context[0].name}}	{{context[0].type}}	{{context[0].interaction}}	{{context[0].notes}}

3) Boundary Map (canonical)

boundary_id	name	responsibility	owns	exposes	depends_on	trust_boundary	owner	notes
bnd_01	{{boundaries[0].name}}	{{boundaries[0].responsibility}}	{{boundaries[0].owns}}	{{boundaries[0].exposes}}	{{boundaries[0].depends_on}}	{{boundaries[0].trust}}	{{boundaries[0].owner}}	{{boundaries[0].notes}}
bnd_02	{{boundaries[1].name}}	{{boundaries[1].responsibility}}	{{boundaries[1].owns}}	{{boundaries[1].exposes}}	{{boundaries[1].depends_on}}	{{boundaries[1].trust}}	{{boundaries[1].owner}}	{{boundaries[1].notes}}

4) Inter-Boundary Communication (required)

from_boundary	to_boundary	mode (sync/async)	contract_type (API/event/msg)	data_shared	rationale
{{comms[0].from}}	{{comms[0].to}}	{{comms[0].mode}}	{{comms[0].contract_type}}	{{comms[0].data}}	{{comms[0].rationale}}

5) Cross-Cutting Concerns Placement (required)

- Authentication/authz boundaries: {{crosscut.auth}}
- Error model ownership: {{crosscut.errors}}
- Logging/tracing placement: {{crosscut.observability}}
- Rate limiting placement: {{crosscut.ratelimits}} | OPTIONAL
- Caching boundaries: {{crosscut.caching}} | OPTIONAL

6) Constraints & Rationale (required)

constraint_id	statement	why	enforcement_point	refs
c_01	{{constraints[0].statement}}	{{constraints[0].why}}	{{constraints[0].enforcement}}	{{constraints[0].refs}}

7) Alternatives Considered (optional)

- {{alternatives[0]}} | OPTIONAL

8) Open Questions (optional)

- {{open_questions[0]}} | OPTIONAL

Cross-References

- Upstream: {{xref:PRD-04}} | OPTIONAL, {{xref:DMG-02}} | OPTIONAL, {{xref:DES-01}} | OPTIONAL, {{xref:RISK-02}} | OPTIONAL

- Downstream: {{xref:ARC-02}}, {{xref:SBDT-01}} | OPTIONAL, {{xref:PMAD-01}} | OPTIONAL, {{xref:ERR-01}} | OPTIONAL, {{xref:RTM-01}} | OPTIONAL, {{xref:WFO-01}} | OPTIONAL

- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Boundary list + responsibilities + ownership + dependencies.

- intermediate: Required. Add comms modes and cross-cutting placements.

- advanced: Required. Add constraints table and alternatives considered.

Unknown Handling

- UNKNOWN_ALLOWED: `diagrams`, `alternatives`, `open_questions`, `notes`
- If any boundary lacks responsibility or ownership → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - boundaries_count >= 4
  - ownership_defined_for_all_boundaries == true
  - dependencies_defined == true
  - crosscutting_defined == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ARC-02

ARC-02 — Core Contracts Overview (APIs, events, entities, shared models)

Header Block

- template_id: ARC-02
- title: Core Contracts Overview (APIs, events, entities, shared models)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-02_Core_Contracts_Overview.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["ARC-01", "PRD-04", "DMG-02", "DMG-04"]
- inputs_required: ["ARC-01", "PRD-04", "DMG-02", "DMG-04", "BRP-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Provide a single place to list the system's core contracts and shared models: primary APIs, key events/messages, canonical entities, and shared schemas. This is a map and index—not the full per-contract detail (those live in API/DATA/RTM/SIC templates).

Inputs Required

- ARC-01: {{xref:ARC-01}}
- PRD-04: {{xref:PRD-04}} | OPTIONAL
- DMG-02: {{xref:DMG-02}} | OPTIONAL

- DMG-04: {{xref:DMG-04}} | OPTIONAL
- BRP-01: {{xref:BRP-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Contract categories list (API/event/message/entity/shared_schema)
- Canonical entity list (with owners)
- API surface overview (high level endpoint groups, not every endpoint)
- Event/message overview (core events/topics, not every low-level event)
- Shared schema/model list (DTOs/shared types, versioned)
- Ownership mapping:
    - contract → boundary owner (from ARC-01)
    - contract → downstream consumers
- Stability classification per contract (stable/experimental/internal)
- Compatibility requirements summary (versioning expectations)

Optional Fields

- Deprecations in progress | OPTIONAL
- Links/pointers to detailed specs (API-01/API-02, DATA-01, RTM-03, SIC-02) | OPTIONAL
- Open questions | OPTIONAL

Rules

- Every contract must have exactly one primary owner boundary.
- Shared models must have versioning or evolution rules (pointer to APIG-06 / DATA-04).
- If a contract is “stable,” it must have a compatibility expectation stated (APIG).
- Do not duplicate detailed schemas here; reference the authoritative doc.

Output Format

1) Contract Registry (canonical)

contract_id	category	name	owner_boundary_id	consumers	stability	versioning_policy_ref	detail_spec_ref	notes
ctr_api_01	api	{{contracts[0].name}}	{{contracts[0].owner}}	{{contracts[0].consumers}}	{{contracts[0].stability}}	{{contracts[0].versioning_ref}}	{{contracts[0].detail_ref}}	{{contracts[0].notes}}
ctr_evt_01	event	{{contracts[1].name}}	{{contracts[1].owner}}	{{contracts[1].consumers}}	{{contracts[1].stability}}	{{contracts[1].versioning_ref}}	{{contracts[1].detail_ref}}	{{contracts[1].notes}}

2) Canonical Entities (required)

entity_id	name	owner_boundary_id	primary_identifiers	lifecycle_notes	detail_spec_ref
{{entities[0].id}}	{{entities[0].name}}	{{entities[0].owner}}	{{entities[0].ids}}	{{entities[0].lifecycle}}	{{entities[0].detail_ref}}

3) API Surface Overview (required)

api_group	purpose	owner_boundary_id	primary_consumers	notes
{{api_groups[0].name}}	{{api_groups[0].purpose}}	{{api_groups[0].owner}}	{{api_groups[0].consumers}}	{{api_groups[0].notes}}

4) Events / Messages Overview (required)

event_or_topic	purpose	producer_boundary_id	consumer_boundaries	delivery_expectation	detail_spec_ref
{{events[0].name}}	{{events[0].purpose}}	{{events[0].producer}}	{{events[0].consumers}}	{{events[0].delivery}}	{{events[0].detail_ref}}

5) Shared Schemas / Models (required)

schema_id	purpose	owner_boundary_id	stability	evolution_rule_ref	notes
{{schemas[0].id}}	{{schemas[0].purpose}}	{{schemas[0].owner}}	{{schemas[0].stability}}	{{schemas[0].evolution_ref}}	{{schemas[0].notes}}

6) Compatibility Summary (required)

- Stable contracts must maintain backward compatibility: {{compat.stable_policy}}
- Experimental contracts policy: {{compat.experimental_policy}} | OPTIONAL
- Deprecation policy reference: {{compat.deprecation_ref}} | OPTIONAL

7) Deprecations (optional)

contract_id	deprecated_on	replacement	timeline	notes
{{deprecations[0].id}}	{{deprecations[0].date}}	{{deprecations[0].replacement}}	{{deprecations[0].timeline}}	{{deprecations[0].notes}}

Cross-References

- Upstream: {{xref:ARC-01}}, {{xref:DMG-02}} | OPTIONAL, {{xref:DMG-04}} | OPTIONAL
- Downstream: {{xref:API-01}} | OPTIONAL, {{xref:API-02}} | OPTIONAL, {{xref:DATA-01}} | OPTIONAL, {{xref:RTM-03}} | OPTIONAL, {{xref:SIC-02}} | OPTIONAL, {{xref:APIG-02}} | OPTIONAL, {{xref:APIG-06}} | OPTIONAL
- Standards: {{standards.rules[STD-NAMING]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Contract registry with owners and categories.
- intermediate: Required. Add stability classification and detail spec refs.
- advanced: Required. Add compatibility summary and deprecations tracking.

Unknown Handling

- UNKNOWN_ALLOWED: `consumers`, `detail_spec_ref`, `deprecations`, `notes`
- If owner_boundary_id is UNKNOWN for any contract → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - every_contract_has_owner == true
  - entity_ownership_present == true
  - stability_present_for_all_contracts == true
  - compatibility_summary_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ARC-03

ARC-03 — Key Workflow Data Flows (end-to-end request/event paths)

Header Block

- template_id: ARC-03
- title: Key Workflow Data Flows (end-to-end request/event paths)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-03_Key_Workflow_Data_Flows.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["DES-01", "ARC-01", "ARC-02", "PRD-04"]
- inputs_required: ["DES-01", "ARC-01", "ARC-02", "PRD-04", "DMG-02", "ERR-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the end-to-end data flows for the system's highest-value workflows: how requests, events, and data move through boundaries, where decisions are made, and where failures can occur. This is the traceable bridge between UX flows and system contracts.

Inputs Required

- DES-01: {{xref:DES-01}} | OPTIONAL
- ARC-01: {{xref:ARC-01}} | OPTIONAL
- ARC-02: {{xref:ARC-02}} | OPTIONAL

- PRD-04: {{xref:PRD-04}} | OPTIONAL
- DMG-02: {{xref:DMG-02}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Workflow list (minimum 5; must cover P0 features)
- For each workflow:
    - wf_id (should align with DES flow_id when possible)
    - linked_feature_ids
    - entry point (screen/action/event)
    - sequence of steps across boundaries (ordered)
    - data objects passed (DTOs/entities)
    - sync/async transitions (where events/queues appear)
    - authorization checks (where enforced)
    - idempotency boundary (if any)
    - failure points + error taxonomy class
    - observability touchpoints (logs/metrics/traces)
    - exit outcomes (success/fail + compensation behavior)
- Coverage table: P0 features mapped to at least one ARC-03 workflow

Optional Fields

- Diagram pointer(s) | OPTIONAL
- Performance/latency expectations (qualitative) | OPTIONAL
- Notes | OPTIONAL

Rules

- Every step must specify: boundary → action → output (request/event/data write).
- Any async step must specify delivery expectation and retry behavior pointer.
- Any auth check must reference the policy model (PMAD) or be marked UNKNOWN.
- Failure points must map to ERR taxonomy (ERR-01) and reason code policy if available.
- Do not define UI behavior here; reference DES for UX and focus on system flow.

Output Format

1) Workflow Index (summary) (required)

wf_id	name	feature_ids	entry_point	boundaries_involved	primary_success_outcome	primary_failure_modes
wf_01	{{workflows[0].name}}	{{workflows[0].feature_ids}}	{{workflows[0].entry}}	{{workflows[0].boundaries}}	{{workflows[0].success}}	{{workflows[0].failures}}
wf_02	{{workflows[1].name}}	{{workflows[1].feature_ids}}	{{workflows[1].entry}}	{{workflows[1].boundaries}}	{{workflows[1].success}}	{{workflows[1].failures}}

2) Workflow Detail Blocks (required, one per workflow)

{{workflows[0].wf_id}} — {{workflows[0].name}}

- **Linked features:** {{workflows[0].feature_ids}}
- **Entry point:** {{workflows[0].entry}}
- **Primary boundaries:** {{workflows[0].boundaries}}
- **Sync/Async:** {{workflows[0].sync_async_summary}} | OPTIONAL

Step-by-step flow

step	boundary	action	input	output	mode (sync/async)	data_written	auth_check	error_class	notes
1	{{workflows[0].steps[0].boundary}}	{{workflows[0].steps[0].action}}	{{workflows[0].steps[0].input}}	{{workflows[0].steps[0].output}}	{{workflows[0].steps[0].mode}}	{{workflows[0].steps[0].data_written}}	{{workflows[0].steps[0].auth}}	{{workflows[0].steps[0].error_class}}	{{workflows[0].steps[0].notes}}
2	{{workflows[0].steps[1].boundary}}	{{workflows[0].steps[1].action}}	{{workflows[0].steps[1].input}}	{{workflows[0].steps[1].output}}	{{workflows[0].steps[1].mode}}	{{workflows[0].steps[1].data_written}}	{{workflows[0].steps[1].auth}}	{{workflows[0].steps[1].error_class}}	{{workflows[0].steps[1].notes}}

Async details (if any)

- Producer step(s): {{workflows[0].async.producers}} | OPTIONAL
- Consumer step(s): {{workflows[0].async.consumers}} | OPTIONAL
- Retry policy pointer: {{workflows[0].async.retry_ref}} | OPTIONAL
- DLQ policy pointer: {{workflows[0].async.dlq_ref}} | OPTIONAL

Idempotency / Concurrency

- Idempotency boundary: {{workflows[0].idempotency.boundary}} | OPTIONAL
- Idempotency key: {{workflows[0].idempotency.key}} | OPTIONAL

- Concurrency conflict rule: {{workflows[0].concurrency.rule}} | OPTIONAL

Failure Points

fail_id	step	condition	error_class	expected_behavior	recovery/compensation
f_01	{{workflows[0].failures[0].step}}	{{workflows[0].failures[0].condition}}	{{workflows[0].failures[0].error_class}}	{{workflows[0].failures[0].behavior}}	{{workflows[0].failures[0].recovery}}

Observability Touchpoints

- Logs: {{workflows[0].obs.logs}}
- Metrics: {{workflows[0].obs.metrics}} | OPTIONAL
- Traces: {{workflows[0].obs.traces}} | OPTIONAL

Exit Outcomes

- Success outcome: {{workflows[0].exit.success}}
- Failure outcome: {{workflows[0].exit.failure}}
- Compensation behavior: {{workflows[0].exit.compensation}} | OPTIONAL

3) P0 Feature Coverage (required)

feature_id	has_workflow	workflow_ids
{{spec.features_by_id[feat_p0].id}}	{{coverage[feat_p0].covered}}	{{coverage[feat_p0].wf_ids}}

Cross-References

- Upstream: {{xref:DES-01}} | OPTIONAL, {{xref:ARC-01}} | OPTIONAL, {{xref:ARC-02}} | OPTIONAL

- Downstream: {{xref:API-01}} | OPTIONAL, {{xref:DATA-01}} | OPTIONAL, {{xref:RTM-01}} | OPTIONAL, {{xref:WFO-01}} | OPTIONAL, {{xref:PMAD-03}} | OPTIONAL, {{xref:ERR-01}} | OPTIONAL

- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. 5 workflows with step tables and basic failure points.

- intermediate: Required. Add async details and observability touchpoints.

- advanced: Required. Add idempotency/concurrency and compensation behavior.

Unknown Handling

- UNKNOWN_ALLOWED: `diagram_pointers`, `performance_expectations`, `async_details`, `idempotency_fields`, `notes`

- If any P0 feature has no workflow mapping → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH

- Pass conditions:

    - required_fields_present == true

    - workflows_count >= 5

    - step_tables_present == true

    - p0_feature_coverage_complete == true

- placeholder_resolution == true
- no_unapproved_unknowns == true

ARC-04

ARC-04 — Authentication Architecture (sessions/JWT/SSO boundaries)

Header Block

- template_id: ARC-04
- title: Authentication Architecture (sessions/JWT/SSO boundaries)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-04_Authentication_Architecture.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["PRD-03", "IAM-02", "PMAD-01", "RISK-02"]
- inputs_required: ["PRD-03", "IAM-01", "IAM-02", "IAM-04", "PMAD-01", "DGP-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how authentication works across the system: identity sources, session/token strategy, SSO boundaries (if any), and the trust model for establishing and maintaining authenticated user context.

Inputs Required

- PRD-03: {{xref:PRD-03}} | OPTIONAL
- IAM-01: {{xref:IAM-01}} | OPTIONAL
- IAM-02: {{xref:IAM-02}} | OPTIONAL

- IAM-04: {{xref:IAM-04}} | OPTIONAL
- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Auth strategy (sessions/JWT/hybrid) with rationale
- Identity sources (email/password, OAuth, SSO, device auth, etc.)
- Token/session lifecycle:
    - issuance rules
    - rotation rules
    - expiry rules
    - revocation rules
    - refresh strategy
- Auth flows (high level):
    - signup
    - login
    - recovery
    - logout
    - SSO handshake (if applicable)
- Trust boundaries:

    - where tokens are accepted
    - which services validate vs delegate
- Threat considerations (high level) and mitigations:
    - replay
    - session fixation
    - token theft
    - CSRF (web)
- Error and user-facing outcomes (ties to ERR/CDX)
- Observability requirements for auth events (audit + logs)

Optional Fields

- MFA strategy | OPTIONAL
- Device/session limits | OPTIONAL
- Notes | OPTIONAL

Rules

- Auth must separate identity (who you are) from authorization (what you can do).
- Any token storage method must respect privacy/security policy; no sensitive tokens in unsafe storage.
- Rotation/revocation must be deterministic; “manual revocation” without strategy is not allowed.
- If SSO exists, define attribute mapping and account linking behavior.

Output Format

1) Strategy Overview (required)

- Strategy: {{auth.strategy}} (sessions/JWT/hybrid)
- Rationale: {{auth.rationale}}
- Identity sources: {{auth.identity_sources}}

2) Token/Session Lifecycle (required)

artifact	issued_when	stored_where	rotated_when	expires_in	revoked_how	notes
access_token	{{lifecycle.access.issued}}	{{lifecycle.access.storage}}	{{lifecycle.access.rotate}}	{{lifecycle.access.expiry}}	{{lifecycle.access.revoke}}	{{lifecycle.access.notes}}
refresh_token	{{lifecycle.refresh.issued}}	{{lifecycle.refresh.storage}}	{{lifecycle.refresh.rotate}}	{{lifecycle.refresh.expiry}}	{{lifecycle.refresh.revoke}}	{{lifecycle.refresh.notes}}
session_cookie	{{lifecycle.session.issued}}	{{lifecycle.session.storage}}	{{lifecycle.session.rotate}}	{{lifecycle.session.expiry}}	{{lifecycle.session.revoke}}	{{lifecycle.session.notes}}

3) Auth Flows (required)

Login

- Steps: {{flows.login.steps}}
- Success outcome: {{flows.login.success}}
- Failure outcomes: {{flows.login.failures}}
- Rate limits / lockouts: {{flows.login.ratelimits}} | OPTIONAL

Signup

- Steps: {{flows.signup.steps}}
- Verification rules: {{flows.signup.verification}} | OPTIONAL

Recovery

- Steps: {{flows.recovery.steps}}
- Security constraints: {{flows.recovery.constraints}}

Logout

- Steps: {{flows.logout.steps}}
- Revocation behavior: {{flows.logout.revocation}}

SSO (optional)

- Provider(s): {{flows.sso.providers}} | OPTIONAL
- Attribute mapping: {{flows.sso.attribute_mapping}} | OPTIONAL
- Account linking rules: {{flows.sso.linking_rules}} | OPTIONAL

4) Trust Boundaries (required)

service_or_boundary	validates_tokens	delegates_to	accepted_artifacts	notes
{{trust[0].name}}	{{trust[0].validates}}	{{trust[0].delegates}}	{{trust[0].artifacts}}	{{trust[0].notes}}

5) Threats & Mitigations (required)

threat	risk	mitigation	owner
replay	{{threats.replay.risk}}	{{threats.replay.mitigation}}	{{threats.replay.owner}}
token_theft	{{threats.token_theft.risk}}	{{threats.token_theft.mitigation}}	{{threats.token_theft.owner}}
csrf	{{threats.csrf.risk}}	{{threats.csrf.mitigation}}	{{threats.csrf.owner}}

6) Auth Event Logging (required)

- Must log events: {{audit.must_log}}

- Redaction rules: {{audit.redaction}}
- Correlation ID policy: {{audit.correlation}} | OPTIONAL

7) Error Outcomes (required)

- User-facing error mapping pointer: {{xref:CDX-04}} | OPTIONAL
- Reason codes pointer: {{xref:ERR-02}} | OPTIONAL

Cross-References

- Upstream: {{xref:IAM-02}} | OPTIONAL, {{xref:IAM-04}} | OPTIONAL, {{xref:PMAD-01}} | OPTIONAL
- Downstream: {{xref:API-02}} | OPTIONAL, {{xref:PMAD-03}} | OPTIONAL, {{xref:ERR-03}} | OPTIONAL, {{xref:AUDIT-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Strategy + lifecycle + core flows (login/logout).
- intermediate: Required. Add trust boundaries and threat mitigations.
- advanced: Required. Add SSO/MFA and detailed logging/redaction rules.

Unknown Handling

- UNKNOWN_ALLOWED: `mfa`, `device_limits`, `sso_details`, `csrf` (if not web), `notes`
- If lifecycle rotation/revocation rules are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - strategy_defined == true
  - lifecycle_defined == true
  - flows_defined == true
  - trust_boundaries_defined == true
  - threats_mitigations_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ARC-05

ARC-05 — Realtime Architecture (channels, presence, delivery, scaling)

Header Block

- template_id: ARC-05
- title: Realtime Architecture (channels, presence, delivery, scaling)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-05_Realtime_Architecture.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["PRD-04", "DES-01", "RTM-01", "RTM-02", "PMAD-01", "RISK-02"]
- inputs_required: ["PRD-04", "DES-01", "RTM-01", "RTM-02", "RTM-03", "RTM-04", "RTM-05", "PMAD-01", "ERR-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the system's realtime architecture at the system level: which realtime use cases exist, what transports/protocols are used, how channels/topics are modeled, how presence/state sync works, and what delivery guarantees and scaling assumptions apply.

Inputs Required

- PRD-04: {{xref:PRD-04}} | OPTIONAL
- DES-01: {{xref:DES-01}} | OPTIONAL

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- RTM-02: {{xref:RTM-02}} | OPTIONAL
- RTM-03: {{xref:RTM-03}} | OPTIONAL
- RTM-04: {{xref:RTM-04}} | OPTIONAL
- RTM-05: {{xref:RTM-05}} | OPTIONAL
- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Realtime scope summary (what is realtime, what is not)
- Use cases covered (minimum 3 for non-trivial systems)
- Transport strategy (WS/WebRTC/pubsub) and selection criteria
- Channel/topic model summary (naming + scoping)
- Presence model summary:
    - online/offline/away definitions
    - TTL/heartbeat policy
    - conflict resolution rule
- Delivery semantics summary:
    - ordering guarantees
    - dedupe strategy
    - ack strategy

    - replay policy (if any)
- Authorization model summary (where checks occur)
- Failure behavior and fallbacks (reconnect strategy, degraded mode)
- Scaling assumptions and bottlenecks (qualitative + key metrics)

Optional Fields

- Multi-region realtime notes | OPTIONAL
- Moderation/abuse hooks pointer | OPTIONAL
- Notes | OPTIONAL

Rules

- Realtime must degrade gracefully: define fallback when realtime is unavailable.
- Presence must define source of truth (server vs client vs hybrid).
- Delivery semantics must be explicit; “best effort” must define what can be dropped.
- Authorization must be enforced server-side for all channel joins and message sends.
- Any scaling assumptions must be traceable to PERF/COST docs later (pointer only).

Output Format

1) Scope Summary (required)

- Realtime included: {{scope.included}}
- Realtime excluded: {{scope.excluded}} | OPTIONAL
- Primary transports: {{scope.transports}}

2) Use Cases (required)

use_case_id	name	purpose	criticality	linked_features	notes
rt_uc_01	{{usecases[0].name}}	{{usecases[0].purpose}}	{{usecases[0].criticality}}	{{usecases[0].feature_ids}}	{{usecases[0].notes}}

3) Transport Strategy (required)

- Primary transport: {{transport.primary}}
- Fallback transport: {{transport.fallback}} | OPTIONAL
- Selection criteria: {{transport.criteria}}
- Connection lifecycle: {{transport.lifecycle}} | OPTIONAL

4) Channel/Topic Model (required)

- Naming convention: {{channels.naming}}
- Scoping rules: {{channels.scoping}}
- Join rules: {{channels.join_rules}}
- Leave/disconnect rules: {{channels.leave_rules}} | OPTIONAL

5) Presence Model (required)

- Presence states: {{presence.states}}
- Heartbeat/TTL: {{presence.ttl_policy}}
- Source of truth: {{presence.source_of_truth}}
- Conflict resolution: {{presence.conflict_resolution}}

6) Delivery Semantics (required)

- Ordering: {{delivery.ordering}}
- Dedupe: {{delivery.dedupe}}
- Ack policy: {{delivery.ack}}
- Replay policy: {{delivery.replay}} | OPTIONAL

7) Authorization Summary (required)

- Where checks happen: {{authz.enforcement_points}}
- Join authorization rule: {{authz.join_rule}}
- Send authorization rule: {{authz.send_rule}}

8) Failure + Fallback Behavior (required)

- Reconnect strategy: {{failure.reconnect}}
- Backoff: {{failure.backoff}} | OPTIONAL
- Degraded mode behavior: {{failure.degraded_mode}}
- User messaging pointer: {{xref:CDX-04}} | OPTIONAL

9) Scaling Assumptions (required)

- Expected concurrency: {{scale.expected_concurrency}} | OPTIONAL
- Message rate assumptions: {{scale.message_rate}} | OPTIONAL
- Bottlenecks: {{scale.bottlenecks}}
- Monitoring needs: {{scale.monitoring}} | OPTIONAL

Cross-References

- Upstream: {{xref:RTM-01}} | OPTIONAL, {{xref:RTM-02}} | OPTIONAL, {{xref:RTM-03}} | OPTIONAL, {{xref:RTM-04}} | OPTIONAL, {{xref:RTM-05}} | OPTIONAL

- Downstream: {{xref:RTM-06}} | OPTIONAL, {{xref:OBS-*}} | *OPTIONAL, {{xref:PERF-}}* | OPTIONAL, {{xref:WFO-*}} | OPTIONAL

- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Scope + transports + presence + delivery basics.

- intermediate: Required. Add authz and failure fallback behavior.

- advanced: Required. Add scaling assumptions and monitoring needs.

Unknown Handling

- UNKNOWN_ALLOWED: `multi_region_notes`, `moderation_hooks`, `expected_concurrency`, `message_rate`, `notes`

- If delivery semantics or presence source of truth is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH

- Pass conditions:

    - required_fields_present == true

    - usecases_present == true

    - transport_strategy_present == true

    - presence_defined == true

- delivery_semantics_defined == true
- authz_defined == true
- fallback_defined == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

ARC-06

ARC-06 — Error Model + Reason Codes (canonical failure taxonomy)

Header Block

- template_id: ARC-06
- title: Error Model + Reason Codes (canonical failure taxonomy)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-06_Error_Model_Reason_Codes.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["BRP-01", "DMG-03", "DES-07", "CDX-04"]
- inputs_required: ["BRP-01", "DMG-03", "DES-07", "CDX-04", "ERR-01", "ERR-02", "ERR-03", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the system-wide error model and reason-code strategy so failures are predictable, mappable to UX copy, and observable. This provides the canonical failure taxonomy that APIs, workflows, and realtime systems must follow.

Inputs Required

- BRP-01: {{xref:BRP-01}} | OPTIONAL
- DMG-03: {{xref:DMG-03}} | OPTIONAL
- DES-07: {{xref:DES-07}} | OPTIONAL

- CDX-04: {{xref:CDX-04}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Error class model:
    - domain/business rule errors
    - validation errors
    - auth/authz errors
    - dependency/integration errors
    - system/unknown errors
- Reason code rules:
    - rc_* naming convention
    - uniqueness and ownership
    - stability policy (never reuse)
- Mapping rules:
    - error class → HTTP status (if applicable)
    - reason_code → UX surface/message policy
    - reason_code → retryability policy
- Safe error payload policy (no leakage)

- Correlation ID policy (whether shown to user)
- Logging/telemetry required fields

Optional Fields

- Versioning/deprecation policy for reason codes | OPTIONAL
- Domain-specific extensions | OPTIONAL
- Notes | OPTIONAL

Rules

- Every user-facing error must have a reason_code OR a policy-defined fallback reason.
- Reason codes must be stable; deprecate, don't reuse.
- Errors must not leak sensitive internal information.
- Retryability must align to idempotency rules (ERR-05 / WFO-03).
- UX mapping must align to DES-07 and CDX-04.

Output Format

1) Error Classes (required)

error_class	description	examples	default_surface	default_retryability
domain_rule	{{classes.domain.desc}}	{{classes.domain.examples}}	{{classes.domain.surface}}	{{classes.domain.retry}}
validation	{{classes.validation.desc}}	{{classes.validation.examples}}	{{classes.validation.surface}}	{{classes.validation.retry}}
authz	{{classes.authz.desc}}	{{classes.authz.examples}}	{{classes.authz.surface}}	{{classes.authz.retry}}

dependency	{{classes.dependency.desc}}	{{classes.dependency.examples}}	{{classes.dependency.surface}}	{{classes.dependency.retry}}
system_unknown	{{classes.system.desc}}	{{classes.system.examples}}	{{classes.system.surface}}	{{classes.system.retry}}

2) Reason Code Rules (required)

- Naming convention: {{reason_codes.naming}} (rc_<domain>_<slug>)
- Ownership rule: {{reason_codes.ownership}}
- Stability rule: {{reason_codes.stability}}
- Fallback reason_code: {{reason_codes.fallback}}

3) Mappings (required)

- Error class → status mapping pointer: {{xref:ERR-03}} | OPTIONAL
- Reason code → UX mapping pointer: {{xref:ERR-04}} | OPTIONAL
- Reason code → retry mapping pointer: {{xref:ERR-05}} | OPTIONAL

4) Safe Payload Policy (required)

- Allowed fields: {{payload.allowed_fields}}
- Forbidden fields: {{payload.forbidden_fields}}
- Internal debug fields policy: {{payload.debug_policy}} | OPTIONAL

5) Correlation ID Policy (required)

- Generated where: {{correlation.generated_where}}
- Returned to client: {{correlation.returned_to_client}}
- Shown to user: {{correlation.shown_to_user}}

- Copy pointer (if shown): {{xref:CDX-04}} | OPTIONAL

6) Logging/Telemetry Requirements (required)

- Required log fields: {{telemetry.required_log_fields}}
- Required metric tags: {{telemetry.required_metric_tags}} | OPTIONAL
- Trace correlation rules: {{telemetry.trace_rules}} | OPTIONAL

Cross-References

- Upstream: {{xref:DES-07}} | OPTIONAL, {{xref:CDX-04}} | OPTIONAL, {{xref:BRP-01}} | OPTIONAL
- Downstream: {{xref:ERR-01}}, {{xref:ERR-02}}, {{xref:ERR-03}}, {{xref:ERR-04}}, {{xref:ERR-05}}, {{xref:ERR-06}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Define classes + reason-code rules + safe payload policy.
- intermediate: Required. Add mapping pointers and correlation ID policy.
- advanced: Required. Add telemetry requirements and deprecation policy.

Unknown Handling

- UNKNOWN_ALLOWED: `deprecation_policy`, `domain_extensions`, `notes`, `metric_tags`, `trace_rules`
- If fallback reason_code policy is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - error_classes_present == true
  - reason_code_rules_present == true
  - safe_payload_policy_present == true
  - correlation_policy_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ARC-07

ARC-07 — Integration Architecture (3rd-party boundaries + trust model)

Header Block

- template_id: ARC-07
- title: Integration Architecture (3rd-party boundaries + trust model)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-07_Integration_Architecture.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["ARC-01", "SIC-01", "RISK-03", "COMP-05"]
- inputs_required: ["ARC-01", "SIC-01", "RISK-03", "BRP-01", "DGP-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the system-level integration architecture: what external systems exist, what boundaries/trust assumptions apply, what data crosses the boundary, and the rules for authentication, integrity, retries, and auditing. This is the architecture-level view; per-integration details live in SIC templates.

Inputs Required

- ARC-01: {{xref:ARC-01}} | OPTIONAL
- SIC-01: {{xref:SIC-01}} | OPTIONAL

- RISK-03: {{xref:RISK-03}} | OPTIONAL
- BRP-01: {{xref:BRP-01}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- External systems list (minimum 3 if any integrations exist)
- For each external system:
    - ext_id
    - name
    - purpose
    - direction (inbound/outbound/bidirectional)
    - trust classification (trusted/partially/untrusted)
    - auth mechanism (keys/OAuth/SSO/mTLS/webhook signature)
    - data exchanged (high-level categories)
    - PII classification (high-level, pointer to DGP)
    - integrity guarantees (signatures, replay protection)
    - rate limits/quotas (high-level)
    - failure handling model (retries, DLQ, manual ops)
    - audit/logging requirements
    - owner (internal)

- Integration boundary rules:
    - where validation happens
    - where mapping/transforms happen
    - where secrets live and rotate
- “Do not trust” rules (inputs that must never be accepted without validation)
- Compatibility expectations (versioning, contract stability)

Optional Fields

- Diagrams/pointers | OPTIONAL
- Data residency constraints | OPTIONAL
- Notes | OPTIONAL

Rules

- All inbound data must be validated at the boundary (schema + auth + integrity).
- Secrets must not be stored in code; must reference secrets management policy (ENV/SKM).
- Webhook integrity must include replay protection (timestamp + nonce / signature policy).
- Any PII crossing boundaries must follow DGP classification and retention rules.
- Failures must be categorized and mapped to ERR taxonomy (dependency errors).

Output Format

1) Integration Landscape (required)

ext_id	name	direction	purpose	trust	auth	data_categories	pii_class	owner	notes
ext_01	{{ext[0].name}}	{{ext[0].direction}}	{{ext[0].purpose}}	{{ext[0].trust}}	{{ext[0].auth}}	{{ext[0].data}}	{{ext[0].pii}}	{{ext[0].owner}}	{{ext[0].notes}}

2) Boundary Rules (required)

- Validation at boundary: {{boundary.validation}}
- Mapping/transforms location: {{boundary.mapping_location}}
- Secrets location/rotation pointer: {{boundary.secrets_pointer}} | OPTIONAL
- Observability at boundary: {{boundary.observability}} | OPTIONAL

3) Trust Model Rules (required)

- Trusted vs untrusted inputs policy: {{trust.inputs_policy}}
- Do-not-trust list (required): {{trust.do_not_trust}}
- Integrity rules: {{trust.integrity_rules}}

4) Failure Model (required)

failure_type	expected_behavior	retry_policy	escalation	error_class
timeout	{{failure.timeout.behavior}}	{{failure.timeout.retry}}	{{failure.timeout.escalation}}	dependency
invalid_signature	{{failure.sig.behavior}}	{{failure.sig.retry}}	{{failure.sig.escalation}}	authz
schema_mismatch	{{failure.schema.behavior}}	{{failure.schema.retry}}	{{failure.schema.escalation}}	dependency

5) Compatibility Expectations (required)

- Contract stability: {{compat.stability}}
- Versioning/deprecation: {{compat.versioning}} | OPTIONAL
- Backward compatibility stance: {{compat.backward_compat}}

6) Audit/Logging Requirements (required)

- Required audit events: {{audit.events}}
- Redaction policy: {{audit.redaction}}
- Correlation ID propagation: {{audit.correlation}} | OPTIONAL

Cross-References

- Upstream: {{xref:ARC-01}} | OPTIONAL, {{xref:SIC-01}} | OPTIONAL, {{xref:RISK-03}} | OPTIONAL
- Downstream: {{xref:SIC-02}}, {{xref:SIC-03}}, {{xref:SIC-04}}, {{xref:SIC-05}}, {{xref:SIC-06}} | OPTIONAL, {{xref:ERR-01}} | OPTIONAL, {{xref:SKM-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Inventory + boundary validation + basic failure model.
- intermediate: Required. Add trust classification and integrity guarantees.
- advanced: Required. Add audit/redaction and compatibility expectations.

Unknown Handling

- UNKNOWN_ALLOWED: `diagrams`, `data_residency_constraints`, `rate_limits`, `notes`
- If any inbound integration lacks validation/auth rules → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - integrations_inventory_present == true
  - boundary_rules_present == true
  - trust_rules_present == true
  - failure_model_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ARC-08

ARC-08 — Deployment Topology (services, environments, network boundaries)

Header Block

- template_id: ARC-08
- title: Deployment Topology (services, environments, network boundaries)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-08_Deployment_Topology.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["ARC-01", "SBDT-01", "OPS-01", "ENV-01"]
- inputs_required: ["ARC-01", "SBDT-01", "SBDT-02", "SBDT-03", "OPS-01", "ENV-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the system's deployment topology: what runs where, what environments exist, how network boundaries are segmented, and what isolation constraints apply. This is the architecture-level deployment map; CI/CD details live in OPS/CICD.

Inputs Required

- ARC-01: {{xref:ARC-01}} | OPTIONAL

- SBDT-01: {{xref:SBDT-01}} | OPTIONAL
- SBDT-02: {{xref:SBDT-02}} | OPTIONAL
- SBDT-03: {{xref:SBDT-03}} | OPTIONAL
- OPS-01: {{xref:OPS-01}} | OPTIONAL
- ENV-01: {{xref:ENV-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Environments list (dev/stage/prod at minimum unless explicitly N/A)
- Runtime components list (services, workers, DBs, queues, storage, gateways)
- For each runtime component:
    - component_id
    - type (api/service/worker/db/queue/cache/storage/gateway)
    - owner boundary/service
    - environments deployed to
    - network zone (public/private/internal)
    - ingress/egress rules (high level)
    - secrets dependency (yes/no + pointer)
    - scaling notes (basic)
    - health checks/readiness expectations (high level)
- Network boundary rules:

    - what is public
    - what is private
    - what can talk to what
- Environment parity rules (what must match across envs)
- Deployment safety constraints (migrations, rollouts, rollback posture)

Optional Fields

- Multi-region notes | OPTIONAL
- Blue/green or canary posture | OPTIONAL
- Notes | OPTIONAL

Rules

- Public exposure must be explicit; default is private/internal.
- Any component handling PII must be in approved zones and follow privacy constraints.
- Environment names must match ENV-01 config matrix.
- Deployment safety must reference migration/rollback policies (REL/BDR where applicable).

Output Format

1) Environments (required)

env	purpose	access	data_policy	notes
dev	{{envs.dev.purpose}}	{{envs.dev.access}}	{{envs.dev.data_policy}}	{{envs.dev.notes}}

stage	{{envs.stage.purpose}}	{{envs.stage.access}}	{{envs.stage.data_policy}}	{{envs.stage.notes}}
prod	{{envs.prod.purpose}}	{{envs.prod.access}}	{{envs.prod.data_policy}}	{{envs.prod.notes}}

2) Runtime Components (canonical)

component_id	type	owner_boundary_id	envs	network_zone	ingress	egress	secrets_ref	scaling_notes	health_notes
cmp_api	{{components[0].type}}	{{components[0].owner}}	{{components[0].envs}}	{{components[0].zone}}	{{components[0].ingress}}	{{components[0].egress}}	{{components[0].secrets_ref}}	{{components[0].scaling}}	{{components[0].health}}

3) Network Boundaries (required)

- Public zone definition: {{network.public}}
- Private/internal zone definition: {{network.private}}
- Allowed communication matrix (high level): {{network.allowed_matrix}}
- Deny-by-default rule: {{network.deny_by_default}}

4) Environment Parity Rules (required)

- What must match (configs, versions, schemas): {{parity.must_match}}
- Allowed differences: {{parity.allowed_differences}} | OPTIONAL

5) Deployment Safety Constraints (required)

- Migration safety rule: {{safety.migrations}}
- Rollout posture: {{safety.rollouts}} | OPTIONAL

- Rollback posture: {{safety.rollback}}
- Secrets rotation dependency: {{safety.secrets_rotation}} | OPTIONAL

Cross-References

- Upstream: {{xref:SBDT-02}} | OPTIONAL, {{xref:SBDT-03}} | OPTIONAL, {{xref:ENV-01}} | OPTIONAL, {{xref:OPS-01}} | OPTIONAL
- Downstream: {{xref:OPS-02}} | OPTIONAL, {{xref:CICD-*}}* | *OPTIONAL, {{xref:REL-04}}* | *OPTIONAL, {{xref:BDR-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Environments + runtime components + public/private separation.
- intermediate: Required. Add ingress/egress rules and parity rules.
- advanced: Required. Add deployment safety constraints and rollback posture.

Unknown Handling

- UNKNOWN_ALLOWED: `multi_region_notes`, `canary_posture`, `notes`, `scaling_notes`, `health_notes`
- If any public component lacks ingress rules → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:

- required_fields_present == true
- environments_present == true
- runtime_components_present == true
- network_boundaries_present == true
- deployment_safety_present == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

ARC-09

ARC-09 — Cross-Cutting Concerns (logging, tracing, rate limits, caching touchpoints)

Header Block

- template_id: ARC-09
- title: Cross-Cutting Concerns (logging, tracing, rate limits, caching touchpoints)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-09_Cross_Cutting_Concerns.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["ARC-01", "ARC-02", "ERR-06", "OBS-01", "APIG-01"]
- inputs_required: ["ARC-01", "ARC-02", "ERR-06", "OBS-01", "OBS-03", "APIG-01", "RLIM-01", "CACHE-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how cross-cutting concerns are applied consistently across all boundaries: observability (logs/metrics/traces), rate limiting/abuse controls touchpoints, caching strategy touchpoints, and global policies that must not vary by service without explicit exception.

Inputs Required

- ARC-01: {{xref:ARC-01}} | OPTIONAL

- ARC-02: {{xref:ARC-02}} | OPTIONAL
- OBS-01: {{xref:OBS-01}} | OPTIONAL
- OBS-03: {{xref:OBS-03}} | OPTIONAL
- ERR-06: {{xref:ERR-06}} | OPTIONAL
- APIG-01: {{xref:APIG-01}} | OPTIONAL
- RLIM-01: {{xref:RLIM-01}} | OPTIONAL
- CACHE-01: {{xref:CACHE-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Concern categories (minimum: logging, tracing, metrics, rate limiting, caching)
- For each category:
    - canonical policy statement
    - required fields or behaviors
    - enforcement points (gateway/service/client)
    - exceptions policy (how to deviate)
- Boundary touchpoints map (where each concern applies)
- Required propagation rules (correlation IDs, trace headers, request IDs)
- PII redaction rule for observability
- Verification checklist

Optional Fields

- Cost controls (sampling, retention) | OPTIONAL
- Notes | OPTIONAL

Rules

- Observability fields must be consistent across services (define minimum set).
- Correlation/trace IDs must propagate across all hops including async boundaries.
- Rate limiting must be enforceable at a deterministic point (edge/gateway preferred).
- Caching must not violate correctness or authorization; never cache privileged data without scope keys.
- Any exception must be documented with rationale and owner.

Output Format

1) Canonical Policies (required)

concern	policy	enforcement_points	required_fields_or_rules	exceptions_allowed	notes
logging	{{policies.logging.policy}}	{{policies.logging.enforcement}}	{{policies.logging.required}}	{{policies.logging.exceptions}}	{{policies.logging.notes}}
tracing	{{policies.tracing.policy}}	{{policies.tracing.enforcement}}	{{policies.tracing.required}}	{{policies.tracing.exceptions}}	{{policies.tracing.notes}}
metrics	{{policies.metrics.policy}}	{{policies.metrics.enforcement}}	{{policies.metrics.required}}	{{policies.metrics.exceptions}}	{{policies.metrics.notes}}
rate_limiting	{{policies.ratelimit.policy}}	{{policies.ratelimit.enforcement}}	{{policies.ratelimit.required}}	{{policies.ratelimit.exceptions}}	{{policies.ratelimit.notes}}
caching	{{policies.caching.policy}}	{{policies.caching.enforcement}}	{{policies.caching.required}}	{{policies.caching.exceptions}}	{{policies.caching.notes}}

2) Touchpoints Map (required)

boundary_id	logging	tracing	metrics	rate_limiting	caching	notes
{{touchpoints[0].boundary}}	{{touchpoints[0].logging}}	{{touchpoints[0].tracing}}	{{touchpoints[0].metrics}}	{{touchpoints[0].ratelimit}}	{{touchpoints[0].caching}}	{{touchpoints[0].notes}}

3) Propagation Rules (required)

- Correlation ID header: {{propagation.correlation_header}}
- Trace context standard: {{propagation.trace_standard}}
- Async propagation (events/jobs): {{propagation.async}}
- Client → server propagation: {{propagation.client_server}}

4) PII Redaction Rules (required)

- Never log: {{redaction.never_log}}
- Allowed with hashing: {{redaction.hash_ok}} | OPTIONAL
- Sampling/retention policy pointer: {{redaction.retention_pointer}} | OPTIONAL

5) Verification Checklist (required)

- {{verify[0]}}
- {{verify[1]}}
- {{verify[2]}}
- {{verify[3]}} | OPTIONAL

Cross-References

- Upstream: {{xref:OBS-01}} | OPTIONAL, {{xref:OBS-03}} | OPTIONAL, {{xref:ERR-06}} | OPTIONAL, {{xref:APIG-01}} | OPTIONAL

- Downstream: {{xref:OPS-05}} | OPTIONAL, {{xref:OBS-04}} | OPTIONAL, {{xref:PERF-*}} | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Policies + propagation rules + basic touchpoints.
- intermediate: Required. Add redaction rules and exceptions policy.
- advanced: Required. Add verification checklist and cost controls.

Unknown Handling

- UNKNOWN_ALLOWED: `cost_controls`, `retention_pointer`, `notes`, `exceptions_allowed`
- If propagation rules are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
    - required_fields_present == true
    - canonical_policies_present == true
    - touchpoints_present == true
    - propagation_rules_present == true
    - redaction_rules_present == true
    - placeholder_resolution == true

- no_unapproved_unknowns == true

ARC-10

ARC-10 — Architecture Constraints & Invariants (must-hold rules + enforcement points)

Header Block

- template_id: ARC-10
- title: Architecture Constraints & Invariants (must-hold rules + enforcement points)
- type: system_architecture
- template_version: 1.0.0
- output_path: 10_app/architecture/ARC-10_Architecture_Constraints_Invariants.md
- compliance_gate_id: TMP-05.PRIMARY.ARCH
- upstream_dependencies: ["ARC-01", "DMG-03", "RISK-02", "SEC-02"]
- inputs_required: ["ARC-01", "DMG-03", "RISK-02", "SEC-02", "APIG-02", "WFO-03", "ERR-05", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the architecture-level constraints and invariants that must always hold, regardless of feature changes: boundary rules, data ownership rules, consistency expectations, security constraints, and operational constraints. These act as hard guardrails for implementation and review gates.

Inputs Required

- ARC-01: {{xref:ARC-01}} | OPTIONAL

- DMG-03: {{xref:DMG-03}} | OPTIONAL
- RISK-02: {{xref:RISK-02}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- APIG-02: {{xref:APIG-02}} | OPTIONAL
- WFO-03: {{xref:WFO-03}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Constraint list (minimum 12 for non-trivial systems)
- For each constraint:
    - con_id
    - statement (must/never)
    - category (boundary/data/security/ops/compat/perf)
    - scope (system/boundary/service)
    - enforcement_points (design review, code review, runtime, tests)
    - detection method (how we know it’s violated)
    - owner (role/team)
    - severity (hard/soft)
    - related docs (refs to ARC/APIG/WFO/ERR/etc.)
- Exception policy (how to request exceptions + approvals)

Optional Fields

- Examples (good/bad) | OPTIONAL
- Notes | OPTIONAL

Rules

- “Hard” constraints must have detection method + enforcement point.
- Constraints must be non-redundant with DMG invariants; DMG are domain truths; ARC are architecture truths. If overlap exists, cross-reference and keep one authoritative.
- Any exception must be time-bound and recorded (STK decision pointer).

Output Format

1) Constraints Registry (canonical)

con_id	statement	category	scope	enforcement_points	detection_method	owner	severity	related_refs	notes
con_01	{{constraints[0].statement}}	{{constraints[0].category}}	{{constraints[0].scope}}	{{constraints[0].enforcement}}	{{constraints[0].detection}}	{{constraints[0].owner}}	{{constraints[0].severity}}	{{constraints[0].refs}}	{{constraints[0].notes}}
con_02	{{constraints[1].statement}}	{{constraints[1].category}}	{{constraints[1].scope}}	{{constraints[1].enforcement}}	{{constraints[1].detection}}	{{constraints[1].owner}}	{{constraints[1].severity}}	{{constraints[1].refs}}	{{constraints[1].notes}}

2) Hard Constraints Summary (required)

- {{derive:LIST_HARD_CONSTRAINTS(constraints)}} | OPTIONAL

3) Exception Policy (required)

- When exceptions allowed: {{exceptions.when_allowed}}
- Required justification fields: {{exceptions.fields}}
- Approval required: {{exceptions.approval}}
- Time-bound rule: {{exceptions.time_bound}}
- Logging/decision pointer: {{exceptions.decision_pointer}} | OPTIONAL

4) Examples (optional)

- Good example: {{examples.good}} | OPTIONAL
- Bad example: {{examples.bad}} | OPTIONAL

Cross-References

- Upstream: {{xref:ARC-01}} | OPTIONAL, {{xref:DMG-03}} | OPTIONAL, {{xref:RISK-02}} | OPTIONAL
- Downstream: {{xref:APIG-04}} | OPTIONAL, {{xref:QA-05}} | OPTIONAL, {{xref:RELIA-*}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. 12 constraints with enforcement points.
- intermediate: Required. Add detection methods and severity.
- advanced: Required. Add exception policy and examples.

Unknown Handling

- UNKNOWN_ALLOWED: `examples`, `notes`, `decision_pointer`
- If any hard constraint lacks detection_method → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ARCH
- Pass conditions:
  - required_fields_present == true
  - constraints_count >= 12
  - hard_constraints_have_detection == true
  - exception_policy_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

System Interfaces & Integration Contracts (SIC)

- System Interfaces & Integration Contracts (SIC)
  SIC-01 External Interface Inventory (systems, purpose, direction)
  SIC-02 Contract Spec (per interface: requests/responses/events/auth/errors)
  SIC-03 Webhook Contract Spec (producer/consumer, retries, signatures)
  SIC-04 Data Mapping Contract (field mappings + transforms + validation)
  SIC-05 Integration Failure Modes & Recovery (timeouts, retries, DLQ, fallbacks)
  SIC-06 Vendor/Third-Party Trust Model (data sharing, scopes, audit needs)

SIC-01

SIC-01 — External Interface Inventory (systems, purpose, direction)

Header Block

- template_id: SIC-01
- title: External Interface Inventory (systems, purpose, direction)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-01_External_Interface_Inventory.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["ARC-07", "RISK-03", "COMP-05"]
- inputs_required: ["ARC-07", "RISK-03", "DGP-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

List every external interface the system interacts with and define high-level constraints per interface: directionality, purpose, data sensitivity, trust classification, and where detailed contracts live. This makes integration scope deterministic and auditable.

Inputs Required

- ARC-07: {{xref:ARC-07}} | OPTIONAL
- RISK-03: {{xref:RISK-03}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL

- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Interface list (minimum 3 if integrations exist; otherwise mark N/A)
- For each interface:
    - interface_id
    - external_system_name
    - purpose
    - direction (inbound/outbound/bidirectional)
    - interface_type (REST/GraphQL/webhook/file/SFTP/queue/etc.)
    - auth mechanism (API key/OAuth/mTLS/signature)
    - trust classification (trusted/partial/untrusted)
    - data categories exchanged
    - PII classification pointer (DGP)
    - expected volume/latency class (low/med/high)
    - rate limits/quotas (if known)
    - owner (internal)
    - detailed contract template pointer (SIC-02/SIC-03/etc.)
    - status (planned/active/deprecated)

Optional Fields

- Vendor contact/SLAs | OPTIONAL
- Notes | OPTIONAL

Rules

- If applies == false (no integrations), include 00_NA block only.
- Every interface must have a detailed contract pointer or be flagged as incomplete.
- Trust classification drives validation and audit requirements (SEC/DGP rules).
- Do not include internal APIs here; only cross-org/system boundaries.

Output Format

1) Applicability

- applies: {{sic.applies}} (true/false)
- 00_NA (if not applies): {{sic.na_block}} | OPTIONAL

2) External Interface Inventory (canonical)

interface_id	system	purpose	direction	type	auth	trust	data_categories	pii_class	volume_latency	rate_limits	owner	contract_ref	status	notes
intf_	{{interfaces[0].sy	{{interfaces[0].p	{{interfaces[0].	{{interfaces[0].t	{{interfaces[0].	{{interfaces[0].tr	{{interfaces[0].	{{interfaces[0]	{{interfaces[0]	{{interfaces[0].r	{{interfaces[0].o	{{interfaces[0].co	{{interfaces[0].st	{{interfaces[0].n

01	stem}}	urpose}}	direction}}	ype}}	auth}}	ust}}	data}}	.pii}}	.vol}}	ate}}	wner}}	ntract_ref}}	atus}}	otes}}
in_02	{{interfaces[1].system}}	{{interfaces[1].purpose}}	{{interfaces[1].direction}}	{{interfaces[1].type}}	{{interfaces[1].auth}}	{{interfaces[1].trust}}	{{interfaces[1].data}}	{{interfaces[1].pii}}	{{interfaces[1].vol}}	{{interfaces[1].rate}}	{{interfaces[1].owner}}	{{interfaces[1].contract_ref}}	{{interfaces[1].status}}	{{interfaces[1].notes}}

3) Coverage Checks (required if applies)

- Every interface has contract_ref: {{coverage.contract_refs_complete}}
- Trust classification present: {{coverage.trust_complete}}
- PII classification present: {{coverage.pii_complete}}

Cross-References

- Upstream: {{xref:ARC-07}} | OPTIONAL, {{xref:RISK-03}} | OPTIONAL
- Downstream: {{xref:SIC-02}}, {{xref:SIC-03}}, {{xref:SIC-04}}, {{xref:SIC-05}}, {{xref:SIC-06}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Inventory table with purpose/direction/type/auth.
- intermediate: Required. Add trust classification and PII pointers.
- advanced: Required. Add volume/latency class and coverage checks.

Unknown Handling

- UNKNOWN_ALLOWED: `rate_limits`, `vendor_slas`, `notes`
- If applies == true and any interface lacks contract_ref → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION
- Pass conditions:
    - required_fields_present == true
    - if_applies_then_inventory_present == true
    - contract_refs_complete == true
    - trust_complete == true
    - pii_complete == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SIC-02

SIC-02 — Contract Spec (per interface: requests/responses/events/auth/errors)

Header Block

- template_id: SIC-02
- title: Contract Spec (per interface: requests/responses/events/auth/errors)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-02_Interface_Contract_Specs.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["SIC-01", "ARC-07", "ERR-03"]
- inputs_required: ["SIC-01", "ARC-07", "ERR-03", "DGP-01", "SEC-02", "APIG-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the detailed, enforceable contract for each external interface: authentication, request/response shapes, events, validation rules, error mapping, rate limits, and operational expectations. This is the authoritative specification used to implement and validate integrations.

Inputs Required

- SIC-01: {{xref:SIC-01}}
- ARC-07: {{xref:ARC-07}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- APIG-02: {{xref:APIG-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- One contract block per interface_id
- Per contract:
    - interface_id + external system name
    - direction
    - auth method details (headers/scopes/keys/mTLS)
    - endpoints/events covered (list)
    - request schema(s)
    - response schema(s)
    - validation rules (schema + semantic checks)
    - idempotency expectations (if any)
    - pagination/filtering rules (if any)
    - error handling:
        - status mapping (if HTTP)
        - reason_code mapping policy
        - retryability rules (ties to ERR-05)
    - rate limits/quotas and backoff expectations

    - timeout policy and SLAs (if known)
    - logging/audit requirements (redaction)
    - versioning and compatibility policy (APIG)
- Test requirements (contract tests) summary

Optional Fields

- Example payloads | OPTIONAL
- Sandbox/dev environment notes | OPTIONAL
- Notes | OPTIONAL

Rules

- Contracts must not include secrets; reference where they are stored/rotated.
- Any inbound contract must define signature/integrity and replay protection if applicable.
- All error handling must map to the system error model (ERR).
- Validation must include both schema and semantic checks (where needed).
- Versioning policy must be explicit; if vendor changes, define update cadence.

Output Format

Contract: {{contracts[0].interface_id}} — {{contracts[0].system_name}}

1) Overview

- Direction: {{contracts[0].direction}}
- Interface type: {{contracts[0].type}}

- Purpose: {{contracts[0].purpose}} | OPTIONAL
- Stability: {{contracts[0].stability}} | OPTIONAL

2) Authentication

- Method: {{contracts[0].auth.method}}
- Required headers/scopes: {{contracts[0].auth.requirements}}
- Rotation/expiry policy: {{contracts[0].auth.rotation}} | OPTIONAL
- Integrity/signature: {{contracts[0].auth.integrity}} | OPTIONAL

3) Endpoints / Events Covered

id	kind (endpoint/event)	name	direction	notes
{{contracts[0].items[0].id}}	{{contracts[0].items[0].kind}}	{{contracts[0].items[0].name}}	{{contracts[0].items[0].direction}}	{{contracts[0].items[0].notes}}

4) Schemas

- Request schemas: {{contracts[0].schemas.requests}}
- Response schemas: {{contracts[0].schemas.responses}}
- Shared types: {{contracts[0].schemas.shared}} | OPTIONAL

5) Validation Rules

- Schema validation: {{contracts[0].validation.schema}}
- Semantic validation: {{contracts[0].validation.semantic}} | OPTIONAL
- Rejection behavior: {{contracts[0].validation.rejection_behavior}}

6) Idempotency / Pagination (if applicable)

- Idempotency: {{contracts[0].idempotency}} | OPTIONAL
- Pagination: {{contracts[0].pagination}} | OPTIONAL
- Filtering/sorting: {{contracts[0].filtering}} | OPTIONAL

7) Error Handling

- Status mapping: {{contracts[0].errors.status_mapping}} | OPTIONAL
- Reason codes mapping policy: {{contracts[0].errors.reason_codes}}
- Retryability rules: {{contracts[0].errors.retryability}}
- Backoff expectations: {{contracts[0].errors.backoff}} | OPTIONAL

8) Limits, Timeouts, SLAs

- Rate limits/quotas: {{contracts[0].limits.rate_limits}} | OPTIONAL
- Timeout policy: {{contracts[0].limits.timeouts}} | OPTIONAL
- SLA expectations: {{contracts[0].limits.sla}} | OPTIONAL

9) Logging / Audit

- Required fields: {{contracts[0].logging.required_fields}}
- Redaction rules: {{contracts[0].logging.redaction}}
- Audit events: {{contracts[0].logging.audit_events}} | OPTIONAL

10) Versioning / Compatibility

- Versioning policy: {{contracts[0].versioning.policy}}
- Deprecation policy: {{contracts[0].versioning.deprecation}} | OPTIONAL
- Upgrade cadence: {{contracts[0].versioning.cadence}} | OPTIONAL

11) Test Requirements

- Contract tests required: {{contracts[0].tests.contract_tests}}
- Mocking/stubbing notes: {{contracts[0].tests.mocking}} | OPTIONAL

Cross-References

- Upstream: {{xref:SIC-01}}, {{xref:ARC-07}} | OPTIONAL
- Downstream: {{xref:SIC-05}} | OPTIONAL, {{xref:ERR-05}} | OPTIONAL, {{xref:QA-03}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Auth + item list + schemas + basic error handling.
- intermediate: Required. Add validation rules and rate/timeout expectations.
- advanced: Required. Add versioning policy, audit requirements, and contract test expectations.

Unknown Handling

- UNKNOWN_ALLOWED: `example_payloads`, `sandbox_notes`, `sla`, `rate_limits`, `timeouts`, `notes`
- If auth method details or validation rules are UNKNOWN for an inbound interface → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION

- Pass conditions:
    - required_fields_present == true
    - every_interface_has_contract_block == true
    - auth_defined == true
    - schemas_defined == true
    - validation_defined == true
    - error_handling_defined == true
    - versioning_defined == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SIC-03

SIC-03 — Webhook Contract Spec (producer/consumer, retries, signatures)

Header Block

- template_id: SIC-03
- title: Webhook Contract Spec (producer/consumer, retries, signatures)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-03_Webhook_Contract_Spec.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["SIC-01", "ARC-07", "ERR-05"]
- inputs_required: ["SIC-01", "ARC-07", "ERR-05", "DGP-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the enforceable contract for webhooks (inbound and outbound): payload shape, signature verification, replay protection, retry semantics, idempotency, and operational behavior. This standardizes webhook behavior across vendors and internal producers.

Inputs Required

- SIC-01: {{xref:SIC-01}} | OPTIONAL
- ARC-07: {{xref:ARC-07}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Webhook list (minimum 3 if webhooks are used; otherwise mark N/A)
- For each webhook:
    - webhook_id
    - direction (inbound/outbound)
    - producer system
    - consumer system
    - endpoint URL pattern (for inbound) | OPTIONAL
    - event types supported
    - payload schema (typed)
    - signature scheme:
        - algorithm
        - secret/key reference (no raw secret)
        - header names
        - canonical signing string rules
    - replay protection:
        - timestamp header rules
        - tolerance window

        - nonce/idempotency key rule
    - delivery rules:
        - expected response codes
        - retry schedule/backoff
        - max attempts
        - dedupe rules
    - failure handling:
        - DLQ / quarantine rule
        - alerting rule
    - PII classification and redaction rules
    - observability fields (event_id, delivery_id, correlation_id)

Optional Fields

- Example payloads | OPTIONAL
- Vendor-specific quirks | OPTIONAL
- Notes | OPTIONAL

Rules

- Inbound webhooks must verify signature and replay protection before processing.
- Processing must be idempotent; define the idempotency key.
- Retry behavior must not cause duplicate side effects.
- Never log raw secrets or full sensitive payload fields; apply redaction.

- Outbound webhooks must include a stable event_id and delivery_id.

Output Format

1) Applicability

- applies: {{webhooks.applies}} (true/false)
- 00_NA (if not applies): {{webhooks.na_block}} | OPTIONAL

2) Webhook Contracts (canonical)

webhook_id	direction	producer	consumer	event_types	signature_scheme	replay_protection	idempotency_key	success_codes	retry_policy	max_attempts	pii_class	obs_fields	notes
wh_01	{{items[0].direction}}	{{items[0].producer}}	{{items[0].consumer}}	{{items[0].event_types}}	{{items[0].signature}}	{{items[0].replay}}	{{items[0].idempotency}}	{{items[0].success_codes}}	{{items[0].retry}}	{{items[0].max_attempts}}	{{items[0].pii}}	{{items[0].obs}}	{{items[0].notes}}

3) Signature Verification Rules (required if applies)

- Algorithm: {{signature.algorithm}}
- Headers: {{signature.headers}}
- Canonical string: {{signature.canonical_string_rule}}
- Secret reference: {{signature.secret_ref}}
- Failure behavior: {{signature.failure_behavior}}

4) Replay Protection Rules (required if applies)

- Timestamp header: {{replay.timestamp_header}}
- Tolerance window: {{replay.window}}
- Nonce rule: {{replay.nonce_rule}} | OPTIONAL
- Rejection behavior: {{replay.rejection_behavior}}

5) Retry & Idempotency Rules (required if applies)

- Retry schedule/backoff: {{retry.schedule}}
- Max attempts: {{retry.max_attempts}}
- Dedupe rule: {{retry.dedupe}}
- Idempotency key derivation: {{retry.idempotency_key_derivation}}

6) Failure Handling (required if applies)

- DLQ/quarantine: {{failure.dlq}}
- Alerting rule: {{failure.alerting}}
- Manual re-drive policy: {{failure.redrive}} | OPTIONAL

7) Observability & Redaction (required if applies)

- Required fields to log: {{obs.required_fields}}
- Redaction policy: {{obs.redaction}}

Cross-References

- Upstream: {{xref:SIC-01}} | OPTIONAL, {{xref:ARC-07}} | OPTIONAL

- Downstream: {{xref:SIC-05}} | OPTIONAL, {{xref:ERR-05}} | OPTIONAL, {{xref:OBS-01}} | OPTIONAL

- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required if applies. Define list + signature + retry basics.

- intermediate: Required if applies. Add replay protection + idempotency derivation.

- advanced: Required if applies. Add DLQ/redrive and observability/redaction rules.

Unknown Handling

- UNKNOWN_ALLOWED: `example_payloads`, `vendor_quirks`, `notes`, `nonce_rule`, `manual_redrive_policy`

- If applies == true and signature verification is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION

- Pass conditions:

    - required_fields_present == true

    - if_applies_then_webhooks_present == true

    - signature_rules_present == true

    - replay_protection_present == true

    - retry_idempotency_present == true

- placeholder_resolution == true
- no_unapproved_unknowns == true

SIC-04

SIC-04 — Data Mapping Contract (field mappings + transforms + validation)

Header Block

- template_id: SIC-04
- title: Data Mapping Contract (field mappings + transforms + validation)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-04_Data_Mapping_Contract.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["SIC-02", "DATA-01", "DMG-01"]
- inputs_required: ["SIC-02", "DATA-01", "DMG-01", "DGP-01", "ERR-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define deterministic field-level mappings between external interface schemas and internal canonical models: transforms, defaults, validation, and rejection rules. This prevents “silent mapping drift” and makes integration behavior auditable.

Inputs Required

- SIC-02: {{xref:SIC-02}} | OPTIONAL
- DATA-01: {{xref:DATA-01}} | OPTIONAL
- DMG-01: {{xref:DMG-01}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- One mapping block per interface + object type
- For each mapping:
    - mapping_id
    - interface_id
    - external_object
    - internal_entity_or_dto
    - direction (inbound/outbound)
    - field mapping table:
        - external_field
        - internal_field
        - type conversion
        - transform rule
        - default value rule
        - required/optional
        - validation rule
        - redaction/classification note (PII)
        - failure behavior (drop/reject/quarantine)

        - reason_code on failure (if reject)
    - normalization rules (dates, enums, currency, casing)
    - semantic validation rules (cross-field)
    - versioning notes (what happens when vendor adds fields)

Optional Fields

- Example input/output pairs | OPTIONAL
- Notes | OPTIONAL

Rules

- Required fields must not be silently defaulted unless explicitly allowed.
- Any transformation must be reversible or documented if lossy.
- If validation fails, behavior must be deterministic and mapped to reason codes.
- PII classification must be explicit; no hidden sensitive fields.

Output Format

Mapping: {{mappings[0].mapping_id}}

- interface_id: {{mappings[0].interface_id}}
- direction: {{mappings[0].direction}}
- external_object: {{mappings[0].external_object}}
- internal_entity_or_dto: {{mappings[0].internal_object}}

1) Field Map (canonical)

external_field	internal_field	type_conv	transform	default	required	validation	pii_note	failure_behavior	reason_code
{{mappings[0].fields[0].external}}	{{mappings[0].fields[0].internal}}	{{mappings[0].fields[0].type_conv}}	{{mappings[0].fields[0].transform}}	{{mappings[0].fields[0].default}}	{{mappings[0].fields[0].required}}	{{mappings[0].fields[0].validation}}	{{mappings[0].fields[0].pii}}	{{mappings[0].fields[0].failure}}	{{mappings[0].fields[0].reason_code}}

2) Normalization Rules (required)

- Dates/times: {{mappings[0].normalize.dates}}
- Enums: {{mappings[0].normalize.enums}}
- Currency/number formats: {{mappings[0].normalize.numbers}} | OPTIONAL
- Casing/whitespace: {{mappings[0].normalize.casing}} | OPTIONAL

3) Semantic Validation (required if any)

- {{mappings[0].semantic_validation[0]}} | OPTIONAL

4) Vendor Field Evolution Rules (required)

- Unknown fields behavior: {{mappings[0].evolution.unknown_fields}}
- New required fields behavior: {{mappings[0].evolution.new_required_fields}}
- Deprecation behavior: {{mappings[0].evolution.deprecations}} | OPTIONAL

5) Example Pairs (optional)

- Example input: {{mappings[0].examples.input}} | OPTIONAL
- Example output: {{mappings[0].examples.output}} | OPTIONAL

Cross-References

- Upstream: {{xref:SIC-02}} | OPTIONAL, {{xref:DATA-01}} | OPTIONAL, {{xref:DMG-01}} | OPTIONAL
- Downstream: {{xref:SIC-05}} | OPTIONAL, {{xref:QA-03}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Field map + basic transforms + required flags.
- intermediate: Required. Add deterministic failure behaviors + reason codes.
- advanced: Required. Add semantic validation and vendor evolution rules.

Unknown Handling

- UNKNOWN_ALLOWED: `example_pairs`, `notes`, `currency_formats`, `semantic_validation` (if none)
- If any required field has failure_behavior UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION
- Pass conditions:
    - required_fields_present == true
    - field_map_present == true
    - failure_behaviors_defined == true
    - pii_notes_present == true
    - evolution_rules_present == true

- placeholder_resolution == true
- no_unapproved_unknowns == true

SIC-05

SIC-05 — Integration Failure Modes & Recovery (timeouts, retries, DLQ, fallbacks)

Header Block

- template_id: SIC-05
- title: Integration Failure Modes & Recovery (timeouts, retries, DLQ, fallbacks)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-05_Integration_Failure_Recovery.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["SIC-01", "SIC-02", "SIC-03", "ERR-05", "RELIA-01"]
- inputs_required: ["SIC-01", "SIC-02", "SIC-03", "ERR-05", "RELIA-01", "OBS-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the deterministic failure handling model for integrations: what can fail, how we detect it, how we retry, when we stop, what goes to DLQ/quarantine, and what user/system fallbacks apply. This ensures integrations fail safely and recover predictably.

Inputs Required

- SIC-01: {{xref:SIC-01}} | OPTIONAL

- SIC-02: {{xref:SIC-02}} | OPTIONAL
- SIC-03: {{xref:SIC-03}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL
- RELIA-01: {{xref:RELIA-01}} | OPTIONAL
- OBS-04: {{xref:OBS-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Failure mode catalog (minimum 10)
- For each failure mode:
    - fail_id
    - interface_id
    - operation (endpoint/event/webhook/job)
    - failure_type (timeout/5xx/4xx/schema/signature/rate_limit/network/partial)
    - detection method (status/timeout/validation)
    - classification (transient/permanent/unknown)
    - retry policy (none/immediate/backoff/scheduled)
    - max attempts
    - idempotency requirement (yes/no + key)
    - DLQ/quarantine rule
    - fallback behavior (system + user-facing)
    - alerting rule + severity

- observability fields required
- reason_code mapping (if exposed to client)

Optional Fields

- Vendor escalation runbook pointer | OPTIONAL
- Manual backfill/reconciliation steps | OPTIONAL
- Notes | OPTIONAL

Rules

- Transient failures must use bounded backoff; never infinite retry.
- Permanent failures must not retry; must quarantine or reject deterministically.
- Any retryable operation must be idempotent (define key).
- DLQ entries must be re-drivable with safety checks.
- User-facing fallbacks must align with DES-07/CDX-04 when applicable.

Output Format

1) Failure Modes Catalog (canonical)

fail_id	**interface_id**	**operation**	**failure_type**	**detection**	**class**	**retry_policy**	**max_attempts**	**idempotency**	**dlq_rule**	**fallback**	**alerting**	**obs_fields**	**reason_code**	**notes**
f_	{{fails[0].interfac	{{fails[0].oper	{{fails[0].t	{{fails[0].dete	{{fails[0].c	{{fails[0].ret	{{fails[0].	{{fails[0].idempo	{{fails[0	{{fails[0].fallb	{{fails[0].alert	{{fails[0	{{fails[0].reason	{{fails[0].n

01	e_id}}	ation}}	ype}}	ction}}	lass}}	ry_policy}}	max}}	tency}}	].dlq}}	ack}}	ing}}	].obs}}	_code}}	otes}}
f_02	{{fails[1].interface_id}}	{{fails[1].operation}}	{{fails[1].type}}	{{fails[1].detection}}	{{fails[1].class}}	{{fails[1].retry_policy}}	{{fails[1].max}}	{{fails[1].idempotency}}	{{fails[1].dlq}}	{{fails[1].fallback}}	{{fails[1].alerting}}	{{fails[1].obs}}	{{fails[1].reason_code}}	{{fails[1].notes}}

2) Retry Policy Defaults (required)

- Backoff strategy: {{retry.defaults.backoff}}
- Jitter rule: {{retry.defaults.jitter}} | OPTIONAL
- Max attempts default: {{retry.defaults.max_attempts}}
- When to switch to DLQ: {{retry.defaults.dlq_threshold}}

3) DLQ / Quarantine Rules (required)

- DLQ entry must include: {{dlq.required_fields}}
- Re-drive conditions: {{dlq.redrive_conditions}}
- Re-drive safety checks: {{dlq.safety_checks}}
- Data retention window: {{dlq.retention}} | OPTIONAL

4) Alerts & SLO Impact (required)

alert_type	trigger	severity	response_sla	owner
integration_down	{{alerts.down.trigger}}	{{alerts.down.severity}}	{{alerts.down.sla}}	{{alerts.down.owner}}
dlq_growth	{{alerts.dlq.trigger}}	{{alerts.dlq.severity}}	{{alerts.dlq.sla}}	{{alerts.dlq.owner}}

5) User-facing Fallback Guidance (required if user-visible)

- Copy pointer: {{xref:CDX-04}} | OPTIONAL
- UX surface pointer: {{xref:DES-07}} | OPTIONAL

Cross-References

- Upstream: {{xref:SIC-02}} | OPTIONAL, {{xref:SIC-03}} | OPTIONAL, {{xref:ERR-05}} | OPTIONAL
- Downstream: {{xref:OPS-06}} | OPTIONAL, {{xref:IRP-*}} | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Failure catalog + retry defaults + DLQ basics.
- intermediate: Required. Add idempotency keys and alerting rules.
- advanced: Required. Add re-drive safety checks and SLO impact mapping.

Unknown Handling

- UNKNOWN_ALLOWED: `vendor_escalation_runbook`, `manual_reconciliation`, `notes`, `retention_window`
- If any retryable operation lacks idempotency definition → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION
- Pass conditions:

- required_fields_present == true
- failure_modes_count >= 10
- retry_defaults_present == true
- dlq_rules_present == true
- alerting_present == true
- retryables_have_idempotency == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

SIC-06

SIC-06 — Vendor/Third-Party Trust Model (data sharing, scopes, audit needs)

Header Block

- template_id: SIC-06
- title: Vendor/Third-Party Trust Model (data sharing, scopes, audit needs)
- type: system_interfaces_integration_contracts
- template_version: 1.0.0
- output_path: 10_app/integrations/SIC-06_Vendor_Trust_Model.md
- compliance_gate_id: TMP-05.PRIMARY.INTEGRATION
- upstream_dependencies: ["SIC-01", "ARC-07", "DGP-01", "COMP-05"]
- inputs_required: ["SIC-01", "ARC-07", "DGP-01", "SEC-02", "COMP-05", "AUDIT-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how we trust and govern third-party vendors: what data they can access, what scopes/permissions apply, how we audit their access, and what compliance constraints exist. This prevents accidental over-sharing and supports vendor risk management.

Inputs Required

- SIC-01: {{xref:SIC-01}} | OPTIONAL
- ARC-07: {{xref:ARC-07}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL

- SEC-02: {{xref:SEC-02}} | OPTIONAL
- COMP-05: {{xref:COMP-05}} | OPTIONAL
- AUDIT-01: {{xref:AUDIT-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Vendor list (from SIC-01) with trust tiers
- For each vendor:
    - vendor_id (align to interface_id/ext_id)
    - trust tier (low/medium/high)
    - data access scope (categories + PII class)
    - auth scope model (OAuth scopes/keys/mTLS)
    - least-privilege constraints
    - audit requirements (events to log, retention, review cadence)
    - data retention/sharing rules
    - breach notification expectations (if known)
    - exit strategy (how to revoke and remove data)
- Review cadence (access reviews, vendor reviews)
- Compliance constraints summary (data residency, HIPAA/PCI/etc if applicable)

Optional Fields

- Vendor scoring rubric | OPTIONAL

- Contracts/SLA pointers | OPTIONAL
- Notes | OPTIONAL

Rules

- Least privilege is mandatory: no broad scopes without justification.
- High-trust vendors still require auditability; trust does not remove logging.
- Exit strategy must be defined for every vendor with data access.
- Any high-PII sharing requires explicit approval and retention rules.

Output Format

1) Trust Tiers (required)

tier	meaning	typical_controls	approval_required
low	{{tiers.low.meaning}}	{{tiers.low.controls}}	{{tiers.low.approval}}
medium	{{tiers.med.meaning}}	{{tiers.med.controls}}	{{tiers.med.approval}}
high	{{tiers.high.meaning}}	{{tiers.high.controls}}	{{tiers.high.approval}}

2) Vendor Trust Registry (canonical)

vendor_id	interface_id	trust_tier	data_scope	pii_class	auth_scopes	least_privilege_rules	audit_requirements	retention_rules	exit_strategy	review_cadence	notes
v_01	{{vendors[0].interface_id}}	{{vendors[0].tier}}	{{vendors[0].data_scope}}	{{vendors[0].pii}}	{{vendors[0].auth_scopes}}	{{vendors[0].least_priv}}	{{vendors[0].audit}}	{{vendors[0].retention}}	{{vendors[0].exit}}	{{vendors[0].review}}	{{vendors[0].notes}}

3) Review Cadence (required)

- Access review cadence: {{cadence.access_review}}
- Vendor review cadence: {{cadence.vendor_review}}
- Audit evidence required: {{cadence.evidence}} | OPTIONAL

4) Compliance Constraints (required if any)

- Data residency constraints: {{compliance.data_residency}} | OPTIONAL
- Regulated data constraints: {{compliance.regulated}} | OPTIONAL
- Breach notification expectations: {{compliance.breach_notify}} | OPTIONAL

Cross-References

- Upstream: {{xref:SIC-01}} | OPTIONAL, {{xref:DGP-01}} | OPTIONAL, {{xref:COMP-05}} | OPTIONAL
- Downstream: {{xref:GOVOPS-*}}* | *OPTIONAL, {{xref:COMP-02}}* | *OPTIONAL, {{xref:AUDIT-}}* | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Vendor list + trust tier + data scope + exit strategy.
- intermediate: Required. Add least privilege rules and audit requirements.
- advanced: Required. Add review cadence and compliance constraints.

Unknown Handling

- UNKNOWN_ALLOWED: `breach_notification`, `contracts_pointers`, `notes`, `vendor_scoring_rubric`
- If exit_strategy is UNKNOWN for any vendor with data access → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.INTEGRATION
- Pass conditions:
  - required_fields_present == true
  - vendors_present == true
  - trust_tier_present == true
  - least_privilege_present == true
  - audit_requirements_present == true
  - exit_strategies_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

Service Boundaries & Deployment Topology (SBDT)

- Service Boundaries & Deployment Topology (SBDT)
  SBDT-01 Service Boundary Map (responsibilities, ownership, dependencies)
  SBDT-02 Runtime Topology (services, workers, queues, storage, networks)
  SBDT-03 Environment Topology (dev/stage/prod parity, isolation rules)
  SBDT-04 Scaling Model (horizontal/vertical, bottlenecks, capacity assumptions)
  SBDT-05 Resilience Topology (redundancy, failover, circuit boundaries)
  SBDT-06 Deployment Constraints (rollouts, canary, migration safety)

SBDT-01

SBDT-01 — Service Boundary Map (responsibilities, ownership, dependencies)

Header Block

- template_id: SBDT-01
- title: Service Boundary Map (responsibilities, ownership, dependencies)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-01_Service_Boundary_Map.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["ARC-01", "ARC-08", "RISK-03"]
- inputs_required: ["ARC-01", "ARC-08", "RISK-03", "STK-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the service boundary decomposition: which runtime services/workers exist, what each owns, how they depend on each other, and who is accountable. This is the enforceable boundary map used to prevent “service sprawl” and unclear ownership.

Inputs Required

- ARC-01: {{xref:ARC-01}} | OPTIONAL
- ARC-08: {{xref:ARC-08}} | OPTIONAL

- RISK-03: {{xref:RISK-03}} | OPTIONAL
- STK-01: {{xref:STK-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Service list (minimum 3 for non-trivial systems)
- For each service:
    - service_id
    - name
    - service_type (api/service/worker/gateway/realtime)
    - owner_boundary_id (ARC-01)
    - responsibilities (bullets)
    - owns (data/entities/events)
    - exposes (APIs/topics/queues)
    - depends_on (other services + external deps)
    - trust zone (public/private/internal)
    - SLA criticality (P0/P1/P2)
    - primary failure modes (top 3)
- Dependency directionality rules (no cycles unless justified)
- Ownership policy (how ownership changes)

Optional Fields

- Diagram pointer | OPTIONAL
- Notes | OPTIONAL

Rules

- Each owned entity/event must map back to a single boundary owner.
- Each dependency must state purpose; “depends on X” without why is not allowed.
- Any dependency cycle must be explicitly documented with mitigation.
- Public exposure must be explicit (default private).

Output Format

1) Services Registry (canonical)

service_id	name	type	owner_boundary_id	owns	exposes	depends_on	trust_zone	criticality	failure_modes	notes
svc_01	{{services[0].name}}	{{services[0].type}}	{{services[0].owner}}	{{services[0].owns}}	{{services[0].exposes}}	{{services[0].deps}}	{{services[0].zone}}	{{services[0].crit}}	{{services[0].fails}}	{{services[0].notes}}
svc_02	{{services[1].name}}	{{services[1].type}}	{{services[1].owner}}	{{services[1].owns}}	{{services[1].exposes}}	{{services[1].deps}}	{{services[1].zone}}	{{services[1].crit}}	{{services[1].fails}}	{{services[1].notes}}

2) Responsibility Blocks (required, one per service)

{{services[0].service_id}} — {{services[0].name}}

- Responsibilities:

    - {{services[0].responsibilities[0]}}
    - {{services[0].responsibilities[1]}} | OPTIONAL
- Owns: {{services[0].owns}}
- Exposes: {{services[0].exposes}}
- Dependencies: {{services[0].deps}}
- Failure modes: {{services[0].fails}}

3) Dependency Rules (required)

- No-cycle policy: {{deps.no_cycle_policy}}
- If cycle exists, required mitigations: {{deps.cycle_mitigations}} | OPTIONAL
- External dependency policy: {{deps.external_policy}} | OPTIONAL

4) Ownership Policy (required)

- Ownership assignment rule: {{ownership.assignment_rule}}
- Ownership change process: {{ownership.change_process}}

Cross-References

- Upstream: {{xref:ARC-01}} | OPTIONAL, {{xref:ARC-08}} | OPTIONAL
- Downstream: {{xref:SBDT-02}}, {{xref:SBDT-05}} | OPTIONAL, {{xref:RELIA-01}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Services list with responsibilities and dependencies.

- intermediate: Required. Add owns/exposes and criticality.
- advanced: Required. Add failure modes and ownership policy.

Unknown Handling

- UNKNOWN_ALLOWED: `diagram_pointer`, `notes`
- If any service lacks owner_boundary_id → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - services_count >= 3
    - every_service_has_owner == true
    - dependencies_defined == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SBDT-02

SBDT-02 — Runtime Topology (services, workers, queues, storage, networks)

Header Block

- template_id: SBDT-02
- title: Runtime Topology (services, workers, queues, storage, networks)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-02_Runtime_Topology.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["SBDT-01", "ARC-08", "OPS-02"]
- inputs_required: ["SBDT-01", "ARC-08", "OPS-02", "ENV-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the runtime topology: the concrete runtime components and how they connect (services, workers, queues, caches, databases, storage, gateways) including network zones and data flow direction.

Inputs Required

- SBDT-01: {{xref:SBDT-01}} | OPTIONAL
- ARC-08: {{xref:ARC-08}} | OPTIONAL
- OPS-02: {{xref:OPS-02}} | OPTIONAL

- ENV-01: {{xref:ENV-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Component inventory (minimum: api, db, storage; plus queues/workers if used)
- For each component:
    - component_id
    - type
    - owner_service_id
    - network_zone (public/private/internal)
    - communicates_with (list)
    - protocols (http/ws/grpc/queue)
    - data stores accessed (if any)
    - secrets dependency pointer
    - scaling model (basic)
- Network segmentation rules (allowed paths)
- Data flow table (major flows)
- Health/readiness expectations (high level)
- “Single points of failure” list (if any) and mitigation pointer

Optional Fields

- Diagram pointer | OPTIONAL

- Notes | OPTIONAL

Rules

- All communications must be declared; undeclared connections are disallowed.
- Public ingress must be through explicit gateways/edges.
- Secrets are never embedded; reference secrets management.
- Any SPOF must have a mitigation plan or documented acceptance.

Output Format

1) Component Inventory (canonical)

component_id	type	owner_service_id	zone	communicates_with	protocols	datastores	secrets_ref	scaling	health_notes	notes
comp_01	{{components[0].type}}	{{components[0].owner}}	{{components[0].zone}}	{{components[0].comms}}	{{components[0].protocols}}	{{components[0].datastores}}	{{components[0].secrets}}	{{components[0].scaling}}	{{components[0].health}}	{{components[0].notes}}

2) Network Segmentation Rules (required)

- Allowed ingress paths: {{network.ingress}}
- Allowed egress paths: {{network.egress}}
- Deny-by-default: {{network.deny_default}}

3) Major Data Flows (required)

flow_id	source_component	dest_component	protocol	data_category	notes
flow_01	{{flows[0].src}}	{{flows[0].dst}}	{{flows[0].protocol}}	{{flows[0].data}}	{{flows[0].notes}}

4) Health/Readiness Expectations (required)

- Readiness checks required: {{health.readiness}}
- Liveness checks required: {{health.liveness}} | OPTIONAL
- Dependency health propagation: {{health.dependency_propagation}} | OPTIONAL

5) SPOFs (required if any)

spof_id	component_id	why_spof	mitigation	accepted_by
spof_01	{{spofs[0].component}}	{{spofs[0].why}}	{{spofs[0].mitigation}}	{{spofs[0].accepted_by}}

Cross-References

- Upstream: {{xref:SBDT-01}} | OPTIONAL, {{xref:ARC-08}} | OPTIONAL
- Downstream: {{xref:SBDT-05}} | OPTIONAL, {{xref:RELIA-02}} | OPTIONAL, {{xref:OPS-05}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Inventory + zones + major flows.
- intermediate: Required. Add segmentation rules and health expectations.
- advanced: Required. Add SPOFs list and mitigation pointers.

Unknown Handling

- UNKNOWN_ALLOWED: `diagram_pointer`, `notes`, `spofs` (if none, explicitly state none)
- If any public component lacks defined ingress path → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - component_inventory_present == true
    - segmentation_rules_present == true
    - major_flows_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SBDT-03

SBDT-03 — Environment Topology (dev/stage/prod parity, isolation rules)

Header Block

- template_id: SBDT-03
- title: Environment Topology (dev/stage/prod parity, isolation rules)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-03_Environment_Topology.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["ARC-08", "ENV-01", "OPS-01"]
- inputs_required: ["ARC-08", "ENV-01", "OPS-01", "DGP-01", "SEC-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how environments are structured and isolated (dev/stage/prod), what parity guarantees exist, how data is handled in each environment, and what isolation/network rules apply. This ensures safe testing without contaminating production data or credentials.

Inputs Required

- ARC-08: {{xref:ARC-08}} | OPTIONAL
- ENV-01: {{xref:ENV-01}} | OPTIONAL
- OPS-01: {{xref:OPS-01}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Environment list (dev/stage/prod at minimum unless explicitly N/A)
- For each environment:
    - env_id
    - purpose
    - access policy (who can access)
    - data policy (synthetic vs real, PII rules)
    - secrets policy (separate keys, rotation expectations)
    - service parity expectations (what must match prod)
    - isolation rules (network, accounts, DB separation)
    - integrations policy (real vendors vs sandbox)
    - observability policy (retention/sampling)
- Promotion rules (dev → stage → prod) (high level)
- “No prod data in non-prod” rule (explicit) (or exception policy)

Optional Fields

- Preview environments (per PR) | OPTIONAL
- Notes | OPTIONAL

Rules

- Environments must use different secrets; no shared production credentials.
- Production data must not be copied to lower environments unless sanitized and approved.
- Sandbox integrations must be used in non-prod by default.
- Parity must cover critical configs and schema versions.

Output Format

1) Environment Matrix (canonical)

env_id	purpose	access_policy	data_policy	secrets_policy	parity_expectations	isolation_rules	integrations_policy	obs_policy	notes
dev	{{envs.dev.purpose}}	{{envs.dev.access}}	{{envs.dev.data}}	{{envs.dev.secrets}}	{{envs.dev.parity}}	{{envs.dev.isolation}}	{{envs.dev.integrations}}	{{envs.dev.obs}}	{{envs.dev.notes}}
stage	{{envs.stage.purpose}}	{{envs.stage.access}}	{{envs.stage.data}}	{{envs.stage.secrets}}	{{envs.stage.parity}}	{{envs.stage.isolation}}	{{envs.stage.integrations}}	{{envs.stage.obs}}	{{envs.stage.notes}}
prod	{{envs.prod.purpose}}	{{envs.prod.access}}	{{envs.prod.data}}	{{envs.prod.secrets}}	{{envs.prod.parity}}	{{envs.prod.isolation}}	{{envs.prod.integrations}}	{{envs.prod.obs}}	{{envs.prod.notes}}

2) Promotion Rules (required)

- Dev → Stage rule: {{promotion.dev_to_stage}}
- Stage → Prod rule: {{promotion.stage_to_prod}}

- Approval gates: {{promotion.approvals}} | OPTIONAL

3) Data Handling Rules (required)

- No prod data in non-prod: {{data.no_prod_in_nonprod}}
- Sanitization requirement (if any copy): {{data.sanitization}} | OPTIONAL
- Exception policy: {{data.exception_policy}} | OPTIONAL

Cross-References

- Upstream: {{xref:ENV-01}} | OPTIONAL, {{xref:ARC-08}} | OPTIONAL
- Downstream: {{xref:OPS-04}} | OPTIONAL, {{xref:CICD-*}}* | *OPTIONAL, {{xref:BDR-}}* | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Matrix with data + secrets separation rules.
- intermediate: Required. Add parity and integration sandbox rules.
- advanced: Required. Add promotion gates and explicit exception policies.

Unknown Handling

- UNKNOWN_ALLOWED: `preview_envs`, `notes`, `obs_policy_details`, `promotion.approvals`
- If secrets_policy is UNKNOWN for any environment → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - environment_matrix_present == true
    - secrets_separation_defined == true
    - data_policy_defined == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SBDT-04

SBDT-04 — Scaling Model (horizontal/vertical, bottlenecks, capacity assumptions)

Header Block

- template_id: SBDT-04
- title: Scaling Model (horizontal/vertical, bottlenecks, capacity assumptions)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-04_Scaling_Model.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["SBDT-02", "PERF-02", "COST-01"]
- inputs_required: ["SBDT-02", "PERF-02", "LOAD-01", "COST-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": false, "intermediate": true, "advanced": true}

Purpose

Define the scaling strategy and capacity assumptions: how each major component scales, what bottlenecks exist, what capacity targets are assumed, and what triggers scaling or re-architecture decisions.

Inputs Required

- SBDT-02: {{xref:SBDT-02}} | OPTIONAL

- PERF-02: {{xref:PERF-02}} | OPTIONAL
- LOAD-01: {{xref:LOAD-01}} | OPTIONAL
- COST-01: {{xref:COST-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Applicability (true/false). If false, mark N/A.
- Scaling assumptions:
    - peak concurrent users
    - request rate
    - message rate (if realtime)
    - storage growth
- Component scaling plan (for each major component):
    - scaling axis (CPU/mem/instances/partitions)
    - horizontal vs vertical stance
    - known bottlenecks
    - caching strategy touchpoints
    - DB scaling posture (read replicas, sharding, etc.) (high level)
- Trigger thresholds (what causes action)
- Risks and mitigations (high level)
- Monitoring requirements (what to watch)

Optional Fields

- Future multi-region scaling notes | OPTIONAL
- Notes | OPTIONAL

Rules

- If applies == false, include 00_NA block only.
- Assumptions must be explicitly stated; “unknown” must include a plan to measure.
- Scaling triggers must be measurable (p95 latency, queue depth, CPU, error rate).
- Bottlenecks must map to mitigation actions (cache, partition, queue, optimize).

Output Format

1) Applicability

- applies: {{scale.applies}} (true/false)
- 00_NA (if not applies): {{scale.na_block}} | OPTIONAL

2) Capacity Assumptions (required if applies)

- Peak concurrent users: {{assumptions.concurrent_users}} | OPTIONAL
- Peak RPS: {{assumptions.rps}} | OPTIONAL
- Realtime msg rate: {{assumptions.msg_rate}} | OPTIONAL
- Storage growth: {{assumptions.storage_growth}} | OPTIONAL

3) Component Scaling Plan (required if applies)

component_id	scaling_axis	strategy (H/V)	bottlenecks	mitigations	triggers	monitoring

{{components[0].id}}	{{components[0].axis}}	{{components[0].strategy}}	{{components[0].bottlenecks}}	{{components[0].mitigations}}	{{components[0].triggers}}	{{components[0].monitoring}}

4) Trigger Thresholds (required if applies)

trigger_id	metric	threshold	action	owner
trg_01	{{triggers[0].metric}}	{{triggers[0].threshold}}	{{triggers[0].action}}	{{triggers[0].owner}}

5) Risks & Mitigations (required if applies)

- {{risks[0]}}
- {{risks[1]}} | OPTIONAL

Cross-References

- Upstream: {{xref:SBDT-02}} | OPTIONAL, {{xref:PERF-02}} | OPTIONAL, {{xref:COST-01}} | OPTIONAL
- Downstream: {{xref:PERF-05}} | OPTIONAL, {{xref:RELIA-02}} | OPTIONAL, {{xref:OPS-05}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Not required.
- intermediate: Required if applies. Define assumptions + scaling table.
- advanced: Required if applies. Add triggers and monitoring.

Unknown Handling

- UNKNOWN_ALLOWED: `assumption_values`, `future_multi_region_notes`, `notes`
- If applies == true and triggers are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - if_applies_then_scaling_plan_present == true
    - if_applies_then_triggers_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SBDT-05

SBDT-05 — Resilience Topology (redundancy, failover, circuit boundaries)

Header Block

- template_id: SBDT-05
- title: Resilience Topology (redundancy, failover, circuit boundaries)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-05_Resilience_Topology.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["SBDT-02", "RELIA-02", "SLO-01", "BDR-01"]
- inputs_required: ["SBDT-02", "RELIA-02", "SLO-01", "BDR-01", "OBS-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": false, "intermediate": true, "advanced": true}

Purpose

Define how resilience is achieved at the topology level: redundancy, failover posture, circuit boundaries, graceful degradation, and recovery objectives. This is the architecture-to-operations bridge for reliability.

Inputs Required

- SBDT-02: {{xref:SBDT-02}} | OPTIONAL
- RELIA-02: {{xref:RELIA-02}} | OPTIONAL
- SLO-01: {{xref:SLO-01}} | OPTIONAL

- BDR-01: {{xref:BDR-01}} | OPTIONAL
- OBS-04: {{xref:OBS-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Applicability (true/false). If false, mark N/A.
- Critical dependency inventory (DB, cache, queues, vendors)
- For each critical dependency:
    - dependency_id
    - component_id
    - redundancy posture (none/single-AZ/multi-AZ/multi-region)
    - failover method (automatic/manual)
    - circuit breaker boundary (where trips)
    - degradation behavior (read-only, queue, cached, disabled feature)
    - RTO/RPO targets (if applicable)
    - monitoring/alerting requirement
- Graceful degradation rules per user-facing area
- Recovery playbook pointers

Optional Fields

- Chaos testing plan pointer | OPTIONAL
- Notes | OPTIONAL

Rules

- If applies == false, include 00_NA block only.
- Every P0 critical dependency must have a degradation behavior defined.
- Failover method must be explicit; “we’ll handle it” is not allowed.
- Circuit breakers must have clear thresholds and reset policy (pointer ok).

Output Format

1) Applicability

- applies: {{resilience.applies}} (true/false)
- 00_NA (if not applies): {{resilience.na_block}} | OPTIONAL

2) Critical Dependencies (canonical)

dependency_id	component_id	redundancy	failover	circuit_boundary	degradation_behavior	rto_rpo	monitoring	notes
dep_01	{{deps[0].component_id}}	{{deps[0].redundancy}}	{{deps[0].failover}}	{{deps[0].circuit}}	{{deps[0].degradation}}	{{deps[0].rto_rpo}}	{{deps[0].monitoring}}	{{deps[0].notes}}

3) Degradation Rules (required if applies)

user_area_or_feature	dependency	degraded_mode	user_message_pointer	recovery_condition
{{degrade[0].area}}	{{degrade[0].dep}}	{{degrade[0].mode}}	{{degrade[0].msg_ptr}}	{{degrade[0].recovery}}

4) Recovery Playbooks (required if applies)

- DB recovery pointer: {{playbooks.db}} | OPTIONAL

- Queue recovery pointer: {{playbooks.queue}} | OPTIONAL
- Vendor outage pointer: {{playbooks.vendor}} | OPTIONAL
- Incident process pointer: {{playbooks.incident}} | OPTIONAL

5) Monitoring/Alerting Requirements (required if applies)

- Required alerts: {{alerts.required}}
- Paging vs ticketing rules: {{alerts.paging_rules}} | OPTIONAL

Cross-References

- Upstream: {{xref:RELIA-02}} | OPTIONAL, {{xref:SLO-01}} | OPTIONAL, {{xref:BDR-01}} | OPTIONAL
- Downstream: {{xref:IRP-01}} | OPTIONAL, {{xref:RELIA-05}} | OPTIONAL, {{xref:LOAD-*}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Not required.
- intermediate: Required if applies. Define dependencies + degradation behavior.
- advanced: Required if applies. Add circuit boundaries and recovery pointers.

Unknown Handling

- UNKNOWN_ALLOWED: `chaos_testing_pointer`, `notes`, `rto_rpo` (if not set yet but must be planned)

- If applies == true and any P0 dependency lacks degradation_behavior → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - if_applies_then_deps_present == true
    - if_applies_then_degradation_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

SBDT-06

SBDT-06 — Deployment Constraints (rollouts, canary, migration safety)

Header Block

- template_id: SBDT-06
- title: Deployment Constraints (rollouts, canary, migration safety)
- type: service_boundaries_deployment_topology
- template_version: 1.0.0
- output_path: 10_app/topology/SBDT-06_Deployment_Constraints.md
- compliance_gate_id: TMP-05.PRIMARY.TOPOLOGY
- upstream_dependencies: ["ARC-08", "REL-01", "REL-04", "DATA-04"]
- inputs_required: ["ARC-08", "REL-01", "REL-04", "DATA-04", "CICD-03", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the hard constraints and required practices for safe deployment: rollout strategy, canary/phased deploy rules, migration safety, backward compatibility requirements, and rollback posture. This prevents unsafe releases that break running systems.

Inputs Required

- ARC-08: {{xref:ARC-08}} | OPTIONAL
- REL-01: {{xref:REL-01}} | OPTIONAL
- REL-04: {{xref:REL-04}} | OPTIONAL

- DATA-04: {{xref:DATA-04}} | OPTIONAL
- CICD-03: {{xref:CICD-03}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Rollout strategy policy (default)
- Canary/phased deploy rules (when required)
- Backward compatibility rules during rollout (old+new running together)
- Migration safety rules:
    - expand/contract approach (if applicable)
    - ordering constraints (code vs schema)
    - migration verification requirements
- Rollback rules:
    - what can be rolled back safely
    - what cannot (irreversible migrations)
    - rollback triggers
- Required pre-deploy checks (gate checklist pointer)
- Required post-deploy verification steps

Optional Fields

- Feature flag rollout pointers | OPTIONAL
- Notes | OPTIONAL

Rules

- Any rollout that runs mixed versions must guarantee compatibility.
- Schema changes must be compatible with both old and new code during transition.
- Rollback posture must be explicit before shipping.
- If a migration is irreversible, it must require explicit approval.

Output Format

1) Default Rollout Strategy (required)

- Default strategy: {{rollout.default}} (all-at-once/canary/phased/blue-green)
- When canary is mandatory: {{rollout.canary_when_required}}
- Abort conditions: {{rollout.abort_conditions}} | OPTIONAL

2) Compatibility Rules (required)

- Mixed-version compatibility rule: {{compat.mixed_version}}
- API compatibility rule: {{compat.api}}
- Event/message compatibility rule: {{compat.events}} | OPTIONAL

3) Migration Safety Rules (required)

- Approach: {{migrations.approach}} (expand/contract, etc.)
- Ordering: {{migrations.ordering}} (schema first vs code first)
- Verification: {{migrations.verification}}
- Data backfill rule: {{migrations.backfill}} | OPTIONAL

4) Rollback Rules (required)

- Rollback triggers: {{rollback.triggers}}
- Safe rollback actions: {{rollback.safe_actions}}
- Unsafe/blocked rollback cases: {{rollback.unsafe_cases}}
- Roll-forward rule (when rollback not possible): {{rollback.roll_forward}} | OPTIONAL

5) Pre/Post Deploy Checks (required)

- Pre-deploy gate pointer: {{checks.predeploy_gate_pointer}}
- Post-deploy verification steps: {{checks.postdeploy_steps}}

Cross-References

- Upstream: {{xref:REL-01}} | OPTIONAL, {{xref:DATA-04}} | OPTIONAL, {{xref:CICD-03}} | OPTIONAL
- Downstream: {{xref:RELOPS-02}} | OPTIONAL, {{xref:QA-05}} | OPTIONAL, {{xref:IRP-01}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Default rollout + rollback triggers + pre/post checks.
- intermediate: Required. Add compatibility and migration ordering rules.
- advanced: Required. Add abort conditions and irreversible migration approval rules.

Unknown Handling

- UNKNOWN_ALLOWED: `feature_flag_pointers`, `abort_conditions`, `notes`, `backfill_rules`

- If migration safety ordering is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.TOPOLOGY
- Pass conditions:
    - required_fields_present == true
    - rollout_strategy_present == true
    - compatibility_rules_present == true
    - migration_safety_present == true
    - rollback_rules_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

Permission Model & Authorization Design (PMAD)

- Permission Model & Authorization Design (PMAD)
  PMAD-01 Permission Model Overview (roles, resources, actions)
  PMAD-02 AuthZ Policy Rules (RBAC/ABAC, inheritance, exceptions)
  PMAD-03 Enforcement Points Map (UI/API/service/DB)
  PMAD-04 Permission Check Patterns (standard decision flow + reason codes)
  PMAD-05 Privileged Operations Policy (admin/mod/support actions, approvals)
  PMAD-06 Audit Requirements for AuthZ (what must be logged)

PMAD-01

PMAD-01 — Permission Model Overview (roles, resources, actions)

Header Block

- template_id: PMAD-01
- title: Permission Model Overview (roles, resources, actions)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-01_Permission_Model_Overview.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PRD-03", "BRP-02", "DMG-02", "ARC-04"]
- inputs_required: ["PRD-03", "BRP-02", "DMG-02", "IAM-03", "ARC-04", "STK-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical permission model: roles, resources, actions, and high-level policy principles. This is the system-wide authorization vocabulary and structure that all enforcement points must follow.

Inputs Required

- PRD-03: {{xref:PRD-03}} | OPTIONAL
- BRP-02: {{xref:BRP-02}} | OPTIONAL
- DMG-02: {{xref:DMG-02}} | OPTIONAL

- IAM-03: {{xref:IAM-03}} | OPTIONAL
- ARC-04: {{xref:ARC-04}} | OPTIONAL
- STK-01: {{xref:STK-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Role model:
    - role_id list (canonical)
    - role descriptions
    - role hierarchy/inheritance rules (if any)
- Resource model:
    - resource_id list (canonical)
    - resource ownership/tenancy rules (org/user scoped)
- Action model:
    - action_id list (canonical verbs: read/create/update/delete/approve/moderate/etc.)
    - action semantics (what each means)
- Permission expression model (how rules are written):
    - RBAC and/or ABAC statement
    - conditions vocabulary (owner, org_member, scope, status, etc.)
- Default-deny policy statement
- System-wide constraints:
    - least privilege

- separation of duties (if applicable)
- sensitive operation classification

Optional Fields

- Multi-tenant model notes | OPTIONAL
- “Guest/Visitor” access rules | OPTIONAL
- Notes | OPTIONAL

Rules

- Do not invent roles/resources/actions that conflict with PRD-03/DMG.
- Every permission rule must be expressible using the defined action/resource vocabulary.
- Default-deny applies when policy cannot be evaluated.
- Sensitive operations must require explicit permissions and audit logging.

Output Format

1) Roles (canonical)

role_id	name	description	inherits_from	notes
{{roles[0].id}}	{{roles[0].name}}	{{roles[0].desc}}	{{roles[0].inherits}}	{{roles[0].notes}}
{{roles[1].id}}	{{roles[1].name}}	{{roles[1].desc}}	{{roles[1].inherits}}	{{roles[1].notes}}

2) Resources (canonical)

resource_id	description	scope (user/org/global)	tenancy_key	notes
{{resources[0].id}}	{{resources[0].desc}}	{{resources[0].scope}}	{{resources[0].tenancy_key}}	{{resources[0].notes}}

3) Actions (canonical)

action_id	description	notes
read	{{actions.read.desc}}	{{actions.read.notes}}
create	{{actions.create.desc}}	{{actions.create.notes}}
update	{{actions.update.desc}}	{{actions.update.notes}}
delete	{{actions.delete.desc}}	{{actions.delete.notes}}

4) Permission Expression Model (required)

- Model type: {{policy.model_type}} (RBAC/ABAC/hybrid)
- Condition vocabulary: {{policy.conditions}}
- Example rule form: {{policy.example_rule_form}} | OPTIONAL
- Default deny statement: {{policy.default_deny}}

5) System-wide Constraints (required)

- Least privilege: {{constraints.least_privilege}}
- Separation of duties: {{constraints.sod}} | OPTIONAL
- Sensitive operation classification: {{constraints.sensitive_ops}}

Cross-References

- Upstream: {{xref:PRD-03}} | OPTIONAL, {{xref:BRP-02}} | OPTIONAL, {{xref:IAM-03}} | OPTIONAL
- Downstream: {{xref:PMAD-02}}, {{xref:PMAD-03}}, {{xref:PMAD-05}} | OPTIONAL, {{xref:ERR-04}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Roles/resources/actions tables + default deny.
- intermediate: Required. Add expression model and scope/tenancy rules.
- advanced: Required. Add sensitive ops classification and separation of duties (if used).

Unknown Handling

- UNKNOWN_ALLOWED: `role_inheritance`, `multi_tenant_notes`, `guest_access_rules`, `notes`
- If default_deny policy is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ
- Pass conditions:
    - required_fields_present == true
    - roles_resources_actions_present == true
    - expression_model_present == true
    - default_deny_defined == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

PMAD-02

PMAD-02 — AuthZ Policy Rules (RBAC/ABAC, inheritance, exceptions)

Header Block

- template_id: PMAD-02
- title: AuthZ Policy Rules (RBAC/ABAC, inheritance, exceptions)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-02_AuthZ_Policy_Rules.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PMAD-01", "BRP-02", "BRP-04"]
- inputs_required: ["PMAD-01", "BRP-02", "BRP-04", "ERR-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the actual authorization policy rules: what roles can perform which actions on which resources, what ABAC conditions apply, how inheritance works, and how exceptions are handled. This is the enforceable ruleset used by all authz enforcement points.

Inputs Required

- PMAD-01: {{xref:PMAD-01}}
- BRP-02: {{xref:BRP-02}} | OPTIONAL
- BRP-04: {{xref:BRP-04}} | OPTIONAL

- ERR-02: {{xref:ERR-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Policy rules list (minimum 25 for non-trivial products)
- For each rule:
    - policy_id
    - role_id(s)
    - resource_id
    - action_id(s)
    - conditions (ABAC predicates)
    - allow/deny outcome
    - priority/order (if multiple match)
    - exception hooks (break-glass/admin override) (if any)
    - reason_code on deny (rc_*)
    - audit requirement (yes/no + event type)
- Inheritance rules (if role hierarchy exists)
- Conflict resolution rules (what happens if allow and deny both match)

Optional Fields

- Policy grouping by domain/boundary | OPTIONAL
- Notes | OPTIONAL

Rules

- Denies must be explicit when needed; define precedence rules.
- Every deny must map to a reason_code or policy fallback reason.
- Exception hooks must be auditable and time-bound if possible.
- Conditions vocabulary must match PMAD-01; no ad-hoc predicates.

Output Format

1) Policy Rules (canonical)

policy_id	roles	resource	actions	conditions	outcome	priority	exception_hook	deny_reason_code	audit_required	notes
pol_001	{{policies[0].roles}}	{{policies[0].resource}}	{{policies[0].actions}}	{{policies[0].conditions}}	{{policies[0].outcome}}	{{policies[0].priority}}	{{policies[0].exception}}	{{policies[0].deny_rc}}	{{policies[0].audit}}	{{policies[0].notes}}
pol_002	{{policies[1].roles}}	{{policies[1].resource}}	{{policies[1].actions}}	{{policies[1].conditions}}	{{policies[1].outcome}}	{{policies[1].priority}}	{{policies[1].exception}}	{{policies[1].deny_rc}}	{{policies[1].audit}}	{{policies[1].notes}}

2) Inheritance Rules (required if hierarchy exists)

- Role inheritance model: {{inheritance.model}} | OPTIONAL
- Inherited permissions behavior: {{inheritance.behavior}} | OPTIONAL

- Override rules: {{inheritance.override}} | OPTIONAL

3) Conflict Resolution (required)

- Deny vs allow precedence: {{conflict.precedence}}
- Priority ordering rule: {{conflict.priority_rule}}
- Fallback deny reason: {{conflict.fallback_reason_code}} | OPTIONAL

4) Exception Hooks (required if any)

hook_id	who_can_use	scope	time_bound	audit_event	notes
exhook_01	{{exceptions[0].who}}	{{exceptions[0].scope}}	{{exceptions[0].time_bound}}	{{exceptions[0].audit_event}}	{{exceptions[0].notes}}

Cross-References

- Upstream: {{xref:PMAD-01}} | OPTIONAL, {{xref:BRP-02}} | OPTIONAL
- Downstream: {{xref:PMAD-03}}, {{xref:PMAD-04}} | OPTIONAL, {{xref:ERR-02}} | OPTIONAL, {{xref:AUDIT-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Policy table with roles/resources/actions and outcomes.
- intermediate: Required. Add conditions, deny reason codes, and audit flags.
- advanced: Required. Add conflict resolution, inheritance, and exception hook details.

Unknown Handling

- UNKNOWN_ALLOWED: `policy_grouping`, `inheritance_rules` (if none), `exception_hooks` (if none), `notes`
- If any deny outcome lacks deny_reason_code → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ
- Pass conditions:
    - required_fields_present == true
    - policies_count >= 25
    - deny_reason_codes_present == true
    - conflict_resolution_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

PMAD-03

PMAD-03 — Enforcement Points Map (UI/API/service/DB)

Header Block

- template_id: PMAD-03
- title: Enforcement Points Map (UI/API/service/DB)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-03_Enforcement_Points_Map.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PMAD-02", "ARC-01", "API-01", "DATA-01"]
- inputs_required: ["PMAD-02", "ARC-01", "API-01", "DATA-01", "IAN-05", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define where authorization is enforced across the stack: UI gating, API gateway checks, service-layer checks, and data-layer guards. This prevents “security gaps” where a permission is enforced in one layer but bypassable in another.

Inputs Required

- PMAD-02: {{xref:PMAD-02}} | OPTIONAL
- ARC-01: {{xref:ARC-01}} | OPTIONAL
- API-01: {{xref:API-01}} | OPTIONAL

- DATA-01: {{xref:DATA-01}} | OPTIONAL
- IAN-05: {{xref:IAN-05}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Enforcement layer definitions (UI/API/service/DB)
- Enforcement point inventory (minimum 20 for non-trivial products)
- For each enforcement point:
    - ep_id
    - layer (ui/api/service/db)
    - resource_id
    - action_id(s)
    - where_in_code (module/route/service name) | OPTIONAL placeholder
    - decision function (policy check method)
    - required inputs (subject, resource, context)
    - deny behavior (status, reason_code, UX surface pointer)
    - logging/audit requirement
    - test requirement (unit/contract/e2e)
- Consistency rule: API and service enforcement cannot rely on UI-only checks
- Coverage check: every PMAD-02 policy appears in at least one enforcement point

Optional Fields

- Caching policy for authz decisions | OPTIONAL
- Notes | OPTIONAL

Rules

- UI gating is for UX only; real enforcement must occur server-side.
- DB-level guards (if used) must align with service-level policies; do not create conflicting logic.
- Deny responses must map to reason codes and avoid leakage.
- Each enforcement point must specify test coverage expectation.

Output Format

1) Layer Rules (required)

- UI: {{layers.ui_rule}}
- API: {{layers.api_rule}}
- Service: {{layers.service_rule}}
- DB: {{layers.db_rule}} | OPTIONAL

2) Enforcement Points Inventory (canonical)

ep_id	layer	resource	actions	decision_method	required_inputs	deny_behavior	audit_required	test_type	code_ref	notes
ep_001	{{eps[0].layer}}	{{eps[0].resource}}	{{eps[0].actions}}	{{eps[0].decision}}	{{eps[0].inputs}}	{{eps[0].deny}}	{{eps[0].audit}}	{{eps[0].test}}	{{eps[0].code_ref}}	{{eps[0].notes}}

ep_002	{{eps[1].layer}}	{{eps[1].resource}}	{{eps[1].actions}}	{{eps[1].decision}}	{{eps[1].inputs}}	{{eps[1].deny}}	{{eps[1].audit}}	{{eps[1].test}}	{{eps[1].code_ref}}	{{eps[1].notes}}

3) Coverage Check (required)

policy_id	has_enforcement_point	ep_ids
{{policies[0].policy_id}}	{{coverage[0].covered}}	{{coverage[0].ep_ids}}

4) Server-Side Guarantee Statement (required)

- Server-side enforcement required for all restricted actions: {{guarantees.server_side_required}}
- UI-only checks are non-authoritative: {{guarantees.ui_non_authoritative}}

Cross-References

- Upstream: {{xref:PMAD-02}} | OPTIONAL, {{xref:IAN-05}} | OPTIONAL
- Downstream: {{xref:PMAD-04}}, {{xref:PMAD-06}} | OPTIONAL, {{xref:QA-02}} | OPTIONAL, {{xref:TINF-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Inventory + layer rules + deny behavior.
- intermediate: Required. Add audit/test expectations and policy coverage check.
- advanced: Required. Add caching policy and DB guard alignment (if used).

Unknown Handling

- UNKNOWN_ALLOWED: `code_ref`, `caching_policy`, `notes`, `db_layer` (if not used)
- If coverage check shows missing policies → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ
- Pass conditions:
  - required_fields_present == true
  - enforcement_points_count >= 20
  - coverage_complete_for_policies == true
  - deny_behavior_defined == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

PMAD-04

PMAD-04 — Permission Check Patterns (standard decision flow + reason codes)

Header Block

- template_id: PMAD-04
- title: Permission Check Patterns (standard decision flow + reason codes)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-04_Permission_Check_Patterns.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PMAD-02", "ERR-02", "ERR-03"]
- inputs_required: ["PMAD-02", "ERR-02", "ERR-03", "ARC-06", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the standard pattern for performing permission checks and generating consistent deny outcomes: what inputs are required, how decisions are logged, which reason codes to emit, and how to avoid inconsistent authz logic across services.

Inputs Required

- PMAD-02: {{xref:PMAD-02}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL

- ARC-06: {{xref:ARC-06}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Standard decision flow (step-by-step)
- Required inputs (subject/resource/action/context)
- Decision outcomes (allow/deny/unknown) and handling rules
- Deny response rules:
    - status mapping
    - reason_code mapping
    - redaction rules
- Logging/audit rules:
    - what to log
    - what not to log
    - correlation ID inclusion
- Performance rules (caching/short-circuit rules) (high level)
- Example patterns (minimum 3):
    - owner-only access
    - role-based access
    - conditional (ABAC) access
- Test requirements (what must be tested)

Optional Fields

- Policy evaluation caching design | OPTIONAL
- Notes | OPTIONAL

Rules

- Unknown evaluation results must default to deny (per PMAD-01 default-deny).
- Deny must always output a reason_code (policy-defined or fallback).
- Logging must be redacted; never log secrets or sensitive resource contents.
- If caching decisions, cache must be scoped by subject + resource + action + context.

Output Format

1) Standard Decision Flow (required)

1. Collect inputs: {{flow.step1}}
2. Normalize context: {{flow.step2}}
3. Evaluate matching policies: {{flow.step3}}
4. Resolve conflicts: {{flow.step4}}
5. Emit decision + reason_code: {{flow.step5}}
6. Log/audit event: {{flow.step6}}
7. Return response: {{flow.step7}}

2) Required Inputs (required)

- subject: {{inputs.subject}}
- resource: {{inputs.resource}}
- action: {{inputs.action}}

- context: {{inputs.context}}

3) Outcomes (required)

outcome	meaning	response	reason_code_rule
allow	{{outcomes.allow.meaning}}	{{outcomes.allow.response}}	{{outcomes.allow.rc_rule}}
deny	{{outcomes.deny.meaning}}	{{outcomes.deny.response}}	{{outcomes.deny.rc_rule}}
unknown	{{outcomes.unknown.meaning}}	{{outcomes.unknown.response}}	{{outcomes.unknown.rc_rule}}

4) Deny Response Rules (required)

- Status mapping pointer: {{xref:ERR-03}} | OPTIONAL
- Fallback deny reason_code: {{deny.fallback_reason_code}}
- Redaction rule: {{deny.redaction}}
- Correlation ID policy: {{deny.correlation_id}} | OPTIONAL

5) Logging/Audit Rules (required)

- Must log: {{log.must_log}}
- Must not log: {{log.must_not_log}}
- Audit event name: {{log.audit_event_name}} | OPTIONAL

6) Performance & Caching (required)

- Short-circuit rules: {{perf.short_circuit}}
- Caching allowed: {{perf.caching_allowed}}
- Cache key scope: {{perf.cache_key_scope}} | OPTIONAL

7) Example Patterns (required, min 3)

Pattern A — Owner-only

- Rule form: {{examples.owner.rule}}
- Inputs needed: {{examples.owner.inputs}}
- Deny reason_code: {{examples.owner.deny_rc}}

Pattern B — Role-based

- Rule form: {{examples.role.rule}}
- Inputs needed: {{examples.role.inputs}}
- Deny reason_code: {{examples.role.deny_rc}}

Pattern C — ABAC conditional

- Rule form: {{examples.abac.rule}}
- Inputs needed: {{examples.abac.inputs}}
- Deny reason_code: {{examples.abac.deny_rc}}

8) Test Requirements (required)

- Unit tests: {{tests.unit}}
- Integration/contract tests: {{tests.contract}} | OPTIONAL
- E2E coverage: {{tests.e2e}} | OPTIONAL

Cross-References

- Upstream: {{xref:PMAD-02}} | OPTIONAL, {{xref:ERR-02}} | OPTIONAL, {{xref:ARC-06}} | OPTIONAL

- Downstream: {{xref:PMAD-03}} | OPTIONAL, {{xref:PMAD-06}} | OPTIONAL, {{xref:TINF-*}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Decision flow + outcomes + deny rules.
- intermediate: Required. Add logging rules and examples.
- advanced: Required. Add caching/perf constraints and test requirements depth.

Unknown Handling

- UNKNOWN_ALLOWED: `policy_eval_caching`, `notes`, `audit_event_name`, `correlation_id`
- If fallback deny reason_code is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ
- Pass conditions:
    - required_fields_present == true
    - decision_flow_present == true
    - outcomes_defined == true
    - deny_rules_present == true
    - examples_count >= 3
    - placeholder_resolution == true

- no_unapproved_unknowns == true

PMAD-05

PMAD-05 — Privileged Operations Policy (admin/mod/support actions, approvals)

Header Block

- template_id: PMAD-05
- title: Privileged Operations Policy (admin/mod/support actions, approvals)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-05_Privileged_Operations_Policy.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PMAD-01", "PMAD-02", "ADMIN-01", "AUDIT-01"]
- inputs_required: ["PMAD-01", "PMAD-02", "ADMIN-01", "AUDIT-01", "STK-04", "DGP-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the policy for privileged operations (admin/mod/support): what actions are privileged, who can perform them, what approvals are required, what safeguards apply (2-person rule, time-bound access), and what must be audited.

Inputs Required

- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- PMAD-02: {{xref:PMAD-02}} | OPTIONAL
- ADMIN-01: {{xref:ADMIN-01}} | OPTIONAL

- AUDIT-01: {{xref:AUDIT-01}} | OPTIONAL
- STK-04: {{xref:STK-04}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Privileged action catalog (minimum 15)
- For each privileged action:
    - priv_action_id
    - action description
    - resource scope (what it affects)
    - who can execute (role/tier)
    - approval required (none/1-step/2-person/manager/legal)
    - justification required (yes/no + fields)
    - time-bound requirement (yes/no + duration)
    - audit event name
    - data sensitivity (PII level)
    - UI surface where performed (admin console) | OPTIONAL
    - emergency override allowed (yes/no) + rules
- Break-glass policy (when/why/how)
- Post-action review requirements (spot checks, approvals)
- Deny behaviors + reason codes policy

Optional Fields

- Training requirement | OPTIONAL
- Notes | OPTIONAL

Rules

- All privileged operations must be auditable with before/after context (redacted).
- If break-glass exists, it must be time-bound and reviewed.
- High-PII operations require stricter approvals and minimum exposure.
- Privileged actions must have explicit deny reason codes and consistent UX messaging.

Output Format

1) Privileged Action Catalog (canonical)

priv_action_id	description	scope	executor_roles	approval	justification_fields	time_bound	audit_event	pii_level	emergency_override	notes
pa_01	{{actions[0].desc}}	{{actions[0].scope}}	{{actions[0].roles}}	{{actions[0].approval}}	{{actions[0].justification}}	{{actions[0].time_bound}}	{{actions[0].audit_event}}	{{actions[0].pii}}	{{actions[0].override}}	{{actions[0].notes}}
pa_02	{{actions[1].desc}}	{{actions[1].scope}}	{{actions[1].roles}}	{{actions[1].approval}}	{{actions[1].justification}}	{{actions[1].time_bound}}	{{actions[1].audit_event}}	{{actions[1].pii}}	{{actions[1].override}}	{{actions[1].notes}}

2) Break-Glass Policy (required)

- When allowed: {{breakglass.when_allowed}}
- Who can invoke: {{breakglass.who}}

- Time bound duration: {{breakglass.duration}}
- Required justification: {{breakglass.justification}}
- Required review after: {{breakglass.post_review}}
- Logging requirements: {{breakglass.logging}}

3) Approval Workflow Rules (required)

- Default approval path: {{approvals.default_path}}
- 2-person rule conditions: {{approvals.two_person_conditions}} | OPTIONAL
- Denial/timeout handling: {{approvals.denial_handling}} | OPTIONAL

4) Post-Action Review (required)

- Review cadence: {{review.cadence}}
- Sample size rule: {{review.sample_size}} | OPTIONAL
- Who reviews: {{review.owner}}
- Escalation rules: {{review.escalation}} | OPTIONAL

5) Deny Behavior + Reason Codes (required)

- Deny reason_code source: {{xref:ERR-02}} | OPTIONAL
- UX copy pointer: {{xref:CDX-04}} | OPTIONAL
- Default deny reason_code: {{deny.default_rc}}

Cross-References

- Upstream: {{xref:ADMIN-01}} | OPTIONAL, {{xref:AUDIT-01}} | OPTIONAL, {{xref:STK-04}} | OPTIONAL

- Downstream: {{xref:PMAD-06}} | OPTIONAL, {{xref:ADMIN-03}} | OPTIONAL, {{xref:SECOPS-*}} | OPTIONAL

- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Catalog + break-glass + audit event requirement.

- intermediate: Required. Add approval workflows and post-action review rules.

- advanced: Required. Add PII sensitivity policies and emergency override controls.

Unknown Handling

- UNKNOWN_ALLOWED: `training_requirement`, `notes`, `sample_size_rule`, `two_person_conditions`

- If any privileged action lacks audit_event or approval rule → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ

- Pass conditions:

    - required_fields_present == true

    - privileged_actions_count >= 15

    - break_glass_policy_present == true

    - approvals_defined == true

    - audit_events_defined == true

- placeholder_resolution == true
- no_unapproved_unknowns == true

PMAD-06

PMAD-06 — Audit Requirements for AuthZ (what must be logged)

Header Block

- template_id: PMAD-06
- title: Audit Requirements for AuthZ (what must be logged)
- type: permission_model_authorization_design
- template_version: 1.0.0
- output_path: 10_app/authz/PMAD-06_Audit_Requirements_for_AuthZ.md
- compliance_gate_id: TMP-05.PRIMARY.AUTHZ
- upstream_dependencies: ["PMAD-03", "PMAD-04", "AUDIT-01", "DGP-01"]
- inputs_required: ["PMAD-03", "PMAD-04", "AUDIT-01", "DGP-01", "OBS-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the audit logging requirements specifically for authorization: what decisions and actions must be logged, what fields are required, how redaction works, retention rules, and how audit trails support investigations and compliance.

Inputs Required

- PMAD-03: {{xref:PMAD-03}} | OPTIONAL
- PMAD-04: {{xref:PMAD-04}} | OPTIONAL
- AUDIT-01: {{xref:AUDIT-01}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- OBS-01: {{xref:OBS-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Audit event categories:
    - auth decisions (allow/deny)
    - privileged actions
    - policy changes (if applicable)
    - access grant/revoke events
- Required audit fields:
    - timestamp
    - actor (user/service)
    - actor_role(s)
    - action
    - resource + resource identifiers
    - decision outcome
    - reason_code
    - enforcement_point (where decision made)
    - correlation_id / trace_id
    - source IP / device (where applicable)
    - before/after snapshots (for privileged actions) (redacted)

- Redaction rules (PII)
- Retention rules and access controls (who can view audit logs)
- Verification checklist

Optional Fields

- Export/reporting requirements | OPTIONAL
- Notes | OPTIONAL

Rules

- Audit logs must be tamper-evident or protected (pointer to audit system rules).
- Access to audit logs must be least-privilege and itself auditable.
- Reason codes are mandatory for denies and privileged actions.
- Before/after snapshots must avoid storing sensitive fields unnecessarily.

Output Format

1) Audit Event Categories (required)

- Auth decisions: {{categories.auth_decisions}}
- Privileged actions: {{categories.privileged}}
- Access grants/revokes: {{categories.grants}}
- Policy changes: {{categories.policy_changes}} | OPTIONAL

2) Required Fields (required)

field	required	description	redaction_rule

timestamp	true	{{fields.timestamp.desc}}	{{fields.timestamp.redaction}}
actor_id	true	{{fields.actor_id.desc}}	{{fields.actor_id.redaction}}
action	true	{{fields.action.desc}}	{{fields.action.redaction}}
resource	true	{{fields.resource.desc}}	{{fields.resource.redaction}}
decision	true	{{fields.decision.desc}}	{{fields.decision.redaction}}
reason_code	true	{{fields.reason.desc}}	{{fields.reason.redaction}}
correlation_id	true	{{fields.correlation.desc}}	{{fields.correlation.redaction}}

3) Redaction Rules (required)

- Never store: {{redaction.never_store}}
- Store hashed: {{redaction.hash}} | OPTIONAL
- Store truncated: {{redaction.truncate}} | OPTIONAL

4) Retention & Access Control (required)

- Retention window: {{retention.window}}
- Who can view: {{retention.view_roles}}
- Access logging required: {{retention.access_logged}}
- Export policy: {{retention.export_policy}} | OPTIONAL

5) Verification Checklist (required)

- {{verify[0]}}
- {{verify[1]}}
- {{verify[2]}}
- {{verify[3]}} | OPTIONAL

Cross-References

- Upstream: {{xref:AUDIT-01}} | OPTIONAL, {{xref:DGP-01}} | OPTIONAL, {{xref:OBS-01}} | OPTIONAL
- Downstream: {{xref:ADMIN-03}} | OPTIONAL, {{xref:IRP-*}} | OPTIONAL, {{xref:COMP-02}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Required fields + retention + redaction basics.
- intermediate: Required. Add event categories and access control policy.
- advanced: Required. Add export/reporting and tamper-evidence pointers.

Unknown Handling

- UNKNOWN_ALLOWED: `export_requirements`, `notes`, `policy_changes_category`
- If retention window or view roles are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.AUTHZ
- Pass conditions:
    - required_fields_present == true
    - reason_code_mandatory == true
    - retention_defined == true

- access_controls_defined == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

Error Model & Reason Codes (ERR)

- Error Model & Reason Codes (ERR)
  ERR-01 Error Taxonomy (classes, categories, severity)
  ERR-02 Reason Codes Registry (rc_* catalog + meanings)
  ERR-03 API Error Contract (shape, status mapping, localization hooks)
  ERR-04 UX Error Mapping Rules (reason_code → DES/CDX surfaces)
  ERR-05 Retryability & Idempotency Rules (which errors retry, how)
  ERR-06 Observability Requirements for Errors (logs/metrics/traces fields)

ERR-01

ERR-01 — Error Taxonomy (classes, categories, severity)

Header Block

- template_id: ERR-01
- title: Error Taxonomy (classes, categories, severity)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-01_Error_Taxonomy.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ARC-06", "BRP-01", "DES-07"]
- inputs_required: ["ARC-06", "BRP-01", "DES-07", "CDX-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical error taxonomy used across the system: classes, categories, severities, and the baseline handling expectations. This is the shared language for errors so APIs, jobs, realtime, and UX map failures consistently.

Inputs Required

- ARC-06: {{xref:ARC-06}} | OPTIONAL
- BRP-01: {{xref:BRP-01}} | OPTIONAL
- DES-07: {{xref:DES-07}} | OPTIONAL

- CDX-04: {{xref:CDX-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Error classes (minimum):
    - validation
    - domain_rule
    - authn/authz
    - dependency
    - conflict/concurrency
    - system_unknown
- Category model (subclasses or tags) (e.g., input_missing, quota_exceeded)
- Severity model (P0/P1/P2 or equivalent) and definitions
- Default handling expectations per class:
    - typical UX surface (inline/toast/banner/modal)
    - retryability default (yes/no)
    - logging level (info/warn/error)
- Cross-system consistency rule: same class means same handling posture

Optional Fields

- Domain-specific category extensions | OPTIONAL
- Notes | OPTIONAL

Rules

- Taxonomy must align with ARC-06; if conflict, ARC-06 is architecture-authoritative.
- Every emitted error must map to exactly one class (primary) and may have categories/tags.
- Severity is about impact, not “how scary the message looks.”
- Default retryability must align to idempotency rules (ERR-05).

Output Format

1) Classes (required)

error_class	description	common_sources	default_surface	default_retryability	log_level
validation	{{classes.validation.desc}}	{{classes.validation.sources}}	{{classes.validation.surface}}	{{classes.validation.retry}}	{{classes.validation.log}}
domain_rule	{{classes.domain.desc}}	{{classes.domain.sources}}	{{classes.domain.surface}}	{{classes.domain.retry}}	{{classes.domain.log}}
authz	{{classes.authz.desc}}	{{classes.authz.sources}}	{{classes.authz.surface}}	{{classes.authz.retry}}	{{classes.authz.log}}
dependency	{{classes.dependency.desc}}	{{classes.dependency.sources}}	{{classes.dependency.surface}}	{{classes.dependency.retry}}	{{classes.dependency.log}}
conflict	{{classes.conflict.desc}}	{{classes.conflict.sources}}	{{classes.conflict.surface}}	{{classes.conflict.retry}}	{{classes.conflict.log}}
system_unknown	{{classes.system.desc}}	{{classes.system.sources}}	{{classes.system.surface}}	{{classes.system.retry}}	{{classes.system.log}}

2) Categories/Tags (required)

category_id	meaning	applies_to_classes	examples
cat_input_missing	{{cats.input_missing.meaning}}	{{cats.input_missing.classes}}	{{cats.input_missing.examples}}

cat_quota_exceeded	{{cats.quota.meaning}}	{{cats.quota.classes}}	{{cats.quota.examples}}

3) Severity Model (required)

severity	definition	example_impacts	owner_response
P0	{{sev.p0.def}}	{{sev.p0.impacts}}	{{sev.p0.response}}
P1	{{sev.p1.def}}	{{sev.p1.impacts}}	{{sev.p1.response}}
P2	{{sev.p2.def}}	{{sev.p2.impacts}}	{{sev.p2.response}}

4) Consistency Rules (required)

- Single primary class rule: {{rules.single_primary_class}}
- Tagging rule: {{rules.tagging}}
- Handling consistency rule: {{rules.handling_consistency}}

Cross-References

- Upstream: {{xref:ARC-06}} | OPTIONAL, {{xref:DES-07}} | OPTIONAL, {{xref:CDX-04}} | OPTIONAL
- Downstream: {{xref:ERR-02}}, {{xref:ERR-03}}, {{xref:ERR-04}}, {{xref:ERR-05}}, {{xref:ERR-06}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Define classes and severity model.
- intermediate: Required. Add categories and default handling expectations.
- advanced: Required. Add explicit consistency rules and extensions policy.

Unknown Handling

- UNKNOWN_ALLOWED: `domain_extensions`, `notes`, `examples`
- If any class lacks default surface or retryability → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
  - required_fields_present == true
  - classes_present == true
  - severity_model_present == true
  - default_handling_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ERR-02

ERR-02 — Reason Codes Registry (rc_* catalog + meanings)

Header Block

- template_id: ERR-02
- title: Reason Codes Registry (rc_* catalog + meanings)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-02_Reason_Codes_Registry.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-01", "ARC-06"]
- inputs_required: ["ERR-01", "ARC-06", "BRP-01", "DMG-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Create the canonical registry of reason codes (`rc_*`) used across the system. This enables deterministic error mapping, consistent UX messaging, and consistent observability. Reason codes are stable identifiers and must never be reused.

Inputs Required

- ERR-01: {{xref:ERR-01}} | OPTIONAL
- ARC-06: {{xref:ARC-06}} | OPTIONAL
- BRP-01: {{xref:BRP-01}} | OPTIONAL

- DMG-01: {{xref:DMG-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Naming convention rules for reason codes
- Registry entries (minimum 40 for non-trivial products; justify if smaller)
- For each reason code:
    - reason_code (rc_<domain>_<slug>)
    - error_class (from ERR-01)
    - meaning (one sentence)
    - user_action_guidance (what to do)
    - default_http_status (if applicable)
    - retryable_default (true/false)
    - data_sensitivity (does message involve PII?) (yes/no)
    - owner_boundary/service
    - related_policy_or_rule (BRP/PMAD/etc pointer)
    - used_by (API endpoints/jobs/webhooks) | OPTIONAL
    - deprecation_status (active/deprecated)
    - replacement_reason_code (if deprecated)

Optional Fields

- Localization key mapping | OPTIONAL

- Notes | OPTIONAL

Rules

- Reason codes are immutable IDs; deprecate instead of changing meaning.
- Every deny/validation/business-rule error must map to a reason code (or the fallback policy).
- A reason code must map to exactly one primary error_class.
- If retryable_default is true, ERR-05 must define the exact retry behavior.

Output Format

1) Naming Rules (required)

- Convention: {{rules.naming}} (rc_<area>_<slug>)
- Allowed characters: {{rules.allowed_chars}}
- Uniqueness rule: {{rules.uniqueness}}
- Deprecation rule: {{rules.deprecation}}

2) Reason Codes Registry (canonical)

reason_code	error_class	meaning	user_guidance	http_status	retryable	pii_sensitive	owner	related_ref	used_by	status	replacement	notes
rc_001	{{codes[0].class}}	{{codes[0].meaning}}	{{codes[0].guidance}}	{{codes[0].status}}	{{codes[0].retryable}}	{{codes[0].pii}}	{{codes[0].owner}}	{{codes[0].ref}}	{{codes[0].used_by}}	{{codes[0].status_flag}}	{{codes[0].replacement}}	{{codes[0].notes}}

rc_002	{{codes[1].class}}	{{codes[1].meaning}}	{{codes[1].guidance}}	{{codes[1].status}}	{{codes[1].retryable}}	{{codes[1].pii}}	{{codes[1].owner}}	{{codes[1].ref}}	{{codes[1].used_by}}	{{codes[1].status_flag}}	{{codes[1].replacement}}	{{codes[1].notes}}

3) Coverage Checks (required)

- Reason codes cover all deny policies (PMAD): {{coverage.pmads_covered}} | OPTIONAL
- Reason codes cover all API error cases (API-03): {{coverage.api_covered}} | OPTIONAL
- No duplicates: {{coverage.no_duplicates}}

Cross-References

- Upstream: {{xref:ERR-01}} | OPTIONAL, {{xref:ARC-06}} | OPTIONAL
- Downstream: {{xref:ERR-03}}, {{xref:ERR-04}}, {{xref:ERR-05}}, {{xref:CDX-04}} | OPTIONAL
- Standards: {{standards.rules[STD-NAMING]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Naming rules + baseline registry entries.
- intermediate: Required. Add status/retryability and owner mapping.
- advanced: Required. Add deprecation/replacement and coverage checks.

Unknown Handling

- UNKNOWN_ALLOWED: `used_by`, `localization_key_mapping`, `notes`, `replacement_reason_code` (if active)

- If coverage.no_duplicates is false → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
    - required_fields_present == true
    - naming_rules_present == true
    - codes_count >= 40 (or justified)
    - error_class_present_for_all_codes == true
    - no_duplicates == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

ERR-03

ERR-03 — API Error Contract (shape, status mapping, localization hooks)

Header Block

- template_id: ERR-03
- title: API Error Contract (shape, status mapping, localization hooks)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-03_API_Error_Contract.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-01", "ERR-02", "APIG-01", "DGP-01"]
- inputs_required: ["ERR-01", "ERR-02", "APIG-01", "DGP-01", "CDX-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical error response contract for APIs: payload shape, required fields, status mapping rules, localization hooks, correlation ID inclusion, and safe disclosure/redaction rules. This ensures every API returns errors consistently.

Inputs Required

- ERR-01: {{xref:ERR-01}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- APIG-01: {{xref:APIG-01}} | OPTIONAL

- DGP-01: {{xref:DGP-01}} | OPTIONAL
- CDX-04: {{xref:CDX-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Error payload schema (canonical fields)
- Required fields:
    - error_id (unique per occurrence)
    - reason_code (rc_*)
    - error_class
    - message_key (localization hook) OR safe_message (if allowed)
    - http_status
    - correlation_id (trace/request)
    - timestamp
- Optional fields:
    - field_errors (per input field)
    - retry_after (for rate limits)
    - docs_url (optional)
    - debug_ref (internal-only)
- Status mapping rules:
    - error_class/category → status code
    - specific reason_code overrides

- Localization rules:
    - how clients map reason_code → localized copy
    - server-side vs client-side localization policy
- Redaction rules:
    - what fields never returned
    - what can be returned for debug
- Contract test requirements (baseline set)

Optional Fields

- GraphQL error mapping notes | OPTIONAL
- Notes | OPTIONAL

Rules

- reason_code is mandatory for all non-2xx errors (except truly unknown fallback with explicit rc_unknown).
- Error payload must not leak internal stack traces or sensitive identifiers.
- Status codes must be consistent across endpoints; do not “choose per endpoint.”
- Field errors must use stable input field names (aligned to API schemas).
- Correlation ID must always be present and consistent with observability propagation.

Output Format

1) Canonical Error Payload Schema (required)

```
{
 "error_id": "{{error.error_id}}",
 "reason_code": "{{error.reason_code}}",
```

```
 "error_class": "{{error.error_class}}",
 "http_status": {{error.http_status}},
 "message_key": "{{error.message_key}}",
 "safe_message": "{{error.safe_message}}",
 "correlation_id": "{{error.correlation_id}}",
 "timestamp": "{{error.timestamp}}",
 "field_errors": [
   {
     "field": "{{error.field_errors[0].field}}",
     "reason_code": "{{error.field_errors[0].reason_code}}",
     "message_key": "{{error.field_errors[0].message_key}}"
   }
 ],
 "retry_after": {{error.retry_after}},
 "docs_url": "{{error.docs_url}}"
}
```

2) Required/Optional Fields (required)

- Required: error_id, reason_code, error_class, http_status, correlation_id, timestamp, (message_key OR safe_message)
- Optional: field_errors, retry_after, docs_url, debug_ref

3) Status Mapping Rules (required)

error_class	default_status	overrides (reason_code → status)	notes
validation	{{status.validation.default}}	{{status.validation.overrides}}	{{status.validation.notes}}
authz	{{status.authz.default}}	{{status.authz.overrides}}	{{status.authz.notes}}
domain_rule	{{status.domain.default}}	{{status.domain.overrides}}	{{status.domain.notes}}
dependency	{{status.dependency.default}}	{{status.dependency.overrides}}	{{status.dependency.notes}}
system_unknown	{{status.system.default}}	{{status.system.overrides}}	{{status.system.notes}}

4) Localization Hooks (required)

- Server localization policy: {{l10n.server_policy}}
- Client localization policy: {{l10n.client_policy}}
- message_key format: {{l10n.message_key_format}} | OPTIONAL
- If safe_message used, constraints: {{l10n.safe_message_constraints}}

5) Redaction Rules (required)

- Never return: {{redaction.never_return}}
- Allowed (safe debug): {{redaction.allowed_debug}} | OPTIONAL
- debug_ref policy: {{redaction.debug_ref_policy}} | OPTIONAL

6) Contract Tests (required)

- Must test presence of required fields: {{tests.required_fields}}
- Must test status mapping correctness: {{tests.status_mapping}}
- Must test no leakage (no stack traces): {{tests.no_leakage}}
- Must test field_errors shape: {{tests.field_errors}} | OPTIONAL

Cross-References

- Upstream: {{xref:ERR-01}} | OPTIONAL, {{xref:ERR-02}} | OPTIONAL, {{xref:DGP-01}} | OPTIONAL
- Downstream: {{xref:API-03}} | OPTIONAL, {{xref:ERR-04}} | OPTIONAL, {{xref:QA-03}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Payload schema + required fields + default status mapping.
- intermediate: Required. Add localization policy and redaction rules.
- advanced: Required. Add overrides and contract test requirements.

Unknown Handling

- UNKNOWN_ALLOWED: `graphql_notes`, `docs_url_policy`, `notes`, `overrides` (if none)
- If reason_code is not mandatory → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
    - required_fields_present == true
    - payload_schema_present == true
    - status_mapping_present == true
    - redaction_rules_present == true
    - contract_tests_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

ERR-04

ERR-04 — UX Error Mapping Rules (reason_code → DES/CDX surfaces)

Header Block

- template_id: ERR-04
- title: UX Error Mapping Rules (reason_code → DES/CDX surfaces)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-04_UX_Error_Mapping_Rules.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-02", "DES-07", "CDX-04", "A11YD-05"]
- inputs_required: ["ERR-02", "DES-07", "CDX-04", "CDX-01", "A11YD-05", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define deterministic rules for how each reason_code is presented in the UX: which surface is used (inline/toast/banner/modal), what copy key is used, whether retry is offered, and what accessibility requirements apply. This prevents inconsistent error UX across screens and platforms.

Inputs Required

- ERR-02: {{xref:ERR-02}} | OPTIONAL
- DES-07: {{xref:DES-07}} | OPTIONAL

- CDX-04: {{xref:CDX-04}} | OPTIONAL
- CDX-01: {{xref:CDX-01}} | OPTIONAL
- A11YD-05: {{xref:A11YD-05}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Mapping table (minimum 30 reason_code mappings; justify if smaller)
- For each mapping:
    - reason_code
    - default_surface (inline/toast/banner/modal/fullscreen)
    - message_key or msg_id (from CDX-04)
    - user_guidance (short)
    - retry_allowed (true/false)
    - retry_action (what happens on retry) | OPTIONAL
    - escalation_action (contact support, report, etc.) | OPTIONAL
    - accessibility requirement (announce, focus move, etc.)
    - platform notes (web/mobile)
- Global rules:
    - precedence rules (if multiple errors occur)
    - field error handling rules
    - unknown reason_code fallback behavior

Optional Fields

- Screen-specific overrides | OPTIONAL
- Notes | OPTIONAL

Rules

- Must align with DES-07 surface rules and A11YD-05 focus/announcement rules.
- Every mapped reason_code must exist in ERR-02 registry.
- Unknown reason_code must map to a safe generic message and correlation ID guidance policy.
- Field-level validation errors must use inline + focus rules.

Output Format

1) Reason Code → UX Mapping (canonical)

reason_code	surface	cdx_msg_id_or_key	guidance	retry_allowed	retry_action	escalation_action	a11y_behavior	platform_notes	notes
{{map[0].reason_code}}	{{map[0].surface}}	{{map[0].cdx_ref}}	{{map[0].guidance}}	{{map[0].retry}}	{{map[0].retry_action}}	{{map[0].escalation}}	{{map[0].a11y}}	{{map[0].platform}}	{{map[0].notes}}
{{map[1].reason_code}}	{{map[1].surface}}	{{map[1].cdx_ref}}	{{map[1].guidance}}	{{map[1].retry}}	{{map[1].retry_action}}	{{map[1].escalation}}	{{map[1].a11y}}	{{map[1].platform}}	{{map[1].notes}}

2) Global Rules (required)

- Multiple errors precedence: {{rules.precedence}}
- Field errors handling: {{rules.field_errors}}
- Unknown reason_code fallback: {{rules.unknown_fallback}}
- Correlation ID display policy: {{rules.correlation_display}} | OPTIONAL

3) Overrides (optional)

override_id	screen_id	reason_code	surface_override	copy_override	rationale
ov_01	{{overrides[0].screen_id}}	{{overrides[0].reason_code}}	{{overrides[0].surface}}	{{overrides[0].copy}}	{{overrides[0].rationale}}

4) Coverage Checks (required)

- All reason_codes referenced exist: {{coverage.codes_exist}}
- Minimum mappings met: {{coverage.min_met}}
- A11y behaviors specified: {{coverage.a11y_complete}}

Cross-References

- Upstream: {{xref:ERR-02}} | OPTIONAL, {{xref:DES-07}} | OPTIONAL, {{xref:CDX-04}} | OPTIONAL, {{xref:A11YD-05}} | OPTIONAL
- Downstream: {{xref:FE-07}} | OPTIONAL, {{xref:MOB-*}} | OPTIONAL, {{xref:QA-02}} | OPTIONAL
- Standards: {{standards.rules[STD-A11Y]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Mapping table + unknown fallback.
- intermediate: Required. Add precedence rules and a11y behaviors.
- advanced: Required. Add overrides and coverage checks.

Unknown Handling

- UNKNOWN_ALLOWED: `screen_overrides`, `notes`, `retry_action`, `escalation_action`, `platform_notes`
- If unknown fallback is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
    - required_fields_present == true
    - mappings_count >= 30 (or justified)
    - codes_exist == true
    - a11y_complete == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

ERR-05

ERR-05 — Retryability & Idempotency Rules (which errors retry, how)

Header Block

- template_id: ERR-05
- title: Retryability & Idempotency Rules (which errors retry, how)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-05_Retryability_Idempotency_Rules.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-01", "ERR-02", "WFO-03", "APIG-01"]
- inputs_required: ["ERR-01", "ERR-02", "WFO-03", "APIG-01", "SIC-05", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define deterministic retryability rules across the system: which error classes/reason codes are retryable, under what conditions, what backoff is used, and what idempotency guarantees are required to safely retry without duplicating side effects.

Inputs Required

- ERR-01: {{xref:ERR-01}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- WFO-03: {{xref:WFO-03}} | OPTIONAL

- APIG-01: {{xref:APIG-01}} | OPTIONAL
- SIC-05: {{xref:SIC-05}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Retry policy defaults (for client retries and server retries)
- Retryability rules by error class (default posture)
- Reason-code overrides (explicit list)
- Idempotency requirements:
    - which operations must be idempotent
    - idempotency key format
    - key scope (subject/resource/action)
    - storage/ttl rules (high level)
- Backoff rules:
    - base delay
    - multiplier
    - max delay
    - jitter
    - max attempts
- “Do not retry” list (hard)
- Special handling:
    - rate limiting (Retry-After)

    - conflicts (refresh-and-retry)
    - dependency timeouts (circuit behavior pointer)

Optional Fields

- UI retry behavior pointer (DES-07) | OPTIONAL
- Notes | OPTIONAL

Rules

- Never retry non-idempotent operations unless an idempotency key is enforced.
- Client retries must be bounded; avoid retry storms.
- Server retries (jobs/webhooks) must use DLQ/quarantine after max attempts.
- Retry behavior must be consistent across endpoints; overrides must be explicit.
- Any retryable reason_code must specify whether retry is automatic or user-initiated.

Output Format

1) Defaults (required)

- Client retry default: {{defaults.client}}
- Server retry default: {{defaults.server}}
- Max attempts default: {{defaults.max_attempts}}
- Jitter policy: {{defaults.jitter}} | OPTIONAL

2) Retryability by Error Class (required)

error_class	default_retryable	notes

validation	false	{{classes.validation.notes}}
domain_rule	false	{{classes.domain.notes}}
authz	false	{{classes.authz.notes}}
dependency	{{classes.dependency.retryable}}	{{classes.dependency.notes}}
conflict	{{classes.conflict.retryable}}	{{classes.conflict.notes}}
system_unknown	{{classes.system.retryable}}	{{classes.system.notes}}

3) Reason-Code Overrides (required)

reason_code	retryable	retry_mode (auto/user)	backoff_profile	idempotency_required	notes
{{overrides[0].rc}}	{{overrides[0].retryable}}	{{overrides[0].mode}}	{{overrides[0].backoff}}	{{overrides[0].idempotency}}	{{overrides[0].notes}}

4) Backoff Profiles (required)

profile_id	base_ms	multiplier	max_ms	jitter	max_attempts	notes
backoff_default	{{backoff.default.base}}	{{backoff.default.mult}}	{{backoff.default.max}}	{{backoff.default.jitter}}	{{backoff.default.attempts}}	{{backoff.default.notes}}
backoff_aggressive	{{backoff.aggr.base}}	{{backoff.aggr.mult}}	{{backoff.aggr.max}}	{{backoff.aggr.jitter}}	{{backoff.aggr.attempts}}	{{backoff.aggr.notes}}

5) Idempotency Rules (required)

- Key format: {{idempotency.key_format}} (e.g., idem_<subject><*action*><hash>)
- Key scope: {{idempotency.scope}}
- Storage/TTL rule: {{idempotency.ttl}}
- Collision handling: {{idempotency.collision_handling}} | OPTIONAL

6) Do-Not-Retry List (required)

- {{do_not_retry[0]}}
- {{do_not_retry[1]}} | OPTIONAL

7) Special Cases (required)

- Rate limits: {{special.rate_limits}}
- Conflicts: {{special.conflicts}}
- Dependency timeouts: {{special.dependency_timeouts}}

Cross-References

- Upstream: {{xref:WFO-03}} | OPTIONAL, {{xref:APIG-01}} | OPTIONAL, {{xref:SIC-05}} | OPTIONAL
- Downstream: {{xref:DES-07}} | OPTIONAL, {{xref:WFO-05}} | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Class defaults + do-not-retry list + idempotency key format.
- intermediate: Required. Add backoff profiles and reason-code overrides.
- advanced: Required. Add special cases and explicit retry modes.

Unknown Handling

- UNKNOWN_ALLOWED: `ui_retry_pointer`, `collision_handling`, `notes`, `aggressive_profile`

- If any retryable override lacks idempotency_required definition → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
    - required_fields_present == true
    - class_defaults_present == true
    - backoff_profiles_present == true
    - idempotency_rules_present == true
    - do_not_retry_present == true
    - retryable_overrides_have_idempotency == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

ERR-06 — Observability Requirements for Errors (logs/metrics/traces fields)

Header Block

- template_id: ERR-06
- title: Observability Requirements for Errors (logs/metrics/traces fields)
- type: error_model_reason_codes

- template_version: 1.0.0
- output_path: 10_app/errors/ERR-06_Error_Observability_Requirements.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-01", "OBS-01", "OBS-03"]
- inputs_required: ["ERR-01", "OBS-01", "OBS-02", "OBS-03", "DGP-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the required observability fields and metrics for errors so failures can be diagnosed and tracked consistently across services and boundaries. This covers log field schema, metric dimensions, trace linkage, and redaction requirements.

Inputs Required

- ERR-01: {{xref:ERR-01}} | OPTIONAL
- OBS-01: {{xref:OBS-01}} | OPTIONAL
- OBS-02: {{xref:OBS-02}} | OPTIONAL
- OBS-03: {{xref:OBS-03}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Required log fields for any error event
- Required metric events for errors (counters, rates)

- Required trace linkage fields
- Dimension/tag policy:
    - allowed tags (reason_code, service, endpoint, tenant, severity)
    - high-cardinality ban list (no user_id unless hashed/allowed)
- Redaction rules (PII)
- Sampling/retention policy pointer (if needed)
- Coverage checks (every service emits required fields)

Optional Fields

- Dashboard requirements | OPTIONAL
- Notes | OPTIONAL

Rules

- Error logs must include reason_code + correlation_id at minimum.
- Do not emit high-cardinality identifiers as metric tags.
- PII must be redacted or omitted per DGP.
- Traces must link error occurrences to spans consistently.

Output Format

1) Required Log Fields (required)

field	required	description	redaction_rule
timestamp	true	{{fields.timestamp.desc}}	{{fields.timestamp.redaction}}
service	true	{{fields.service.desc}}	{{fields.service.redaction}}

endpoint_or_op	true	{{fields.endpoint.desc}}	{{fields.endpoint.redaction}}
error_class	true	{{fields.error_class.desc}}	{{fields.error_class.redaction}}
reason_code	true	{{fields.reason_code.desc}}	{{fields.reason_code.redaction}}
severity	true	{{fields.severity.desc}}	{{fields.severity.redaction}}
correlation_id	true	{{fields.correlation.desc}}	{{fields.correlation.redaction}}

2) Error Metrics (required)

metric_name	type	increments_when	tags_allowed	notes
errors_total	counter	{{metrics.errors_total.when}}	{{metrics.errors_total.tags}}	{{metrics.errors_total.notes}}
errors_by_reason	counter	{{metrics.errors_by_reason.when}}	{{metrics.errors_by_reason.tags}}	{{metrics.errors_by_reason.notes}}
p0_error_rate	rate	{{metrics.p0.when}}	{{metrics.p0.tags}}	{{metrics.p0.notes}}

3) Trace Linkage (required)

- Required span attributes: {{tracing.span_attrs}}
- Error span event format: {{tracing.event_format}} | OPTIONAL
- Correlation propagation rule: {{tracing.correlation_rule}}

4) Dimension/Tag Policy (required)

- Allowed tags: {{tags.allowed}}
- Banned high-cardinality tags: {{tags.banned}}
- Hashing rule (if any): {{tags.hashing}} | OPTIONAL

5) Redaction Rules (required)

- Never include: {{redaction.never_include}}
- Allowed if hashed: {{redaction.allowed_hashed}} | OPTIONAL

6) Coverage Checks (required)

- Every service emits required fields: {{coverage.services_complete}}
- Metric tags comply with policy: {{coverage.tags_compliant}}

Cross-References

- Upstream: {{xref:OBS-01}} | OPTIONAL, {{xref:OBS-03}} | OPTIONAL, {{xref:DGP-01}} | OPTIONAL
- Downstream: {{xref:OBS-05}} | OPTIONAL, {{xref:ALRT-*}}* | *OPTIONAL, {{xref:PERF-*}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Log field minimum + metric names.
- intermediate: Required. Add tag policy and trace linkage.
- advanced: Required. Add coverage checks and redaction rigor.

Unknown Handling

- UNKNOWN_ALLOWED: `dashboard_requirements`, `hashing_rule`, `notes`, `retention_pointer`
- If required log fields are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
  - required_fields_present == true
  - log_fields_present == true
  - metrics_present == true
  - tag_policy_present == true
  - coverage_checks_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

ERR-06

ERR-06 — Observability Requirements for Errors (logs/metrics/traces fields)

Header Block

- template_id: ERR-06
- title: Observability Requirements for Errors (logs/metrics/traces fields)
- type: error_model_reason_codes
- template_version: 1.0.0
- output_path: 10_app/errors/ERR-06_Error_Observability_Requirements.md
- compliance_gate_id: TMP-05.PRIMARY.ERRORS
- upstream_dependencies: ["ERR-01", "OBS-01", "OBS-03"]
- inputs_required: ["ERR-01", "OBS-01", "OBS-02", "OBS-03", "DGP-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the required observability fields and metrics for errors so failures can be diagnosed and tracked consistently across services and boundaries. This covers log field schema, metric dimensions, trace linkage, and redaction requirements.

Inputs Required

- ERR-01: {{xref:ERR-01}} | OPTIONAL
- OBS-01: {{xref:OBS-01}} | OPTIONAL
- OBS-02: {{xref:OBS-02}} | OPTIONAL

- OBS-03: {{xref:OBS-03}} | OPTIONAL
- DGP-01: {{xref:DGP-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Required log fields for any error event
- Required metric events for errors (counters, rates)
- Required trace linkage fields
- Dimension/tag policy:
    - allowed tags (reason_code, service, endpoint, tenant, severity)
    - high-cardinality ban list (no user_id unless hashed/allowed)
- Redaction rules (PII)
- Sampling/retention policy pointer (if needed)
- Coverage checks (every service emits required fields)

Optional Fields

- Dashboard requirements | OPTIONAL
- Notes | OPTIONAL

Rules

- Error logs must include reason_code + correlation_id at minimum.
- Do not emit high-cardinality identifiers as metric tags.

- PII must be redacted or omitted per DGP.
- Traces must link error occurrences to spans consistently.

Output Format

1) Required Log Fields (required)

field	required	description	redaction_rule
timestamp	true	{{fields.timestamp.desc}}	{{fields.timestamp.redaction}}
service	true	{{fields.service.desc}}	{{fields.service.redaction}}
endpoint_or_op	true	{{fields.endpoint.desc}}	{{fields.endpoint.redaction}}
error_class	true	{{fields.error_class.desc}}	{{fields.error_class.redaction}}
reason_code	true	{{fields.reason_code.desc}}	{{fields.reason_code.redaction}}
severity	true	{{fields.severity.desc}}	{{fields.severity.redaction}}
correlation_id	true	{{fields.correlation.desc}}	{{fields.correlation.redaction}}

2) Error Metrics (required)

metric_name	type	increments_when	tags_allowed	notes
errors_total	counter	{{metrics.errors_total.when}}	{{metrics.errors_total.tags}}	{{metrics.errors_total.notes}}
errors_by_reason	counter	{{metrics.errors_by_reason.when}}	{{metrics.errors_by_reason.tags}}	{{metrics.errors_by_reason.notes}}
p0_error_rate	rate	{{metrics.p0.when}}	{{metrics.p0.tags}}	{{metrics.p0.notes}}

3) Trace Linkage (required)

- Required span attributes: {{tracing.span_attrs}}
- Error span event format: {{tracing.event_format}} | OPTIONAL

- Correlation propagation rule: {{tracing.correlation_rule}}

4) Dimension/Tag Policy (required)

- Allowed tags: {{tags.allowed}}
- Banned high-cardinality tags: {{tags.banned}}
- Hashing rule (if any): {{tags.hashing}} | OPTIONAL

5) Redaction Rules (required)

- Never include: {{redaction.never_include}}
- Allowed if hashed: {{redaction.allowed_hashed}} | OPTIONAL

6) Coverage Checks (required)

- Every service emits required fields: {{coverage.services_complete}}
- Metric tags comply with policy: {{coverage.tags_compliant}}

Cross-References

- Upstream: {{xref:OBS-01}} | OPTIONAL, {{xref:OBS-03}} | OPTIONAL, {{xref:DGP-01}} | OPTIONAL
- Downstream: {{xref:OBS-05}} | OPTIONAL, {{xref:ALRT-*}}* | *OPTIONAL, {{xref:PERF-}}* | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Log field minimum + metric names.

- intermediate: Required. Add tag policy and trace linkage.
- advanced: Required. Add coverage checks and redaction rigor.

Unknown Handling

- UNKNOWN_ALLOWED: `dashboard_requirements`, `hashing_rule`, `notes`, `retention_pointer`
- If required log fields are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.ERRORS
- Pass conditions:
    - required_fields_present == true
    - log_fields_present == true
    - metrics_present == true
    - tag_policy_present == true
    - coverage_checks_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

Realtime & Messaging Architecture (RTM)

- Realtime & Messaging Architecture (RTM)
  RTM-01 Realtime Use Cases Catalog (presence, chat, streams, updates)
  RTM-02 Protocol & Transport Map (WS/WebRTC/pubsub, fallback rules)
  RTM-03 Channel/Topic Model (naming, scoping, permissions)
  RTM-04 Delivery Semantics (ordering, dedupe, at-least-once, ack)
  RTM-05 Presence & State Sync Model (source of truth, TTLs, conflicts)
  RTM-06 Abuse/Rate Control for Realtime (limits, moderation hooks)

RTM-01

RTM-01 — Realtime Use Cases Catalog (presence, chat, streams, updates)

Header Block

- template_id: RTM-01
- title: Realtime Use Cases Catalog (presence, chat, streams, updates)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-01_Realtime_UseCases_Catalog.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["PRD-04", "DES-01", "ARC-05", "PMAD-01", "RISK-02"]
- inputs_required: ["PRD-04", "DES-01", "ARC-05", "PMAD-01", "ERR-01", "RISK-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical list of realtime use cases and their required semantics so the system doesn't implement "realtime" inconsistently. This sets expectations for presence, chat/messaging, livestream signaling, live updates, and realtime moderation hooks.

Inputs Required

- PRD-04: {{xref:PRD-04}} | OPTIONAL
- DES-01: {{xref:DES-01}} | OPTIONAL
- ARC-05: {{xref:ARC-05}} | OPTIONAL

- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- RISK-02: {{xref:RISK-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Use case list (minimum 5 if realtime exists; otherwise mark N/A)
- For each use case:
    - rt_usecase_id
    - name
    - purpose
    - linked_feature_ids
    - realtime primitive (presence/message/stream_signal/live_state)
    - criticality (P0/P1/P2)
    - delivery semantics required (ordering/ack/retry/dedupe)
    - authorization requirement (who can join/send)
    - data sensitivity (PII/none)
    - abuse/moderation hooks needed (yes/no + type)
    - scaling assumptions (qualitative)
    - fallback behavior (when realtime unavailable)
    - observability requirements (key fields/metrics)
- Coverage check: every realtime feature has at least one use case defined

Optional Fields

- Multi-region notes | OPTIONAL
- Notes | OPTIONAL

Rules

- If applies == false, include 00_NA block only.
- Every use case must specify fallback behavior.
- Every use case must specify authorization requirements (join/send).
- Delivery semantics must be explicit; “best effort” must state what can be dropped.

Output Format

1) Applicability

- applies: {{rtm.applies}} (true/false)
- 00_NA (if not applies): {{rtm.na_block}} | OPTIONAL

2) Realtime Use Cases (canonical)

rt_usecase_id	name	primitive	feature_ids	criticality	semantics	auth_req	sensitivity	moderation_hooks	fallback	obs_requirements	notes
rtu_01	{{usecases[0].name}}	{{usecases[0].primitive}}	{{usecases[0].feature_ids}}	{{usecases[0].criticality}}	{{usecases[0].semantics}}	{{usecases[0].auth}}	{{usecases[0].sensitivity}}	{{usecases[0].moderation}}	{{usecases[0].fallback}}	{{usecases[0].obs}}	{{usecases[0].notes}}

rtu_02	{{usecases[1].name}}	{{usecases[1].primitive}}	{{usecases[1].feature_ids}}	{{usecases[1].criticality}}	{{usecases[1].semantics}}	{{usecases[1].auth}}	{{usecases[1].sensitivity}}	{{usecases[1].moderation}}	{{usecases[1].fallback}}	{{usecases[1].obs}}	{{usecases[1].notes}}

3) Coverage Checks (required if applies)

- All realtime features covered: {{coverage.features_covered}}
- All use cases have fallback: {{coverage.fallback_complete}}
- All use cases have auth requirements: {{coverage.auth_complete}}

Cross-References

- Upstream: {{xref:ARC-05}} | OPTIONAL, {{xref:PMAD-01}} | OPTIONAL, {{xref:ERR-01}} | OPTIONAL
- Downstream: {{xref:RTM-02}}, {{xref:RTM-03}}, {{xref:RTM-04}}, {{xref:RTM-05}}, {{xref:RTM-06}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required if applies. Define use cases + fallback + auth.
- intermediate: Required if applies. Add delivery semantics and observability needs.
- advanced: Required if applies. Add moderation hooks and scaling assumptions.

Unknown Handling

- UNKNOWN_ALLOWED: `multi_region_notes`, `notes`, `scaling_assumptions` (but must be planned)

- If applies == true and any use case lacks fallback → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true
    - if_applies_then_usecases_present == true
    - fallback_complete == true
    - auth_complete == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

RTM-02

RTM-02 — Protocol & Transport Map (WS/WebRTC/pubsub, fallback rules)

Header Block

- template_id: RTM-02
- title: Protocol & Transport Map (WS/WebRTC/pubsub, fallback rules)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-02_Protocol_Transport_Map.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["RTM-01", "ARC-05", "SBDT-02"]
- inputs_required: ["RTM-01", "ARC-05", "SBDT-02", "PMAD-01", "ERR-05", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define which transports/protocols are used for each realtime use case (WS/WebRTC/pubsub/queue), how connections are established, what the fallback rules are, and what constraints apply (auth, ordering, retries, congestion control).

Inputs Required

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- ARC-05: {{xref:ARC-05}} | OPTIONAL
- SBDT-02: {{xref:SBDT-02}} | OPTIONAL

- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Transport list (WS/WebRTC/pubsub/HTTP fallback/queue)
- For each transport:
    - transport_id
    - protocol
    - intended use cases
    - connection model (client→gateway→service)
    - auth model (when and how validated)
    - reliability guarantees (ack, retries, ordering)
    - congestion/backpressure rules
    - fallback behavior if unavailable
    - limits (message size/rate)
- Mapping table: use_case → primary transport → fallback transport
- Reconnect strategy rules (bounded backoff, jitter)
- Failure modes and expected handling (ties to ERR taxonomy)

Optional Fields

- Mobile network constraints notes | OPTIONAL

- Notes | OPTIONAL

Rules

- Every realtime use case must have a defined fallback.
- Auth must be validated server-side on connect/join and on send.
- Backpressure rules must prevent memory blowups.
- Any retry must respect idempotency/dedupe rules (ERR-05, RTM-04).

Output Format

1) Transports (required)

transport_id	protocol	intended_use	auth_model	guarantees	backpressure	limits	fallback	notes
ws_primary	{{transports.ws.use}}	{{transports.ws.auth}}	{{transports.ws.guarantees}}	{{transports.ws.backpressure}}	{{transports.ws.limits}}	{{transports.ws.fallback}}	{{transports.ws.notes}}	OPTIONAL
webrtc	{{transports.webrtc.use}}	{{transports.webrtc.auth}}	{{transports.webrtc.guarantees}}	{{transports.webrtc.backpressure}}	{{transports.webrtc.limits}}	{{transports.webrtc.fallback}}	{{transports.webrtc.notes}}	OPTIONAL

2) Use Case → Transport Mapping (required)

rt_usecase_id	primary_transport	fallback_transport	rationale
{{map[0].usecase}}	{{map[0].primary}}	{{map[0].fallback}}	{{map[0].rationale}}
{{map[1].usecase}}	{{map[1].primary}}	{{map[1].fallback}}	{{map[1].rationale}}

3) Connection & Reconnect Rules (required)

- Connect handshake steps: {{conn.handshake}}
- Reconnect strategy: {{conn.reconnect}}
- Backoff/jitter policy: {{conn.backoff_jitter}}
- Max reconnect window: {{conn.max_window}} | OPTIONAL

4) Failure Modes (required)

failure_type	detection	expected_behavior	error_class	notes
disconnect	{{fail.disconnect.detect}}	{{fail.disconnect.behavior}}	dependency	{{fail.disconnect.notes}}
auth_fail	{{fail.auth.detect}}	{{fail.auth.behavior}}	authz	{{fail.auth.notes}}
overload	{{fail.overload.detect}}	{{fail.overload.behavior}}	dependency	{{fail.overload.notes}}

Cross-References

- Upstream: {{xref:RTM-01}} | OPTIONAL, {{xref:ARC-05}} | OPTIONAL
- Downstream: {{xref:RTM-03}}, {{xref:RTM-04}}, {{xref:RTM-06}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Transport mapping + fallback + reconnect basics.
- intermediate: Required. Add auth model and reliability guarantees.
- advanced: Required. Add backpressure rules, limits, and failure-mode handling.

Unknown Handling

- UNKNOWN_ALLOWED: `mobile_constraints_notes`, `notes`, `max_reconnect_window`
- If any use case lacks fallback transport → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true
    - transports_present == true
    - mapping_present == true
    - reconnect_rules_present == true
    - failure_modes_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

RTM-03

RTM-03 — Channel/Topic Model (naming, scoping, permissions)

Header Block

- template_id: RTM-03
- title: Channel/Topic Model (naming, scoping, permissions)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-03_Channel_Topic_Model.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["RTM-01", "PMAD-01", "PMAD-02", "ARC-05"]
- inputs_required: ["RTM-01", "PMAD-01", "PMAD-02", "ARC-05", "ERR-02", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical model for realtime channels/topics: naming, scoping, membership rules, permission checks for join/publish/subscribe, and how channel identity maps to domain entities (rooms, streams, orgs).

Inputs Required

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- PMAD-01: {{xref:PMAD-01}} | OPTIONAL
- PMAD-02: {{xref:PMAD-02}} | OPTIONAL

- ARC-05: {{xref:ARC-05}} | OPTIONAL
- ERR-02: {{xref:ERR-02}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Channel types list (minimum 5 if realtime exists)
- For each channel type:
    - channel_type_id
    - purpose
    - naming convention (template string)
    - scope key(s) (org_id, room_id, stream_id, user_id)
    - membership model (open/invite/role-gated)
    - join authorization rule (policy_id or predicate)
    - publish authorization rule
    - subscribe authorization rule
    - visibility (discoverable or hidden)
    - retention/replay policy (if any)
    - moderation hooks (if applicable)
- Global naming rules (allowed chars, length, versioning)
- Channel lifecycle rules (create/join/leave/delete/archive)
- Coverage checks: realtime use cases map to at least one channel type

Optional Fields

- Multi-tenant isolation notes | OPTIONAL
- Notes | OPTIONAL

Rules

- Channel names must be deterministic and derived from stable identifiers.
- Server-side enforcement required for join and publish.
- Private channels must be non-enumerable (no listing without access).
- If replay/retention exists, define retention limits and access controls.

Output Format

1) Naming Rules (required)

- Allowed characters: {{naming.allowed_chars}}
- Max length: {{naming.max_length}}
- Format policy: {{naming.format_policy}}
- Versioning policy: {{naming.versioning}} | OPTIONAL

2) Channel Types (canonical)

channel_type_id	purpose	name_template	scope_keys	membership	join_auth	publish_auth	subscribe_auth	visibility	replay_retention	moderation	notes
ch_room	{{types[0].purpose}}	{{types[0].template}}	{{types[0].scope}}	{{types[0].membership}}	{{types[0].join_auth}}	{{types[0].publish_auth}}	{{types[0].subscribe_auth}}	{{types[0].visibility}}	{{types[0].retention}}	{{types[0].moderation}}	{{types[0].notes}}

ch_dm	{{types[1].purpose}}	{{types[1].template}}	{{types[1].scope}}	{{types[1].membership}}	{{types[1].join_auth}}	{{types[1].publish_auth}}	{{types[1].subscribe_auth}}	{{types[1].visibility}}	{{types[1].retention}}	{{types[1].moderation}}	{{types[1].notes}}

3) Channel Lifecycle Rules (required)

- Create rule: {{lifecycle.create}}
- Join rule: {{lifecycle.join}}
- Leave/disconnect rule: {{lifecycle.leave}}
- Archive/delete rule: {{lifecycle.delete_archive}} | OPTIONAL

4) Use Case Coverage (required)

rt_usecase_id	channel_type_ids
{{coverage[0].usecase}}	{{coverage[0].channels}}
{{coverage[1].usecase}}	{{coverage[1].channels}}

Cross-References

- Upstream: {{xref:PMAD-02}} | OPTIONAL, {{xref:RTM-01}} | OPTIONAL
- Downstream: {{xref:RTM-04}}, {{xref:RTM-06}} | OPTIONAL, {{xref:MSG-*}}* | *OPTIONAL, {{xref:OBS-}}* | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-NAMING]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Channel types + naming + basic auth rules.
- intermediate: Required. Add lifecycle rules and visibility/retention policy.

- advanced: Required. Add coverage checks and isolation notes.

Unknown Handling

- UNKNOWN_ALLOWED: `retention_replay`, `moderation_hooks`, `isolation_notes`, `notes`
- If join_auth or publish_auth is UNKNOWN for any channel type → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true
    - channel_types_present == true
    - auth_rules_present == true
    - lifecycle_rules_present == true
    - coverage_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

RTM-04

RTM-04 — Delivery Semantics (ordering, dedupe, at-least-once, ack)

Header Block

- template_id: RTM-04
- title: Delivery Semantics (ordering, dedupe, at-least-once, ack)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-04_Delivery_Semantics.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["RTM-01", "RTM-02", "ERR-05"]
- inputs_required: ["RTM-01", "RTM-02", "ERR-05", "OBS-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the delivery guarantees and semantics for realtime messages/updates: ordering, deduplication, acknowledgments, retries, replay, and how clients reconcile state. This prevents subtle realtime bugs and makes behavior testable.

Inputs Required

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- RTM-02: {{xref:RTM-02}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL

- OBS-01: {{xref:OBS-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Global delivery posture (at-most-once/at-least-once/exactly-once claim policy)
- Ordering rules:
    - per-channel ordering guarantee (yes/no)
    - cross-channel ordering (no guarantee unless stated)
- Dedupe rules:
    - message_id generation (server/client)
    - dedupe window/ttl
    - idempotency handling for updates
- Ack rules:
    - when server acks
    - client ack requirements (if any)
    - retry triggers
- Replay/backfill rules:
    - when history is replayed
    - retention window policy
    - permission constraints on replay
- Client reconciliation rules:
    - optimistic updates handling

    - conflict resolution
    - resync triggers
- Test requirements (contract tests)

Optional Fields

- Exactly-once disclaimer | OPTIONAL
- Notes | OPTIONAL

Rules

- Do not claim exactly-once unless proven; default to at-least-once with dedupe.
- Ordering must be explicit; if not guaranteed, clients must tolerate reordering.
- Dedupe must be deterministic and scoped (channel + sender + message_id).
- Replay must enforce auth checks (no history leaks).

Output Format

1) Delivery Posture (required)

- Posture: {{delivery.posture}} (at-most-once / at-least-once)
- Exactly-once policy: {{delivery.exactly_once_policy}} | OPTIONAL

2) Ordering Rules (required)

- Per-channel ordering: {{ordering.per_channel}}
- Cross-channel ordering: {{ordering.cross_channel}}
- Ordering exceptions: {{ordering.exceptions}} | OPTIONAL

3) Message Identity & Dedupe (required)

- message_id source: {{dedupe.message_id_source}} (server/client/both)
- message_id format: {{dedupe.message_id_format}}
- dedupe window: {{dedupe.window}}
- dedupe key scope: {{dedupe.scope}}

4) Acknowledgments & Retries (required)

- Server ack condition: {{ack.server_ack_condition}}
- Client ack requirement: {{ack.client_ack}} | OPTIONAL
- Retry trigger rules: {{ack.retry_triggers}}
- Retry backoff profile: {{ack.backoff_profile}} | OPTIONAL

5) Replay / History (required if used)

- Retention window: {{replay.retention}}
- Who can replay: {{replay.auth_rules}}
- Backfill trigger rules: {{replay.backfill_triggers}} | OPTIONAL

6) Client Reconciliation (required)

- Optimistic update rule: {{client.optimistic}}
- Conflict resolution rule: {{client.conflict_resolution}}
- Resync triggers: {{client.resync_triggers}}

7) Test Requirements (required)

- Contract tests: {{tests.contract}}

- Dedupe tests: {{tests.dedupe}} | OPTIONAL
- Ordering tolerance tests: {{tests.ordering}} | OPTIONAL

Cross-References

- Upstream: {{xref:RTM-02}} | OPTIONAL, {{xref:ERR-05}} | OPTIONAL
- Downstream: {{xref:RTM-05}}, {{xref:RTM-06}} | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Posture + ordering + dedupe basics.
- intermediate: Required. Add ack/retry rules and reconciliation rules.
- advanced: Required. Add replay policy and test requirements.

Unknown Handling

- UNKNOWN_ALLOWED: `exactly_once_policy`, `ordering_exceptions`, `client_ack`, `notes`
- If delivery posture or dedupe scope is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true

- posture_defined == true
- ordering_defined == true
- dedupe_defined == true
- ack_retry_defined == true
- reconciliation_defined == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

RTM-05

RTM-05 — Presence & State Sync Model (source of truth, TTLs, conflicts)

Header Block

- template_id: RTM-05
- title: Presence & State Sync Model (source of truth, TTLs, conflicts)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-05_Presence_State_Sync_Model.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["RTM-01", "RTM-02", "RTM-03", "ARC-05"]
- inputs_required: ["RTM-01", "RTM-02", "RTM-03", "ARC-05", "ERR-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how presence and realtime state are represented, synchronized, expired, and reconciled across clients and the server: source of truth, TTLs, heartbeats, conflict resolution, and resync triggers.

Inputs Required

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- RTM-02: {{xref:RTM-02}} | OPTIONAL
- RTM-03: {{xref:RTM-03}} | OPTIONAL

- ARC-05: {{xref:ARC-05}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Presence entity definition (fields + meaning)
- Presence states:
    - online
    - offline
    - away (optional)
    - busy (optional)
- Source of truth (server/client/hybrid) and rationale
- Heartbeat model:
    - heartbeat interval
    - TTL expiry window
    - reconnect behavior
- State sync model:
    - authoritative state vs derived state
    - resync triggers (on connect, on gap, on conflict)
    - delta vs snapshot strategy
- Conflict resolution rules:
    - multiple devices

    - stale updates
    - out-of-order delivery
- Privacy rules:
    - who can see presence
    - opt-out rules
    - “invisible” mode (if any)
- Observability requirements (metrics/fields)

Optional Fields

- Cross-device priority rules | OPTIONAL
- Notes | OPTIONAL

Rules

- Presence must expire automatically if no heartbeats; no “stuck online.”
- State sync must tolerate disconnects and reordering.
- Privacy is mandatory: presence visibility must be controlled by policy.
- If hybrid source-of-truth, define deterministic tie-breaker rules.

Output Format

1) Presence Entity (required)

field	type	meaning	notes
user_id	{{presence.fields.user_id.type}}	{{presence.fields.user_id.meaning}}	{{presence.fields.user_id.notes}}

status	{{presence.fields.status.type}}	{{presence.fields.status.meaning}}	{{presence.fields.status.notes}}
last_seen_at	{{presence.fields.last_seen.type}}	{{presence.fields.last_seen.meaning}}	{{presence.fields.last_seen.notes}}
device_id	{{presence.fields.device_id.type}}	{{presence.fields.device_id.meaning}}	{{presence.fields.device_id.notes}}

2) Presence States (required)

- States: {{presence.states}}
- Transition rules: {{presence.transitions}}
- Derived states (if any): {{presence.derived_states}} | OPTIONAL

3) Source of Truth (required)

- Source: {{truth.source}} (server/client/hybrid)
- Rationale: {{truth.rationale}}
- Tie-breaker rule (if hybrid): {{truth.tiebreaker}} | OPTIONAL

4) Heartbeat & TTL (required)

- Heartbeat interval: {{heartbeat.interval}}
- TTL window: {{heartbeat.ttl}}
- Expiry behavior: {{heartbeat.expiry_behavior}}
- Reconnect behavior: {{heartbeat.reconnect_behavior}}

5) State Sync Strategy (required)

- Snapshot vs delta: {{sync.strategy}}
- Resync triggers: {{sync.resync_triggers}}

- Gap detection: {{sync.gap_detection}} | OPTIONAL

6) Conflict Resolution (required)

- Multi-device rule: {{conflict.multi_device}}
- Out-of-order handling: {{conflict.out_of_order}}
- Stale updates: {{conflict.stale_updates}}

7) Privacy Rules (required)

- Visibility policy: {{privacy.visibility_policy}}
- Opt-out/invisible mode: {{privacy.invisible_mode}} | OPTIONAL
- Audience constraints: {{privacy.audience_constraints}}

8) Observability Requirements (required)

- Metrics: {{obs.metrics}}
- Required fields in logs: {{obs.log_fields}} | OPTIONAL

Cross-References

- Upstream: {{xref:RTM-02}} | OPTIONAL, {{xref:RTM-03}} | OPTIONAL, {{xref:ARC-05}} | OPTIONAL
- Downstream: {{xref:RTS-*}} | *OPTIONAL, {{xref:OBS-}}* | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-PRIVACY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Entity + states + heartbeat/TTL.
- intermediate: Required. Add source-of-truth and sync strategy.
- advanced: Required. Add conflict resolution and privacy rules.

Unknown Handling

- UNKNOWN_ALLOWED: `invisible_mode`, `cross_device_priority`, `notes`, `gap_detection`
- If source of truth or TTL policy is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true
    - presence_entity_present == true
    - ttl_defined == true
    - sync_strategy_defined == true
    - conflict_rules_present == true
    - privacy_rules_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

RTM-06

RTM-06 — Abuse/Rate Control for Realtime (limits, moderation hooks)

Header Block

- template_id: RTM-06
- title: Abuse/Rate Control for Realtime (limits, moderation hooks)
- type: realtime_messaging_architecture
- template_version: 1.0.0
- output_path: 10_app/realtime/RTM-06_Abuse_Rate_Control.md
- compliance_gate_id: TMP-05.PRIMARY.REALTIME
- upstream_dependencies: ["RTM-01", "RTM-03", "RLIM-01", "TNS-01"]
- inputs_required: ["RTM-01", "RTM-03", "RLIM-01", "TNS-01", "PMAD-05", "OBS-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the realtime-specific abuse controls and rate limits: per-connection and per-channel limits, spam/flood detection, moderation hooks, enforcement actions, and how realtime abuse integrates with trust & safety workflows.

Inputs Required

- RTM-01: {{xref:RTM-01}} | OPTIONAL
- RTM-03: {{xref:RTM-03}} | OPTIONAL
- RLIM-01: {{xref:RLIM-01}} | OPTIONAL

- TNS-01: {{xref:TNS-01}} | OPTIONAL
- PMAD-05: {{xref:PMAD-05}} | OPTIONAL
- OBS-04: {{xref:OBS-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Rate limit catalog (minimum 10 realtime-specific limits)
- For each limit:
    - limit_id
    - applies_to (connection/user/channel/ip)
    - scope key (channel_id/user_id/ip)
    - limit value (msgs/sec, joins/min, bytes/sec)
    - enforcement point (gateway/realtime service)
    - action on breach (throttle/drop/disconnect/ban)
    - escalation rule (temporary mute, cooldown, report)
    - observability signals (counters/alerts)
- Abuse detection heuristics (high level):
    - flood/spam patterns
    - repeated join/leave
    - malformed payload abuse
- Moderation hooks:
    - message filtering/hold-for-review

    - user mute/ban actions
    - channel lockdown mode
- Appeal/recovery rules pointer (TNS)
- Test requirements (load + abuse simulations)

Optional Fields

- ML-based spam scoring pointer | OPTIONAL
- Notes | OPTIONAL

Rules

- Limits must be enforceable server-side with deterministic behavior.
- Enforcement actions must be consistent and auditable.
- Moderation actions must map to privileged operations policy (PMAD-05) if admin-driven.
- Controls must minimize false positives; include escalation and recovery.

Output Format

1) Rate Limits (canonical)

limit_id	applies_to	scope_key	limit_value	enforcement_point	breach_action	escalation	obs_signals	notes
rtm_lim_01	{{limits[0].applies_to}}	{{limits[0].scope}}	{{limits[0].value}}	{{limits[0].enforcement}}	{{limits[0].breach}}	{{limits[0].escalation}}	{{limits[0].obs}}	{{limits[0].notes}}
rtm_lim_02	{{limits[1].applies_to}}	{{limits[1].scope}}	{{limits[1].value}}	{{limits[1].enforcement}}	{{limits[1].breach}}	{{limits[1].escalation}}	{{limits[1].obs}}	{{limits[1].notes}}

2) Abuse Heuristics (required)

- Flood/spam patterns: {{abuse.flood_patterns}}
- Join/leave churn: {{abuse.churn_patterns}} | OPTIONAL
- Malformed payload abuse: {{abuse.malformed}} | OPTIONAL

3) Moderation Hooks (required)

hook_id	trigger	action	scope	audit_event	notes
hook_filter_01	{{hooks[0].trigger}}	{{hooks[0].action}}	{{hooks[0].scope}}	{{hooks[0].audit}}	{{hooks[0].notes}}

4) Appeals/Recovery Pointer (required)

- Trust & Safety policy pointer: {{xref:TNS-05}} | OPTIONAL
- Support escalation pointer: {{xref:SUP-02}} | OPTIONAL

5) Test Requirements (required)

- Load test cases: {{tests.load}}
- Abuse simulation cases: {{tests.abuse}}
- Rate limit enforcement tests: {{tests.enforcement}} | OPTIONAL

Cross-References

- Upstream: {{xref:RLIM-01}} | OPTIONAL, {{xref:TNS-01}} | OPTIONAL, {{xref:PMAD-05}} | OPTIONAL
- Downstream: {{xref:OBS-04}} | OPTIONAL, {{xref:ALRT-*}} | OPTIONAL, {{xref:QA-04}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Rate limits + breach actions + moderation hook list.
- intermediate: Required. Add heuristics and observability signals.
- advanced: Required. Add test requirements and appeals pointers.

Unknown Handling

- UNKNOWN_ALLOWED: `ml_spam_scoring`, `notes`, `churn_patterns`, `malformed_payload_patterns`
- If any limit lacks enforcement_point or breach_action → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.REALTIME
- Pass conditions:
    - required_fields_present == true
    - limits_count >= 10
    - enforcement_defined == true
    - moderation_hooks_present == true
    - test_requirements_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

Workflow & Orchestration Design (WFO)

- Workflow & Orchestration Design (WFO)
  WFO-01 Workflow Catalog (sagas/jobs/background tasks by ID)
  WFO-02 Orchestration Patterns (state machine, retries, compensation)
  WFO-03 Idempotency & Concurrency Model (keys, locking, ordering)
  WFO-04 Scheduling & Triggers (cron/events/user actions)
  WFO-05 Failure Handling (DLQ, backoff, poison messages, alerts)
  WFO-06 Workflow Observability (traceability, audit events, metrics)

WFO-01

WFO-01 — Workflow Catalog (sagas/jobs/background tasks by ID)

Header Block

- template_id: WFO-01
- title: Workflow Catalog (sagas/jobs/background tasks by ID)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-01_Workflow_Catalog.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["PRD-04", "ARC-03", "ARC-05", "API-06"]
- inputs_required: ["PRD-04", "ARC-03", "ARC-05", "API-06", "ERR-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical registry of workflows (sagas, background jobs, scheduled tasks) by stable IDs so orchestration is deterministic and traceable. This is the authoritative list of “things that happen over time” in the system.

Inputs Required

- PRD-04: {{xref:PRD-04}} | OPTIONAL
- ARC-03: {{xref:ARC-03}} | OPTIONAL
- ARC-05: {{xref:ARC-05}} | OPTIONAL

- API-06: {{xref:API-06}} | OPTIONAL
- ERR-01: {{xref:ERR-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Workflow list (minimum 10 for non-trivial systems; justify if smaller)
- For each workflow:
    - wf_id
    - name
    - type (saga/job/scheduled)
    - trigger (event/cron/user_action/api)
    - purpose
    - linked_feature_ids
    - owner_service_id
    - inputs (events/commands/data)
    - outputs (events/side effects/data writes)
    - statefulness (stateless/stateful)
    - criticality (P0/P1/P2)
    - expected runtime class (ms/sec/min/hr)
    - concurrency posture (single/parallel/partitioned)
    - failure handling posture (retry/DLQ/manual)
    - observability requirements (trace/log fields)

- auth context (system/user) (if relevant)

Optional Fields

- State machine pointer (WFO-02) | OPTIONAL
- Notes | OPTIONAL

Rules

- wf_id must be stable and never reused for a different workflow.
- Every workflow must declare triggers and failure posture.
- Any user-impacting workflow must map to an error posture (ERR taxonomy) and UX fallback pointer (DES/CDX).
- Concurrency posture must align with idempotency/concurrency rules (WFO-03).

Output Format

1) Workflow Registry (canonical)

wf_id	name	type	trigger	feature_ids	owner_service	inputs	outputs	stateful	criticality	runtime_class	concurrency	failure_posture	obs	notes
wf_001	{{workflows[0].name}}	{{workflows[0].type}}	{{workflows[0].trigger}}	{{workflows[0].feature_ids}}	{{workflows[0].owner}}	{{workflows[0].inputs}}	{{workflows[0].outputs}}	{{workflows[0].stateful}}	{{workflows[0].criticality}}	{{workflows[0].runtime}}	{{workflows[0].concurrency}}	{{workflows[0].failure}}	{{workflows[0].obs}}	{{workflows[0].notes}}
wf	{{workf	{{workf	{{workflo	{{workflow	{{workfl	{{workfl	{{workflo	{{workflo	{{workflo	{{workflo	{{workflow	{{workflo	{{work	{{workf

_002	lows[1].name}}	lows[1].type}}	ws[1].trigger}}	s[1].feature_ids}}	ows[1].owner}}	ows[1].inputs}}	ws[1].outputs}}	ws[1].stateful}}	ws[1].criticality}}	ws[1].runtime}}	s[1].concurrency}}	ws[1].failure}}	flows[1].obs}}	lows[1].notes}}

2) Coverage Checks (required)

- Every workflow has trigger: {{coverage.triggers_complete}}
- Every workflow has failure posture: {{coverage.failure_complete}}
- wf_ids unique: {{coverage.unique_ids}}

Cross-References

- Upstream: {{xref:ARC-03}} | OPTIONAL, {{xref:API-06}} | OPTIONAL, {{xref:ERR-01}} | OPTIONAL
- Downstream: {{xref:WFO-02}}, {{xref:WFO-03}}, {{xref:WFO-05}} | OPTIONAL, {{xref:OBS-*}} | OPTIONAL
- Standards: {{standards.rules[STD-NAMING]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Registry with trigger + owner + failure posture.
- intermediate: Required. Add concurrency posture and runtime class.
- advanced: Required. Add observability and auth context.

Unknown Handling

- UNKNOWN_ALLOWED: `state_machine_pointer`, `notes`, `runtime_class` (if to be measured)

- If any workflow lacks trigger or owner_service_id → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS
- Pass conditions:
    - required_fields_present == true
    - workflows_count >= 10 (or justified)
    - triggers_complete == true
    - failure_complete == true
    - unique_ids == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

WFO-02

WFO-02 — Orchestration Patterns (state machine, retries, compensation)

Header Block

- template_id: WFO-02
- title: Orchestration Patterns (state machine, retries, compensation)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-02_Orchestration_Patterns.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["WFO-01", "ERR-05", "RELIA-02"]
- inputs_required: ["WFO-01", "ERR-05", "RELIA-02", "OBS-03", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the canonical orchestration patterns used across workflows: how state machines are modeled, how retries and backoff are applied, how compensation works, and how workflows transition between states under failure.

Inputs Required

- WFO-01: {{xref:WFO-01}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL
- RELIA-02: {{xref:RELIA-02}} | OPTIONAL

- OBS-03: {{xref:OBS-03}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Pattern catalog (minimum 6 patterns)
- For each pattern:
    - pattern_id
    - name
    - when to use
    - when not to use
    - state model (states + transitions)
    - retry rules (which steps retry)
    - compensation rules (how to undo/repair)
    - timeout rules
    - observability hooks (what to trace/measure)
- Global workflow state definitions (queued/running/waiting/succeeded/failed/cancelled)
- Standard retry/backoff policy pointers
- “Stuck workflow” detection rules (timeouts, watchdog)

Optional Fields

- Diagrams/pointers | OPTIONAL
- Notes | OPTIONAL

Rules

- Retries must be bounded and follow ERR-05 profiles.
- Compensation must be explicit for multi-step workflows with side effects.
- State transitions must be deterministic; no ambiguous “maybe” states.
- Stuck detection must be measurable and alertable.

Output Format

1) Global Workflow States (required)

state	meaning	entry_conditions	exit_conditions
queued	{{states.queued.meaning}}	{{states.queued.entry}}	{{states.queued.exit}}
running	{{states.running.meaning}}	{{states.running.entry}}	{{states.running.exit}}
waiting	{{states.waiting.meaning}}	{{states.waiting.entry}}	{{states.waiting.exit}}
succeeded	{{states.succeeded.meaning}}	{{states.succeeded.entry}}	{{states.succeeded.exit}}
failed	{{states.failed.meaning}}	{{states.failed.entry}}	{{states.failed.exit}}
cancelled	{{states.cancelled.meaning}}	{{states.cancelled.entry}}	{{states.cancelled.exit}}

2) Patterns Catalog (canonical)

pattern_id	name	use_when	avoid_when	retry_rule	compensation_rule	timeout_rule	obs_hooks	notes
pat_saga	{{patterns[0].name}}	{{patterns[0].use_when}}	{{patterns[0].avoid_when}}	{{patterns[0].retry}}	{{patterns[0].comp}}	{{patterns[0].timeout}}	{{patterns[0].obs}}	{{patterns[0].notes}}
pat_job	{{patterns[1].name}}	{{patterns[1].use_when}}	{{patterns[1].avoid_when}}	{{patterns[1].retry}}	{{patterns[1].comp}}	{{patterns[1].timeout}}	{{patterns[1].obs}}	{{patterns[1].notes}}

3) Stuck Workflow Detection (required)

- Watchdog rules: {{stuck.watchdog_rules}}
- Timeout thresholds: {{stuck.timeouts}}
- Escalation actions: {{stuck.escalation}} | OPTIONAL

Cross-References

- Upstream: {{xref:ERR-05}} | OPTIONAL, {{xref:RELIA-02}} | OPTIONAL
- Downstream: {{xref:WFO-05}} | OPTIONAL, {{xref:OPS-06}} | OPTIONAL, {{xref:IRP-*}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Global state definitions + 6 patterns (names + when to use).
- intermediate: Required. Add retry/timeout rules and stuck detection.
- advanced: Required. Add compensation and observability hooks per pattern.

Unknown Handling

- UNKNOWN_ALLOWED: `diagrams`, `notes`, `escalation_actions`
- If retry rules are UNKNOWN for any pattern → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS

- Pass conditions:
    - required_fields_present == true
    - patterns_count >= 6
    - global_states_present == true
    - stuck_detection_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

WFO-03

WFO-03 — Idempotency & Concurrency Model (keys, locking, ordering)

Header Block

- template_id: WFO-03
- title: Idempotency & Concurrency Model (keys, locking, ordering)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-03_Idempotency_Concurrency_Model.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["WFO-01", "ERR-05", "DATA-03", "APIG-01"]
- inputs_required: ["WFO-01", "ERR-05", "DATA-03", "APIG-01", "RTM-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the system's idempotency and concurrency rules so retries, parallelism, and multi-device interactions do not create duplicate side effects or inconsistent state. This document defines keys, locking approaches, ordering rules, and conflict-handling policies.

Inputs Required

- WFO-01: {{xref:WFO-01}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL
- DATA-03: {{xref:DATA-03}} | OPTIONAL

- APIG-01: {{xref:APIG-01}} | OPTIONAL
- RTM-04: {{xref:RTM-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Idempotency policy:
    - which operations require idempotency (API writes, webhooks, jobs)
    - idempotency key format rules
    - key scope rules (subject/resource/action + payload hash policy)
    - TTL/retention for keys
    - storage approach (DB table/cache) (high level)
    - collision handling rule
- Concurrency policy:
    - optimistic vs pessimistic approach (where used)
    - versioning fields (etag/version) policy
    - conflict detection and resolution rules
    - write ordering rules (per entity/per workflow)
- Locking/serialization strategies (if any):
    - per-entity lock
    - per-user lock
    - per-workflow lock
- Realtime concurrency notes (multi-device updates, out-of-order)

- Test requirements (idempotency + race tests)

Optional Fields

- Implementation pattern pointers | OPTIONAL
- Notes | OPTIONAL

Rules

- Any retryable operation must have idempotency semantics defined (ties to ERR-05).
- Locking must be minimal and avoid system-wide locks.
- Conflict resolution must be deterministic and user-visible policy must exist (ERR/DES pointers where needed).
- Ordering guarantees must match RTM-04 if realtime updates are involved.

Output Format

1) Idempotency Rules (required)

- Required operations: {{idem.required_operations}}
- Key format: {{idem.key_format}}
- Key scope: {{idem.scope}}
- TTL/retention: {{idem.ttl}}
- Storage approach: {{idem.storage}} | OPTIONAL
- Collision handling: {{idem.collision_handling}}

2) Idempotency Key Derivation (required)

operation_type	key_components	includes_payload_hash	notes
api_write	{{derive.api_write.components}}	{{derive.api_write.payload_hash}}	{{derive.api_write.notes}}
webhook_inbound	{{derive.webhook.components}}	{{derive.webhook.payload_hash}}	{{derive.webhook.notes}}
job_step	{{derive.job.components}}	{{derive.job.payload_hash}}	{{derive.job.notes}}

3) Concurrency Policy (required)

- Default stance: {{concurrency.default_stance}} (optimistic/pessimistic/hybrid)
- Version field policy: {{concurrency.version_field_policy}}
- Conflict detection: {{concurrency.detect}}
- Conflict resolution: {{concurrency.resolve}}
- User-facing behavior pointer: {{concurrency.user_behavior_pointer}} | OPTIONAL

4) Locking / Serialization (required if used)

lock_type	scope	when_used	timeout	notes
per_entity	{{locks.per_entity.scope}}	{{locks.per_entity.when}}	{{locks.per_entity.timeout}}	{{locks.per_entity.notes}}
per_user	{{locks.per_user.scope}}	{{locks.per_user.when}}	{{locks.per_user.timeout}}	{{locks.per_user.notes}}

5) Ordering Rules (required)

- Per-entity ordering: {{ordering.per_entity}}
- Per-workflow ordering: {{ordering.per_workflow}} | OPTIONAL
- Out-of-order handling rule: {{ordering.out_of_order}}

6) Realtime Concurrency Notes (required if realtime)

- Multi-device update policy: {{rt.multi_device_policy}} | OPTIONAL
- Last-write-wins policy (if used): {{rt.lww_policy}} | OPTIONAL

7) Test Requirements (required)

- Idempotency tests: {{tests.idempotency}}
- Concurrency/race tests: {{tests.races}}
- Replay/dedupe tests (if queues): {{tests.dedupe}} | OPTIONAL

Cross-References

- Upstream: {{xref:ERR-05}} | OPTIONAL, {{xref:DATA-03}} | OPTIONAL, {{xref:RTM-04}} | OPTIONAL
- Downstream: {{xref:WFO-05}} | OPTIONAL, {{xref:API-05}} | OPTIONAL, {{xref:SIC-03}} | OPTIONAL, {{xref:QA-03}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Idempotency operations + key format + conflict policy basics.
- intermediate: Required. Add key derivation table and ordering rules.
- advanced: Required. Add locking strategy and race test requirements.

Unknown Handling

- UNKNOWN_ALLOWED: `storage_approach`, `implementation_patterns`, `notes`, `locks` (if not used), `realtime_notes` (if no realtime)

- If collision handling or conflict resolution is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS
- Pass conditions:
    - required_fields_present == true
    - idempotency_rules_present == true
    - concurrency_policy_present == true
    - ordering_rules_present == true
    - test_requirements_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

WFO-04

WFO-04 — Scheduling & Triggers (cron/events/user actions)

Header Block

- template_id: WFO-04
- title: Scheduling & Triggers (cron/events/user actions)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-04_Scheduling_Triggers.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["WFO-01", "EVT-01", "API-06"]
- inputs_required: ["WFO-01", "EVT-01", "API-06", "ENV-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how workflows are triggered and scheduled: cron schedules, event-driven triggers, user/action triggers, and how trigger configuration differs by environment. This makes workflow activation deterministic and auditable.

Inputs Required

- WFO-01: {{xref:WFO-01}} | OPTIONAL
- EVT-01: {{xref:EVT-01}} | OPTIONAL
- API-06: {{xref:API-06}} | OPTIONAL

- ENV-01: {{xref:ENV-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Trigger types supported (cron/event/user_action/api)
- Trigger catalog (minimum 10 triggers)
- For each trigger:
    - trigger_id
    - wf_id
    - type
    - definition (cron string / event name / route/action)
    - environment enablement (dev/stage/prod)
    - dedupe/idempotency rule (pointer)
    - throttling rule (if needed)
    - failure behavior on trigger (if trigger dispatch fails)
    - observability signals
- Change control policy (who can change schedules)
- Safety rules (prevent accidental high-frequency cron)

Optional Fields

- Timezone policy | OPTIONAL
- Notes | OPTIONAL

Rules

- Every trigger must reference a wf_id in WFO-01.
- Cron schedules must be explicit and validated; no “every minute” without justification.
- Environment enablement must be explicit (default off in dev for risky jobs unless stated).
- Trigger dispatch must be idempotent and observable.

Output Format

1) Trigger Catalog (canonical)

trigger_id	wf_id	type	definition	env_enablement	dedupe_rule_ref	throttle	dispatch_failure_behavior	obs_signals	notes
trg_001	{{triggers[0].wf_id}}	{{triggers[0].type}}	{{triggers[0].definition}}	{{triggers[0].envs}}	{{triggers[0].dedupe_ref}}	{{triggers[0].throttle}}	{{triggers[0].dispatch_fail}}	{{triggers[0].obs}}	{{triggers[0].notes}}
trg_002	{{triggers[1].wf_id}}	{{triggers[1].type}}	{{triggers[1].definition}}	{{triggers[1].envs}}	{{triggers[1].dedupe_ref}}	{{triggers[1].throttle}}	{{triggers[1].dispatch_fail}}	{{triggers[1].obs}}	{{triggers[1].notes}}

2) Change Control Policy (required)

- Who can change schedules: {{change_control.who}}
- Approval required: {{change_control.approval}} | OPTIONAL
- Logging/audit requirement: {{change_control.audit}}

3) Safety Rules (required)

- High-frequency cron prevention: {{safety.high_frequency}}
- Burst trigger throttling: {{safety.burst_throttle}} | OPTIONAL

- Env default enablement stance: {{safety.env_default}}

Cross-References

- Upstream: {{xref:WFO-01}} | OPTIONAL, {{xref:EVT-01}} | OPTIONAL, {{xref:ENV-01}} | OPTIONAL
- Downstream: {{xref:WFO-05}} | OPTIONAL, {{xref:OPS-06}} | OPTIONAL, {{xref:OBS-04}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Trigger table with wf_id mapping and env enablement.
- intermediate: Required. Add change control and safety rules.
- advanced: Required. Add throttle policies and dispatch failure behavior.

Unknown Handling

- UNKNOWN_ALLOWED: `timezone_policy`, `notes`, `throttle`, `approval_required`
- If any trigger references unknown wf_id → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS
- Pass conditions:
    - required_fields_present == true

- triggers_count >= 10
- all_triggers_reference_wf_ids == true
- safety_rules_present == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

WFO-05

WFO-05 — Failure Handling (DLQ, backoff, poison messages, alerts)

Header Block

- template_id: WFO-05
- title: Failure Handling (DLQ, backoff, poison messages, alerts)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-05_Failure_Handling.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["WFO-01", "WFO-02", "ERR-05", "RELIA-01"]
- inputs_required: ["WFO-01", "WFO-02", "ERR-05", "RELIA-01", "OBS-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define deterministic failure handling for workflows: retry/backoff profiles, DLQ/quarantine rules, poison message detection, alerting/escalation, and safe re-drive procedures. This prevents infinite retries, hidden failures, and unsafe manual fixes.

Inputs Required

- WFO-01: {{xref:WFO-01}} | OPTIONAL
- WFO-02: {{xref:WFO-02}} | OPTIONAL
- ERR-05: {{xref:ERR-05}} | OPTIONAL

- RELIA-01: {{xref:RELIA-01}} | OPTIONAL
- OBS-04: {{xref:OBS-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Failure mode catalog for workflows (minimum 12)
- For each failure mode:
    - fail_id
    - wf_id or workflow_type
    - failure_type (timeout/validation/dependency/poison/duplicate)
    - classification (transient/permanent/unknown)
    - retry behavior (profile + max attempts)
    - DLQ/quarantine rule
    - poison detection rule (if applicable)
    - alerting severity + trigger
    - manual intervention procedure pointer (runbook)
    - reason_code mapping (if surfaced)
- Default backoff profiles (reuse ERR-05 where possible)
- DLQ structure:
    - required fields
    - retention
    - re-drive safety checks

- Alerting policy (paging vs ticket)
- Postmortem requirement conditions

Optional Fields

- Auto-remediation rules | OPTIONAL
- Notes | OPTIONAL

Rules

- Transient failures retry with bounded backoff.
- Permanent failures never retry; they quarantine/DLQ.
- Poison messages must be detected and quarantined early.
- Re-drive must be safe: require idempotency keys and validation checks.
- Alerting must be deterministic and tied to thresholds.

Output Format

1) Failure Modes (canonical)

fail_id	wf_id	failure_type	class	retry_profile	max_attempts	dlq_rule	poison_rule	alert_trigger	severity	runbook_ptr	reason_code	notes
wf_fail_01	{{fails[0].wf_id}}	{{fails[0].type}}	{{fails[0].class}}	{{fails[0].retry}}	{{fails[0].max}}	{{fails[0].dlq}}	{{fails[0].poison}}	{{fails[0].alert}}	{{fails[0].severity}}	{{fails[0].runbook}}	{{fails[0].rc}}	{{fails[0].notes}}

wf_fail_02	{{fails[1].wf_id}}	{{fails[1].type}}	{{fails[1].class}}	{{fails[1].retry}}	{{fails[1].max}}	{{fails[1].dlq}}	{{fails[1].poison}}	{{fails[1].alert}}	{{fails[1].severity}}	{{fails[1].runbook}}	{{fails[1].rc}}	{{fails[1].notes}}

2) Backoff Profiles (required)

- Default profile pointer (ERR-05): {{backoff.default_ref}} | OPTIONAL
- Workflow-specific overrides: {{backoff.overrides}} | OPTIONAL

3) DLQ / Quarantine Structure (required)

- Required fields: {{dlq.required_fields}}
- Retention window: {{dlq.retention}}
- Re-drive allowed when: {{dlq.redrive_conditions}}
- Re-drive safety checks: {{dlq.safety_checks}}
- Re-drive auditing: {{dlq.audit}} | OPTIONAL

4) Alerting Policy (required)

- Paging conditions: {{alerts.paging_conditions}}
- Ticketing conditions: {{alerts.ticket_conditions}} | OPTIONAL
- Deduping alerts: {{alerts.dedupe}} | OPTIONAL

5) Postmortem Rules (required)

- Postmortem required when: {{postmortem.when_required}}
- Template pointer: {{postmortem.template_ptr}} | OPTIONAL

Cross-References

- Upstream: {{xref:ERR-05}} | OPTIONAL, {{xref:RELIA-01}} | OPTIONAL
- Downstream: {{xref:OPS-06}} | OPTIONAL, {{xref:IRP-01}} | OPTIONAL, {{xref:RELIA-05}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Failure catalog + DLQ rules + basic alerting.
- intermediate: Required. Add poison detection and re-drive safety.
- advanced: Required. Add postmortem and auto-remediation rules.

Unknown Handling

- UNKNOWN_ALLOWED: `auto_remediation`, `notes`, `ticket_conditions`, `runbook_ptr`
- If DLQ safety checks are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS
- Pass conditions:
    - required_fields_present == true
    - failure_modes_count >= 12
    - dlq_rules_present == true
    - alerting_policy_present == true
    - placeholder_resolution == true

- no_unapproved_unknowns == true

WFO-06

WFO-06 — Workflow Observability (traceability, audit events, metrics)

Header Block

- template_id: WFO-06
- title: Workflow Observability (traceability, audit events, metrics)
- type: workflow_orchestration_design
- template_version: 1.0.0
- output_path: 10_app/workflows/WFO-06_Workflow_Observability.md
- compliance_gate_id: TMP-05.PRIMARY.WORKFLOWS
- upstream_dependencies: ["WFO-01", "OBS-01", "OBS-03"]
- inputs_required: ["WFO-01", "OBS-01", "OBS-03", "AUDIT-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the required observability signals for workflows: traceability fields, audit events, metrics, and dashboards so workflow health can be monitored and diagnosed consistently across services.

Inputs Required

- WFO-01: {{xref:WFO-01}} | OPTIONAL
- OBS-01: {{xref:OBS-01}} | OPTIONAL
- OBS-03: {{xref:OBS-03}} | OPTIONAL

- AUDIT-01: {{xref:AUDIT-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Required trace fields:
    - wf_id, run_id, step_id
    - correlation_id/trace_id
    - entity identifiers (redacted rules)
- Required audit events (for sensitive workflows)
- Metrics catalog:
    - runs_started, runs_succeeded, runs_failed
    - step_duration, queue_depth, retry_count
    - dlq_count
- Tag policy (avoid high-cardinality labels)
- Alert thresholds (what triggers alerts)
- Coverage checks (every workflow emits core signals)

Optional Fields

- Dashboard requirements | OPTIONAL
- Notes | OPTIONAL

Rules

- All workflows must emit wf_id + run_id + correlation_id at minimum.
- Metrics must avoid high-cardinality tags (no user_id as metric label).
- Sensitive workflows must emit audit events (aligned with PMAD-05/PMAD-06).
- Alerts must be deterministic and tied to thresholds.

Output Format

1) Required Trace Fields (required)

field	required	description	redaction
wf_id	true	{{trace.wf_id}}	{{trace.redaction.wf_id}}
run_id	true	{{trace.run_id}}	{{trace.redaction.run_id}}
step_id	true	{{trace.step_id}}	{{trace.redaction.step_id}}
correlation_id	true	{{trace.correlation_id}}	{{trace.redaction.correlation_id}}

2) Workflow Metrics (required)

metric	type	increments_when	tags_allowed	notes
workflow_runs_started	counter	{{metrics.started.when}}	{{metrics.started.tags}}	{{metrics.started.notes}}
workflow_runs_failed	counter	{{metrics.failed.when}}	{{metrics.failed.tags}}	{{metrics.failed.notes}}
workflow_step_duration_ms	histogram	{{metrics.step_dur.when}}	{{metrics.step_dur.tags}}	{{metrics.step_dur.notes}}
workflow_dlq_count	counter	{{metrics.dlq.when}}	{{metrics.dlq.tags}}	{{metrics.dlq.notes}}

3) Audit Events (required if sensitive workflows exist)

- Required audit events: {{audit.events}}
- When emitted: {{audit.when}}

- Redaction rules: {{audit.redaction}}

4) Alert Thresholds (required)

alert_id	trigger_metric	threshold	window	severity	action
wf_alert_01	{{alerts[0].metric}}	{{alerts[0].threshold}}	{{alerts[0].window}}	{{alerts[0].severity}}	{{alerts[0].action}}

5) Coverage Checks (required)

- Every wf_id emits core metrics: {{coverage.metrics_complete}}
- Every wf_id emits trace fields: {{coverage.trace_complete}}
- Tags comply with policy: {{coverage.tags_compliant}}

Cross-References

- Upstream: {{xref:OBS-01}} | OPTIONAL, {{xref:OBS-03}} | OPTIONAL, {{xref:AUDIT-01}} | OPTIONAL
- Downstream: {{xref:ALRT-*}} | *OPTIONAL, {{xref:OPS-05}}* | *OPTIONAL, {{xref:IRP-*}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Core trace fields + basic metrics.
- intermediate: Required. Add tag policy and alert thresholds.
- advanced: Required. Add audit event mapping and coverage checks.

Unknown Handling

- UNKNOWN_ALLOWED: `dashboard_requirements`, `notes`, `alert_actions`
- If coverage checks are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.WORKFLOWS
- Pass conditions:
  - required_fields_present == true
  - trace_fields_present == true
  - metrics_present == true
  - alert_thresholds_present == true
  - coverage_checks_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

API Governance & Versioning (APIG)

- API Governance & Versioning (APIG)
  APIG-01 API Standards (naming, pagination, filtering, consistency)
  APIG-02 Versioning Policy (v1/v2 rules, compat guarantees)
  APIG-03 Deprecation & Sunset Policy (timelines, comms, redirects)
  APIG-04 Review Gate Checklist (what must be true before shipping APIs)
  APIG-05 Compatibility Test Requirements (contract tests, schema checks)
  APIG-06 Schema Evolution Rules (backward compatible changes, migrations)

APIG-01

APIG-01 — API Standards (naming, pagination, filtering, consistency)

Header Block

- template_id: APIG-01
- title: API Standards (naming, pagination, filtering, consistency)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-01_API_Standards.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["DMG-01", "ERR-03", "APIG-02"]
- inputs_required: ["DMG-01", "ERR-03", "APIG-02", "PFS-01", "RLIM-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the enforceable API standards for all endpoints: naming conventions, resource modeling, request/response consistency, pagination/filtering/sorting rules, envelope patterns, error handling shape, and compatibility expectations.

Inputs Required

- DMG-01: {{xref:DMG-01}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL
- APIG-02: {{xref:APIG-02}} | OPTIONAL

- PFS-01: {{xref:PFS-01}} | OPTIONAL
- RLIM-01: {{xref:RLIM-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Naming conventions:
    - endpoint paths
    - resource naming (singular/plural)
    - query parameter naming
    - header naming
- Request/response standards:
    - envelope policy (yes/no)
    - consistent field naming (snake/camel)
    - timestamps format (ISO)
    - id fields naming
- Pagination standard:
    - cursor vs offset
    - required params
    - response fields
- Filtering/sorting standard:
    - allowed operators
    - how to express sort

- Error contract compliance rule (ERR-03)
- Idempotency and retries policy pointer (ERR-05)
- Rate limit standard pointer (RLIM)
- Security standards (auth headers, scopes) pointer
- Compatibility statement (old clients)

Optional Fields

- GraphQL standards | OPTIONAL
- Notes | OPTIONAL

Rules

- Every endpoint must comply with naming and pagination standards if list returns.
- Error payload must comply with ERR-03.
- Filtering/sorting must be deterministic and documented.
- No breaking changes without versioning policy compliance (APIG-02/APIG-03).

Output Format

1) Naming Conventions (required)

- Path style: {{naming.paths}}
- Resource nouns: {{naming.resources}}
- Query params: {{naming.query_params}}
- Headers: {{naming.headers}} | OPTIONAL

2) Request/Response Standards (required)

- Envelope policy: {{standards.envelope}}
- Field casing: {{standards.casing}}
- Timestamp format: {{standards.timestamp}}
- ID fields: {{standards.id_fields}}

3) Pagination Standard (required)

- Pagination type: {{pagination.type}} (cursor/offset)
- Request params: {{pagination.request_params}}
- Response fields: {{pagination.response_fields}}
- Limits policy: {{pagination.limits_policy}} | OPTIONAL

4) Filtering/Sorting Standard (required)

- Filter operators: {{filters.operators}}
- Filter expression format: {{filters.format}}
- Sort format: {{sort.format}}
- Default sort: {{sort.default}} | OPTIONAL

5) Error + Retry + Rate Limit Standards (required)

- Error contract: {{xref:ERR-03}} | OPTIONAL
- Retry/idempotency policy: {{xref:ERR-05}} | OPTIONAL
- Rate limit standard: {{xref:RLIM-01}} | OPTIONAL

6) Compatibility Statement (required)

- Backward compatibility rule: {{compat.backward}}
- Deprecation rule pointer: {{xref:APIG-03}} | OPTIONAL

Cross-References

- Upstream: {{xref:DMG-01}} | OPTIONAL, {{xref:ERR-03}} | OPTIONAL
- Downstream: {{xref:API-02}} | OPTIONAL, {{xref:APIG-04}} | OPTIONAL, {{xref:APIG-05}} | OPTIONAL
- Standards: {{standards.rules[STD-NAMING]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Naming + basic request/response + error contract pointer.
- intermediate: Required. Add pagination and filtering/sorting.
- advanced: Required. Add compatibility and deprecation constraints.

Unknown Handling

- UNKNOWN_ALLOWED: `graphql_standards`, `headers`, `envelope_policy` (if decided later), `notes`
- If pagination type is UNKNOWN for list endpoints → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
    - required_fields_present == true

- naming_conventions_present == true
- pagination_standard_present == true
- filtering_sorting_present == true
- error_contract_pointer_present == true
- placeholder_resolution == true
- no_unapproved_unknowns == true

APIG-02

APIG-02 — Versioning Policy (v1/v2 rules, compat guarantees)

Header Block

- template_id: APIG-02
- title: Versioning Policy (v1/v2 rules, compat guarantees)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-02_Versioning_Policy.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["APIG-01", "ARC-02"]
- inputs_required: ["APIG-01", "ARC-02", "STK-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the API versioning and compatibility guarantees: what constitutes a breaking change, how versions are expressed, how clients migrate, and what rules govern forward/backward compatibility.

Inputs Required

- APIG-01: {{xref:APIG-01}} | OPTIONAL
- ARC-02: {{xref:ARC-02}} | OPTIONAL
- STK-04: {{xref:STK-04}} | OPTIONAL

- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Versioning scheme:
    - path-based (/v1)
    - header-based
    - query-based (discouraged)
- Compatibility guarantees (what is safe to change)
- Breaking change definition list
- Allowed non-breaking changes list
- Client migration policy:
    - how clients discover versions
    - how long old versions supported
- Version ownership + approval (who can bump versions)
- Version sunset/deprecation pointer (APIG-03)

Optional Fields

- Mobile app compatibility notes | OPTIONAL
- Notes | OPTIONAL

Rules

- No breaking changes in-place on a stable version.

- Version bump requires compatibility tests (APIG-05) and review gate (APIG-04).
- Old versions must have explicit sunset timelines.
- Any version scheme must be deterministic and documented.

Output Format

1) Versioning Scheme (required)

- Primary scheme: {{version.scheme}}
- Secondary scheme (if any): {{version.secondary}} | OPTIONAL
- Where version is expressed: {{version.location}}

2) Breaking Changes (required)

- {{breaking[0]}}
- {{breaking[1]}}
- {{breaking[2]}} | OPTIONAL

3) Non-Breaking Changes (required)

- {{nonbreaking[0]}}
- {{nonbreaking[1]}}
- {{nonbreaking[2]}} | OPTIONAL

4) Compatibility Guarantees (required)

- Backward compatibility promise: {{compat.backward_promise}}
- Forward compatibility promise: {{compat.forward_promise}} | OPTIONAL
- Schema evolution stance: {{compat.schema_evolution}} | OPTIONAL

5) Client Migration Policy (required)

- Discovery method: {{migration.discovery}}
- Support window: {{migration.support_window}}
- Upgrade guidance: {{migration.guidance}} | OPTIONAL

6) Ownership + Approval (required)

- Owner: {{ownership.owner}}
- Approval gate: {{ownership.approval_gate}} | OPTIONAL
- Decision log pointer: {{xref:STK-04}} | OPTIONAL

Cross-References

- Upstream: {{xref:APIG-01}} | OPTIONAL
- Downstream: {{xref:APIG-03}}, {{xref:APIG-04}}, {{xref:APIG-05}}, {{xref:APIG-06}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Scheme + breaking definition + support window.
- intermediate: Required. Add compatibility guarantees and migration policy.
- advanced: Required. Add ownership/approval and schema evolution stance.

Unknown Handling

- UNKNOWN_ALLOWED: `mobile_notes`, `secondary_scheme`, `notes`

- If breaking change definition is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
    - required_fields_present == true
    - versioning_scheme_present == true
    - breaking_changes_present == true
    - migration_policy_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

APIG-03

APIG-03 — Deprecation & Sunset Policy (timelines, comms, redirects)

Header Block

- template_id: APIG-03
- title: Deprecation & Sunset Policy (timelines, comms, redirects)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-03_Deprecation_Sunset_Policy.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["APIG-02", "REL-02", "RELOPS-01"]
- inputs_required: ["APIG-02", "REL-02", "RELOPS-01", "SUP-03", "STK-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define how APIs are deprecated and sunset: timelines, communication requirements, redirect/compatibility strategies, deprecation headers, and enforcement rules. This prevents surprise breakage and makes migration predictable.

Inputs Required

- APIG-02: {{xref:APIG-02}} | OPTIONAL
- REL-02: {{xref:REL-02}} | OPTIONAL
- RELOPS-01: {{xref:RELOPS-01}} | OPTIONAL

- SUP-03: {{xref:SUP-03}} | OPTIONAL
- STK-04: {{xref:STK-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Deprecation stages (announce → warn → restrict → sunset)
- Timeline policy (minimum lead time per stage)
- Communication channels (docs/release notes/in-app emails if relevant)
- Technical mechanisms:
    - deprecation headers
    - warning logs/metrics
    - compatibility shims (if allowed)
    - redirects (if applicable)
- Client detection and reporting:
    - how to identify usage of deprecated endpoints/versions
    - reporting cadence
- Approval requirements for sunset (who signs off)
- Exceptions policy

Optional Fields

- Customer-specific contracts (enterprise SLAs) | OPTIONAL
- Notes | OPTIONAL

Rules

- Sunsetting requires prior announcement and measurable usage monitoring.
- Breaking clients must have a migration path documented.
- Deprecation must be tracked in changelog/version notes (REL).
- Exceptions must be time-bound.

Output Format

1) Deprecation Stages (required)

stage	meaning	minimum_duration	signals	enforcement
announce	{{stages.announce.meaning}}	{{stages.announce.duration}}	{{stages.announce.signals}}	{{stages.announce.enforcement}}
warn	{{stages.warn.meaning}}	{{stages.warn.duration}}	{{stages.warn.signals}}	{{stages.warn.enforcement}}
restrict	{{stages.restrict.meaning}}	{{stages.restrict.duration}}	{{stages.restrict.signals}}	{{stages.restrict.enforcement}}
sunset	{{stages.sunset.meaning}}	{{stages.sunset.duration}}	{{stages.sunset.signals}}	{{stages.sunset.enforcement}}

2) Timeline Policy (required)

- Minimum lead time for announce: {{timeline.announce_lead}}
- Warn duration: {{timeline.warn_duration}}
- Sunset minimum total window: {{timeline.total_min_window}}

3) Communication Requirements (required)

- Required channels: {{comms.channels}}

- Required artifacts (release notes, docs): {{comms.artifacts}}
- Support macros pointer: {{xref:SUP-03}} | OPTIONAL

4) Technical Mechanisms (required)

- Deprecation headers: {{tech.headers}}
- Warning metrics/logs: {{tech.metrics_logs}}
- Compatibility shims: {{tech.shims}} | OPTIONAL
- Redirect rules: {{tech.redirects}} | OPTIONAL

5) Usage Monitoring & Reporting (required)

- Usage detection method: {{usage.detection}}
- Reporting cadence: {{usage.cadence}}
- Thresholds to proceed to sunset: {{usage.thresholds}} | OPTIONAL

6) Approval & Exceptions (required)

- Sunset approval required by: {{approval.required_by}}
- Decision log pointer: {{xref:STK-04}} | OPTIONAL
- Exceptions allowed when: {{exceptions.when}}
- Time-bound exception rule: {{exceptions.time_bound}}

Cross-References

- Upstream: {{xref:APIG-02}} | OPTIONAL, {{xref:REL-02}} | OPTIONAL
- Downstream: {{xref:APIG-04}}, {{xref:APIG-05}} | OPTIONAL, {{xref:RELOPS-04}} | OPTIONAL

- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Stages + timeline + communication requirements.
- intermediate: Required. Add technical mechanisms and usage reporting.
- advanced: Required. Add thresholds, approvals, and exception governance.

Unknown Handling

- UNKNOWN_ALLOWED: `enterprise_contracts`, `notes`, `redirect_rules`, `shims`
- If timeline policy is UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
    - required_fields_present == true
    - stages_present == true
    - timeline_present == true
    - comms_present == true
    - monitoring_present == true
    - approvals_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

APIG-04

APIG-04 — Review Gate Checklist (what must be true before shipping APIs)

Header Block

- template_id: APIG-04
- title: Review Gate Checklist (what must be true before shipping APIs)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-04_Review_Gate_Checklist.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["APIG-01", "ERR-03", "PMAD-03", "QA-05"]
- inputs_required: ["APIG-01", "ERR-03", "PMAD-03", "QA-05", "SEC-02", "RLIM-01", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the mandatory checklist that must pass before an API (endpoint group or version) can ship. This is the quality/gov gate that enforces consistency, security, compatibility, and test coverage.

Inputs Required

- APIG-01: {{xref:APIG-01}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL
- PMAD-03: {{xref:PMAD-03}} | OPTIONAL

- QA-05: {{xref:QA-05}} | OPTIONAL
- SEC-02: {{xref:SEC-02}} | OPTIONAL
- RLIM-01: {{xref:RLIM-01}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Checklist categories:
    - spec completeness
    - security/authz
    - error contract
    - pagination/filtering
    - rate limits
    - observability
    - testing
    - compatibility/versioning
- Checklist items (minimum 35)
- For each item:
    - check_id
    - statement
    - pass condition (objective)
    - evidence required (link/pointer)
- Failure handling (what happens if a check fails)

- Waiver/exception policy (if allowed)

Optional Fields

- Review roles (who reviews) | OPTIONAL
- Notes | OPTIONAL

Rules

- Checklist items must be testable/verifiable.
- Exceptions (if allowed) must be time-bound and recorded.
- No shipping if authz enforcement is missing.
- Compatibility checks required for stable APIs.

Output Format

1) Checklist (required, min 35)

- {{checks[0].statement}} — evidence: {{checks[0].evidence}} | OPTIONAL
- {{checks[1].statement}} — evidence: {{checks[1].evidence}} | OPTIONAL
- {{checks[2].statement}} — evidence: {{checks[2].evidence}} | OPTIONAL
- {{checks[3].statement}} — evidence: {{checks[3].evidence}} | OPTIONAL
- {{checks[4].statement}} — evidence: {{checks[4].evidence}} | OPTIONAL

2) Failure Handling (required)

- If any check fails: {{failure.if_fail}}
- Escalation path: {{failure.escalation}} | OPTIONAL

3) Waiver Policy (required)

- Waivers allowed: {{waiver.allowed}}
- Required fields for waiver: {{waiver.fields}}
- Approval required: {{waiver.approval}}
- Time-bound rule: {{waiver.time_bound}}

Cross-References

- Upstream: {{xref:APIG-01}} | OPTIONAL, {{xref:PMAD-03}} | OPTIONAL, {{xref:ERR-03}} | OPTIONAL
- Downstream: {{xref:APIG-05}}, {{xref:REL-01}} | OPTIONAL, {{xref:RELOPS-05}} | OPTIONAL
- Standards: {{standards.rules[STD-SECURITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Checklist with core categories (authz, errors, tests).
- intermediate: Required. Add pass conditions and evidence requirements.
- advanced: Required. Add waiver governance and escalation paths.

Unknown Handling

- UNKNOWN_ALLOWED: `review_roles`, `notes`, `escalation_path`
- If waiver policy allows unlimited waivers → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
    - required_fields_present == true
    - checklist_count >= 35
    - waiver_policy_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true

APIG-05

APIG-05 — Compatibility Test Requirements (contract tests, schema checks)

Header Block

- template_id: APIG-05
- title: Compatibility Test Requirements (contract tests, schema checks)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-05_Compatibility_Test_Requirements.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["APIG-01", "APIG-02", "API-02", "ERR-03"]
- inputs_required: ["APIG-01", "APIG-02", "API-02", "ERR-03", "TINF-01", "CICD-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": false, "intermediate": true, "advanced": true}

Purpose

Define the minimum compatibility/contract test suite required to ship and evolve APIs without breaking clients: schema checks, contract tests, backward-compat enforcement, and CI requirements.

Inputs Required

- APIG-01: {{xref:APIG-01}} | OPTIONAL

- APIG-02: {{xref:APIG-02}} | OPTIONAL
- API-02: {{xref:API-02}} | OPTIONAL
- ERR-03: {{xref:ERR-03}} | OPTIONAL
- TINF-01: {{xref:TINF-01}} | OPTIONAL
- CICD-04: {{xref:CICD-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Applicability (true/false). If false, mark N/A.
- Contract test categories:
    - schema compatibility checks (OpenAPI/JSON schema)
    - error contract checks (ERR-03)
    - authz contract checks (PMAD enforcement)
    - pagination/filtering contract checks
    - versioning compatibility checks (APIG-02)
- Required CI execution rules (when and where run)
- Failure policy (block merges/releases)
- Test artifact requirements (reports, diffs)

Optional Fields

- Golden files strategy | OPTIONAL
- Notes | OPTIONAL

Rules

- If applies == false, include 00_NA block only.
- Compatibility tests must run in CI for protected branches.
- Failing compatibility tests must block release.
- Schema diffs must be reviewed and approved when breaking.

Output Format

1) Applicability

- applies: {{compat_tests.applies}} (true/false)
- 00_NA (if not applies): {{compat_tests.na_block}} | OPTIONAL

2) Required Test Categories (required if applies)

category	required	description	tooling_hint	notes
schema_compat	true	{{tests.schema.desc}}	{{tests.schema.tooling}}	{{tests.schema.notes}}
error_contract	true	{{tests.error.desc}}	{{tests.error.tooling}}	{{tests.error.notes}}
authz_contract	true	{{tests.authz.desc}}	{{tests.authz.tooling}}	{{tests.authz.notes}}
pagination_filtering	true	{{tests.pfs.desc}}	{{tests.pfs.tooling}}	{{tests.pfs.notes}}
versioning	true	{{tests.version.desc}}	{{tests.version.tooling}}	{{tests.version.notes}}

3) CI Execution Rules (required if applies)

- When run: {{ci.when}}

- Required branches: {{ci.branches}}
- Required environments: {{ci.envs}} | OPTIONAL
- Artifacts produced: {{ci.artifacts}}

4) Failure Policy (required if applies)

- Block merge on failure: {{policy.block_merge}}
- Block release on failure: {{policy.block_release}}
- Exception/waiver allowed: {{policy.waiver_allowed}} | OPTIONAL

5) Artifact Requirements (required if applies)

- Schema diff report: {{artifacts.schema_diff}}
- Contract test report: {{artifacts.test_report}}
- Evidence pointers stored: {{artifacts.evidence_store}} | OPTIONAL

Cross-References

- Upstream: {{xref:APIG-02}} | OPTIONAL, {{xref:API-02}} | OPTIONAL
- Downstream: {{xref:APIG-06}} | OPTIONAL, {{xref:QA-03}} | OPTIONAL, {{xref:RELOPS-05}} | OPTIONAL
- Standards: {{standards.rules[STD-RELIABILITY]}} | OPTIONAL, {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Not required.
- intermediate: Required if applies. Categories + CI rules + failure policy.

- advanced: Required if applies. Add artifact requirements and waiver controls.

Unknown Handling

- UNKNOWN_ALLOWED: `golden_files`, `notes`, `waiver_allowed`, `envs`
- If applies == true and CI rules are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
  - required_fields_present == true
  - if_applies_then_categories_present == true
  - if_applies_then_ci_rules_present == true
  - failure_policy_present == true
  - placeholder_resolution == true
  - no_unapproved_unknowns == true

APIG-06

APIG-06 — Schema Evolution Rules (backward compatible changes, migrations)

Header Block

- template_id: APIG-06
- title: Schema Evolution Rules (backward compatible changes, migrations)
- type: api_governance_versioning
- template_version: 1.0.0
- output_path: 10_app/api_governance/APIG-06_Schema_Evolution_Rules.md
- compliance_gate_id: TMP-05.PRIMARY.APIG
- upstream_dependencies: ["APIG-02", "DATA-04", "ERR-03"]
- inputs_required: ["APIG-02", "DATA-04", "ERR-03", "STK-04", "STANDARDS_INDEX"]
- required_by_skill_level: {"beginner": true, "intermediate": true, "advanced": true}

Purpose

Define the rules for evolving schemas safely (API request/response schemas and shared models): what changes are backward compatible, how to stage migrations, how to deprecate fields, and how to avoid breaking clients during mixed-version deployments.

Inputs Required

- APIG-02: {{xref:APIG-02}} | OPTIONAL
- DATA-04: {{xref:DATA-04}} | OPTIONAL

- ERR-03: {{xref:ERR-03}} | OPTIONAL
- STK-04: {{xref:STK-04}} | OPTIONAL
- STANDARDS_INDEX: {{standards.index}} | OPTIONAL

Required Fields

- Compatible change list (allowed)
- Breaking change list (requires new version)
- Field evolution rules:
    - add optional field policy
    - add required field policy (usually breaking)
    - rename field policy
    - delete field policy
    - change type policy
- Default values policy
- Deprecation annotations policy (how to mark deprecated fields)
- Migration staging rules (expand/contract alignment to DATA-04)
- Mixed-version compatibility statement (old+new)
- Approval requirements for breaking changes

Optional Fields

- GraphQL schema evolution notes | OPTIONAL
- Notes | OPTIONAL

Rules

- Deleting or changing types is breaking unless versioned.
- New required fields are breaking unless defaulting rules are explicit and safe.
- Renames require compatibility windows (support both old and new).
- Mixed-version deployments must be supported during rollout.

Output Format

1) Backward-Compatible Changes (required)

- {{compatible[0]}}
- {{compatible[1]}}
- {{compatible[2]}} | OPTIONAL

2) Breaking Changes (required)

- {{breaking[0]}}
- {{breaking[1]}}
- {{breaking[2]}} | OPTIONAL

3) Field Evolution Rules (required)

change_type	allowed_in_place	required_process	notes
add_optional_field	{{rules.add_optional.allowed}}	{{rules.add_optional.process}}	{{rules.add_optional.notes}}
add_required_field	false	{{rules.add_required.process}}	{{rules.add_required.notes}}
rename_field	{{rules.rename.allowed}}	{{rules.rename.process}}	{{rules.rename.notes}}

delete_field	false	{{rules.delete.process}}	{{rules.delete.notes}}
change_type	false	{{rules.change_type.process}}	{{rules.change_type.notes}}

4) Default Values Policy (required)

- Client defaulting: {{defaults.client}}
- Server defaulting: {{defaults.server}}
- Backfill policy pointer: {{xref:DATA-04}} | OPTIONAL

5) Deprecation Annotations (required)

- How to mark deprecated: {{deprecations.how}}
- Required timeline: {{deprecations.timeline}} | OPTIONAL

6) Migration Staging Rules (required)

- Expand/contract stance: {{staging.expand_contract}}
- Mixed-version support: {{staging.mixed_version_support}}
- Verification requirement: {{staging.verification}} | OPTIONAL

7) Approval Requirements (required)

- Breaking change approval: {{approval.breaking}}
- Decision log pointer: {{xref:STK-04}} | OPTIONAL

Cross-References

- Upstream: {{xref:DATA-04}} | OPTIONAL, {{xref:APIG-02}} | OPTIONAL

- Downstream: {{xref:REL-04}} | OPTIONAL, {{xref:QA-05}} | OPTIONAL, {{xref:APIG-05}} | OPTIONAL
- Standards: {{standards.rules[STD-UNKNOWN-HANDLING]}} | OPTIONAL

Skill Level Requiredness Rules

- beginner: Required. Compatible vs breaking list + field evolution table.
- intermediate: Required. Add staging rules and default value policy.
- advanced: Required. Add verification requirements and approval governance.

Unknown Handling

- UNKNOWN_ALLOWED: `graphql_notes`, `notes`, `verification_requirement`, `timeline`
- If rename/delete/type-change policies are UNKNOWN → block Completeness Gate.

Completeness Gate

- Gate ID: TMP-05.PRIMARY.APIG
- Pass conditions:
    - required_fields_present == true
    - field_evolution_rules_present == true
    - staging_rules_present == true
    - approval_requirements_present == true
    - placeholder_resolution == true
    - no_unapproved_unknowns == true